INSTRUCTOR KIT · MYSQL FOUNDATIONS · COMPLETE WORKBOOK

MySQL Foundations

Your working notebook for all 14 sessions of MySQL Foundations. Read the key ideas, study the worked examples, do the guided labs, predict before you run, and check yourself with the self-check quizzes.

14 sessions · 25 lessons | Write-in exercises & self-checks | Name: _____

CONTENTS

PART 1: SQL FOUNDATIONS · SESSIONS 1 TO 7

S01 **What Databases Are and Installing MySQL**

- 1.1 What Is a Database?
- 1.2 Installing MySQL

S02 **The MySQL Client and Your First Tables**

- 1.3 MySQL CLI & phpMyAdmin
- 2.1 Creating Databases & Tables

S03 **Data Types, Constraints and CRUD**

- 2.2 Data Types & Constraints
- 2.3 CRUD Operations

S04 **Filtering, Sorting and Limiting**

- 3.1 WHERE Clauses, Operators & Filtering
- 3.2 Sorting & Limiting Results

S05 **Aggregates and Subqueries**

- 3.3 Aggregate Functions & GROUP BY
- 3.4 Subqueries & Nested Queries

S06 **Relationships and Foreign Keys**

- 4.1 Table Relationships
- 4.2 Foreign Keys & Referential Integrity

S07 **Joins and the Part 1 Assessment**

- 4.3 Joins

PART 2: BEYOND THE BASICS · SESSIONS 8 TO 14

S08 CTEs and Window Functions

- 5.1 CTEs & Recursive Queries
- 5.2 Window Functions

S09 Indexes and Views

- 6.1 Indexes & Query Optimization
- 6.2 Views

S10 Normalization and Stored Programs

- 6.3 Database Design & Normalization
- 7.1 Stored Procedures & Functions

S11 Transactions and JSON

- 7.2 Transactions
- 7.3 JSON Columns

S12 Users, Permissions and Backups

- 7.4 User Management & Permissions
- 8.1 Importing & Exporting Data

S13 Final Project Kickoff and Studio I

- 8.2 Final Project: Design and Build a Database

S14 Final Project Studio II, Showcase and Next Steps

- 8.2 Final Project: Design and Build a Database
- 8.3 Next Steps & Best Practices

MYSQL FOUNDATIONS · PARTICIPANT WORKBOOK

Part 1: SQL Foundations

Sessions 1 to 7. Modules 1 to 4: install MySQL, build and change tables, query, summarize and join. Runs on its own as a short course and ends with the Part 1 assessment.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 1: What Databases Are and Installing MySQL

Your workbook for Session 1: the environment check, a diagnostic, the key ideas of Lessons 1.1 and 1.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 1.1–1.2 | Session 1 · 3 hours | Name: _____

SESSION GOALS

- Find out whether your computer is ready: the environment check ends with READY, and you know what to do if it does not.
- Explain what a database, a DBMS and SQL are, and why a database beats a spreadsheet once data grows.
- Read a table: its columns, rows, primary key and foreign keys, and how keys connect tables.
- Run your first SQL: create, read, update and delete rows, and read the numbered errors when MySQL refuses bad data.
- Have MySQL 9.7 running on your laptop, log in, check which server answered, and keep a health check you can rerun.

START HERE: IS YOUR COMPUTER READY?

ENVIRONMENT CHECK

- ☐ I can open a terminal in my course folder (the one with `labs/` in it)
- ☐ macOS/Linux: `bash labs/module-01/l1_0-orientation/check_environment.sh` · Windows: `powershell -ExecutionPolicy Bypass -File labs\module-01\l1_0-orientation\check_environment.ps1`
- ☐ The last table says `READY for MySQL Foundations` (or `READY WITH NOTES`). My server version is: _____
- ☐ If it said NOT READY, I wrote my plan here (from the install clinic) and will finish it before Session 2:

Three error numbers to know by heart: **command not found** = the client is not on your PATH · **1045** = the server answered and refused your user name or password · **2002 / 2003** = nothing answered: the server is stopped, or you are knocking at the wrong address.

DIAGNOSTIC: PREDICT, THEN CHECK

Six questions on the spreadsheet and computer skills SQL builds on. Circle your answer before the class reveals it. Nobody collects this. Questions D1 to D5 are about this small sheet of orders:

ORDER_NO	CUSTOMER	CITY	ITEM	QTY	PRICE
1001	Alice Johnson	New York	Dune	1	14.99
1002	Bob Smith	Chicago	1984	2	9.99
1003	Grace Wilson	Chicago	The Hobbit	1	10.99
1004	Dave Brown	Houston	Dune	3	14.99
1005	Frank Miller	New York	Fahrenheit 451	2	10.49
1006	Eve Davis	(empty)	Pride and Prejudice	1	7.99

Missed one? The skill and where it comes back: **D1** filtering rows (WHERE in Lesson 3.1 (and the lesson's first SELECT)) · **D2** sorting (ORDER BY in Lesson 3.2) · **D3** AND versus OR (WHERE with AND/OR in Lesson 3.1) · **D4** a formula column (expressions in SELECT (Lesson 1.1's taste, then 3.3)) · **D5** empty cells (NULL in Lessons 2.2 and 3.3) · **D6** files and folders (the Setup Guide and Lesson 1.3 (running .sql files)). For practice at home: `labs/module-01/l1_0-orientation/diagnostic_starter.sql` does D1 to D5 in SQL, one small gap each.

D1

You filter the city column to show only Chicago. How many rows are left?

A) 1

B) 2

C) 3

D) 6

D2

You sort by price, largest first. Which item is on the top row?

A) Pride and Prejudice

B) The Hobbit

C) Dune

D) 1984

D3

How many rows have the city New York AND a qty of 2 or more?

A) 1

B) 2

C) 4

D) 5

D4

You add a column total = qty × price. What is the total for order 1004?

A) 14.99

B) 17.99

C) 29.98

D) 44.97

D5

=COUNTA(city) counts the filled-in cells of the city column. What does it give?

A) 4

B) 5

C) 6

D) 0

D6

You saved bookstore.sql in your Downloads folder. Your terminal is open in your home folder. Which path names the file?

A) `bookstore.sql`

B) `Downloads/bookstore.sql`

C) `/bookstore.sql`

D) `home/Downloads.sql`

LESSONS

1.1 What Is a Database?

BY THE END YOU CAN

- Explain what a database and a database management system (DBMS) are, and name three problems a database solves that a spreadsheet does not
- Tell relational databases apart from the main non-relational kinds, and pick a sensible type for a simple scenario
- Point to the table, columns, rows, primary key and foreign key in a small example
- Describe what SQL is and match the four CRUD operations to INSERT, SELECT, UPDATE and DELETE
- Sketch the tables of a small online bookstore on paper, with keys and relationships

KEY IDEAS

- **A database is organized data; a DBMS is the server that guards it.** MySQL is a DBMS. Your application never touches the data files: it sends SQL to the server.
- **Why not a spreadsheet?** Copies of the same fact drift apart, nothing enforces types, searches get slow, two editors overwrite each other, and everyone sees every column.
- **Relational databases** keep data in tables with fixed columns, linked by keys. NoSQL databases use other shapes: documents, key-value pairs, graphs, wide columns.
- **Table, column, row.** A table holds one kind of thing; a column is one named, typed piece of information; a row is one thing.
- **Primary key:** unique, never empty, identifies each row (usually an id MySQL hands out). **Foreign key:** another table's primary key, stored to say "this row belongs to that one".
- **Store each fact once and point to it with a key.** A list in a cell or a many-to-many relationship needs its own table.
- **SQL is declarative:** you say what you want, not how to find it. CRUD = INSERT, SELECT, UPDATE, DELETE.
- **The database says no**, with a number: 1062 duplicate key, 1366 wrong type, 1364 missing required value. That is a feature.

WORKED EXAMPLE: FOLLOWING A KEY BY HAND

Keys in action, on two tiny tables: authors and their books. The book does not repeat the author's name; it stores the author's id.

```
CREATE DATABASE kit_shelf;
USE kit_shelf;
CREATE TABLE authors (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
CREATE TABLE books (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    title     VARCHAR(200) NOT NULL,
    author_id INT NOT NULL            -- which author: a copy of authors.id
);
INSERT INTO authors (name) VALUES ('Frank Herbert'), ('George Orwell');
INSERT INTO books (title, author_id) VALUES
    ('Dune', 1), ('1984', 2), ('Animal Farm', 2);
```

```
Database changed
```

Which books did author 2 write, and who is author 2? Follow the key by hand, one table at a time (Lesson 4.3 teaches the JOIN that does it in one query):

```
SELECT id, title, author_id FROM books WHERE author_id = 2;
SELECT id, name FROM authors WHERE id = 2;
```

```
+----+-------------+-----------+
| id | title       | author_id |
+----+-------------+-----------+
|  2 | 1984        |         2 |
|  3 | Animal Farm |         2 |
+----+-------------+-----------+
+----+---------------+
| id | name          |
+----+---------------+
|  2 | George Orwell |
+----+---------------+
```

Now correct the author's name. One row changes, and every book that points at id 2 is up to date at once:

```
UPDATE authors SET name = 'George Orwell (Eric Blair)' WHERE id = 2;
SELECT id, name FROM authors ORDER BY id;
```

```
+----+---------------------------+
| id | name                      |
+----+---------------------------+
|  1 | Frank Herbert             |
|  2 | George Orwell (Eric Blair) |
+----+---------------------------+
```

In a single sheet with the author's name typed next to every book, the same correction would mean finding and fixing every copy. Last, clean up:

```
DROP DATABASE kit_shelf;
```

GUIDED LAB

- Open `labs/module-01/l1_1-what-is-a-database/order_sheet_starter.sql` in a text editor. In the mysql client (started in your course folder) run `SOURCE labs/module-01/l1_1-what-is-a-database/order_sheet_starter.sql`. You see the sheet and empty answers.
- TODO 1 together, TODOs 2 to 4 in pairs: each changes one value. Save the file, SOURCE it again, read the new result.
- For each TODO write one sentence: what is the problem, and how would separate tables with keys prevent it?

- Done when your results match the "Expected when done" comment at the end of the file.
- Finished early? `crud_tour_starter.sql`, then `debug_rules.sql` in the same folder.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code (in a practice database: `CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done). A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Three notes, one DELETE.)

```
CREATE TABLE notes (id INT PRIMARY KEY, title VARCHAR(30) NOT NULL);
INSERT INTO notes VALUES (1, 'Buy milk'), (2, 'Call Bob'), (3, 'Pay rent');
DELETE FROM notes WHERE id = 2;
SELECT * FROM notes;
```

P2

Predict the output, then run it. (email is UNIQUE.)

```
CREATE TABLE members (id INT PRIMARY KEY, email VARCHAR(50) NOT NULL UNIQUE);
INSERT INTO members VALUES (1, 'ana@example.com');
INSERT INTO members VALUES (2, 'ana@example.com');
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Sketch the Bookstore (lesson page, "Project: Sketch the Bookstore"). Paper or a text file. In class: Steps 1 and 2. At home: Steps 3 to 5.

- ☐ Step 1: about five tables from the store's description
- ☐ Step 2: four to six columns per table, each with a type
- ☐ Step 3: a primary key for every table (and why a title or an email is a poor one)
- ☐ Step 4: foreign keys on the "many" side, and two linking tables
- ☐ Step 5: compared with the course design; one difference I can defend

SELF-CHECK

Q1

Which sentence describes MySQL correctly?

A) MySQL is a database: the file that holds your tables

B) MySQL is a database management system: server software that stores databases, enforces their rules and answers SQL

C) MySQL is a query language, a dialect of SQL

D) MySQL is a spreadsheet program that understands SQL

Q2

A `books` table has the columns `id`, `title`, `author_id`, `price`. Which column is most likely a foreign key?

A) `id`

B) `title`

C) `author_id`

D) `price`

Q3

What does the last statement print?

```
CREATE TABLE pets (id INT PRIMARY KEY, name VARCHAR(20), kind VARCHAR(10));
INSERT INTO pets VALUES (1, 'Rex', 'dog'), (2, 'Tom', 'cat'), (3, 'Fido', 'dog
UPDATE pets SET kind = 'cat' WHERE id = 3;
SELECT name FROM pets WHERE kind = 'cat' ORDER BY id;
```

Q4

The pets table from Q3 is still there. What happens when this runs?

```
-- pets now holds ids 1, 2 and 3
INSERT INTO pets VALUES (2, 'Kit', 'cat');
```

A) `ERROR 1062 (23000): Duplicate entry '2' for key 'pets.PRIMARY'`

B) A new row is added: Kit gets id 4 automatically

C) `ERROR 1366 (HY000): Incorrect integer value`

D) Tom's row is replaced by Kit's

Q5

A music app must remember, for millions of logged-in users, which song each is playing right now, and look it up in under a millisecond. Which kind of database fits best?

A) A relational database with a table per song

B) A graph database

C) A document store

D) A key-value store such as Redis

Q6

Carol closes her account. Which statement removes her row but keeps the table and every other customer?

A) `DROP TABLE customers;`

B) `DELETE FROM customers WHERE id = 3;`

C) `UPDATE customers SET email = NULL WHERE id = 3;`

D) `DELETE FROM customers;`

JOURNAL

Think of a list you keep today in a spreadsheet, a notes app or on paper. Which of the spreadsheet problems have you actually run into with it? What would its tables and primary keys be?

1.2 Installing MySQL

BY THE END YOU CAN

- Choose the right MySQL release (9.7 LTS) and installation method for your computer, and recognize when a package gives you something else (an older MySQL, or MariaDB)
- Install MySQL 9.7 LTS with the Windows MSI, the macOS package or Homebrew, the MySQL APT repository, or the official Docker image
- Start and stop the server, and set a strong root password
- Connect with mysql -u root -p and verify the version, user and character set with SQL
- Diagnose the four classic first-day errors: command not found, 1045, 2002 and 2003

KEY IDEAS

- **Two programs:** the server (`mysqld`, listening on port 3306) and a client (the `mysql` command) that sends your SQL to it.
- **Install 9.7 LTS on purpose.** "Latest" is an innovation release (26.x); Ubuntu's own package is 8.0 or 8.4; XAMPP and Debian give you MariaDB.
- **One path per laptop:** Windows MSI + MySQL Configurator; macOS `brew install mysql@9.7` or the DMG; Linux MySQL's APT/Yum repository; anywhere `docker run ... mysql:9.7`.
- **root gets a strong password** and the default `caching_sha2_password` login. `mysql_native_password` is gone in 9.x (error 1524).
- **Log in with** `mysql -u root -p`; a Docker container with `docker exec -it mysql97 mysql -u root -p`, or `-h 127.0.0.1 -P <port>`.
- **Ask the server, not the client:** `SELECT VERSION();` describes the server; `mysql --version` only the program on your laptop.
- **Error triage:** command not found = PATH; 1045 = credentials; 2002/2003 = server stopped or wrong address.

WORKED EXAMPLE: THREE QUESTIONS FOR ANY SERVER

Three questions you can ask any server in a second. A comparison prints 1 for true and 0 for false.

```
SELECT VERSION() LIKE '9.7.%'     AS is_mysql_97,
       VERSION() LIKE '%MariaDB%' AS is_mariadb;
```

```
+-------------+------------+
| is_mysql_97 | is_mariadb |
+-------------+------------+
|           1 |          0 |
+-------------+------------+
```

`SHOW VARIABLES LIKE` is another way to read a setting, with its name next to its value:

```
SHOW VARIABLES LIKE 'character_set_server';
```

```
+----------------------+---------+
| Variable_name        | Value   |
+----------------------+---------+
| character_set_server | utf8mb4 |
+----------------------+---------+
```

And is the server in the strict mode that refuses bad data (the mode the lesson's errors rely on)?

```
SELECT @@sql_mode LIKE '%STRICT_TRANS_TABLES%' AS strict_mode;
```

```
+-------------+
| strict_mode |
+-------------+
|           1 |
+-------------+
```

On MariaDB the first query prints 0 and 1; on a server someone has reconfigured the last one may print 0, and then some of the course's errors become silent warnings.

◆ GUIDED LAB

- SOURCE `labs/module-01/l1_2-installing-mysql/server_facts_starter.sql`. Questions 1 and 2 answer right away.
- TODOs 3 to 6: replace each placeholder SELECT with a real one. TODO 3 gives you the statement; for TODOs 4 and 5 write it from the setting names.
- Copy the answers onto your health-check page (below).
- Finished early? `accounts_check_starter.sql`, then `debug_old_advice.sql`.

SETTING	MY SERVER
Version and edition	
MySQL or MariaDB?	

SETTING	MY SERVER
Character set / collation	
Storage engine / lower_case_table_names	
max_connections	
root's login plugin	

PREDICT, THEN RUN

Write your prediction on the line first, then run the code (in a practice database: `CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done). A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Who did MySQL match you to?)

```
SELECT CURRENT_USER();
```

P4

Predict the output, then run it. (On the course's MySQL 9.7 server.)

```
SELECT VERSION() LIKE '%MariaDB%' AS on_mariadb,
       @@character_set_server   AS charset;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Installation Health Check (lesson page, "Project: Installation Health Check"). Steps 1 and 2 were the setup and the clinic. In class: Steps 3 and 4. At home: Step 5.

- ☐ Step 3: version, edition and port; logged-in user, charset and collation

- ☐ Step 4: root's host and plugin (no `mysql_native_password`)
- ☐ Step 5: stopped the server, saw error 2002 or 2003, started it again
- ☐ My start command: _____ My stop command: _____

SELF-CHECK

Q7

`mysql --version` in a terminal prints 9.7.2. What does that prove?

A) The MySQL server is running

B) Your password is correct

C) The mysql client program is installed and on your PATH

D) Your server is MySQL 9.7, not MariaDB

Q8

`mysql -u root -p` answers `ERROR 1045 (28000): Access denied for user ...`. What do you fix first?

A) The user name or the password

B) Start the server

C) Add mysql to your PATH

D) Reinstall MySQL with the native password plugin

Q9

On the course's MySQL 9.7 server, what does this print?

```
SELECT VERSION() LIKE '9.7.%' AS on_97;
```

Q10

You use Homebrew on a Mac. Which command installs the version this course uses?

A) `brew install mysql`

B) `brew install mariadb`

C) `brew install mysql-client`

D) `brew install mysql@9.7`

Q11

Your MySQL container was started with `-p 3307:3306`. Which command connects to it from a mysql client installed on your laptop?

A) `mysql -u root -p`

B) `mysql -h localhost -P 3306 -u root -p`

C) `mysql -h 127.0.0.1 -P 3307 -u root -p`

D) `mysql -h 127.0.0.1 -P 3306 -u root -p`

Q12

On a fresh MySQL 9.7 install (not Docker), what does this print?

```
SELECT plugin
FROM mysql.user
WHERE user = 'root' AND host = 'localhost';
```

JOURNAL

Write your installation story: which path you chose and why, what went wrong (something usually does), the exact error message you saw, and what fixed it. Next time you install a database, this note will save you an hour.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

In one sentence each: what is the MySQL server, what is a client, and what is a database?

E2

You type `mysql -u root -p` and get `ERROR 2002`. What do you check first? And if it were `ERROR 1045`?

E3

`orders(id, customer_id, order_date)`: name the primary key and the foreign key. Why not store the customer's name in orders?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the bookstore sketch (Steps 3 to 5) and the health check (Step 5)
- ☐ Lesson 1.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 1.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ 1.2 "Before the Next Lesson": log in from a fresh terminal; install a code editor for .sql files; optional phpMyAdmin
- ☐ If my check was not READY: follow my plan and run the check again
- ☐ Optional: `diagnostic_starter.sql`, `crud_tour_starter.sql`, `accounts_check_starter.sql`
- ☐ Bring to Session 2: a READY laptop and my root password, the course folder with `labs/`, my bookstore sketch, this workbook and my journal

GLOSSARY

TERM	MEANING
database	An organized collection of data; in MySQL, a named set of tables inside the server
DBMS	The software that stores databases, enforces their rules and answers queries (MySQL, MariaDB, PostgreSQL)
server (`mysqld`)	The MySQL program that runs in the background, owns the data and listens on port 3306
client	A program you type SQL into, which sends it to the server (`mysql`, phpMyAdmin, Workbench)
SQL	The declarative language of relational databases
CRUD	Create, Read, Update, Delete: INSERT, SELECT, UPDATE, DELETE
table / row / column	All the rows about one kind of thing / one thing / one named, typed piece of information
primary key	A unique, never-empty identifier of each row
foreign key	A copy of another table's primary key that links two rows
schema	The blueprint of tables, columns, types and rules (in MySQL also another word for a database)

TERM	MEANING
LTS release	A long-term support version (9.7) that keeps its features for years
port	The number a server listens on (3306 for MySQL); a client must knock on the right one
root	MySQL's all-powerful administrator account; strong password, administration only
caching_sha2_password	MySQL's default, secure login method (mysql_native_password is gone in 9.x)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 2: The MySQL Client and Your First Tables

Your workbook for Session 2: a warm-up on Session 1, the key ideas of Lessons 1.3 and 2.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 1.3–2.1 | Session 2 · 3 hours | Name: _____

SESSION GOALS

- Move around any MySQL server: log in and out, read the prompts, list databases and tables, look inside a table.
- Tell SQL statements (they work in every client) from mysql client commands such as \G and SOURCE.
- Save SQL in a .sql file and run it with SOURCE and from the shell.
- Create a database with utf8mb4 and a named collation, and tables with columns, types and basic rules.
- Change a table with ALTER TABLE without losing a rule, and build the bookstore from a script you can rerun.

WARM-UP: SESSION 1

Five questions from Lessons 1.1 and 1.2. Four minutes, no notes. Circle your answer; the class checks them together.

W1

Is MySQL a database?

A) Yes: MySQL is the file that holds your tables

B) No: MySQL is the server (a DBMS); a database such as bookstore lives inside it

C) No: MySQL is a query language

D) Yes: every MySQL install is one database

W2

Which error tells you the server is running and answered?

A) `ERROR 2002`

B) `ERROR 2003`

C) `ERROR 1045 (28000): Access denied`

D) None of them

W3

What does the SELECT print?

```
CREATE TABLE pets (id INT PRIMARY KEY, name VARCHAR(20));
INSERT INTO pets VALUES (1, 'Rex'), (2, 'Tom'), (3, 'Fido');
DELETE FROM pets WHERE id = 2;
SELECT * FROM pets;
```

A) Rows 1 Rex and 3 Fido

B) Rows 1 Rex and 2 Fido: the ids are renumbered

C) All three rows

D) `Empty set`

W4

Which statement removes Carol's row (id 3) and keeps every other customer?

A) `DROP TABLE customers;`

B) `DELETE FROM customers;`

C) `DELETE FROM customers WHERE id = 3;`

D) `UPDATE customers SET id = NULL WHERE id = 3;`

W5

`mysql --version` prints 9.7.2. What does that prove about your server?

A) That it is 9.7.2

B) That it is running

C) That it is MySQL, not MariaDB

D) Nothing: it describes the client; ask the server with SELECT VERSION();

LESSONS

1.3 MySQL CLI & phpMyAdmin

BY THE END YOU CAN

- Log in to and out of the mysql client, and read its prompts (mysql>, ->, '>)
- Explore a server with SHOW DATABASES, USE, SHOW TABLES, DESCRIBE and SELECT DATABASE()
- Tell SQL statements apart from client commands such as \G, \c, SOURCE and help
- Save SQL in a .sql file and run it with SOURCE and with mysql < file.sql, knowing what MySQL 9.x's client does differently in scripts
- Find the Structure, SQL, Import and Export tabs in phpMyAdmin and choose the right tool for a task

KEY IDEAS

- **A client** is any program that sends SQL to the server: the mysql command, phpMyAdmin, Workbench. Same statement, same result in all of them.
- **Log in** with `mysql -u root -p` (no space and no password after -p); leave with `exit`, `\q` or Ctrl+D. Ctrl+C cancels a statement; it does not log you out.
- **Find your way:** `SHOW DATABASES;` · `USE name;` · `SELECT DATABASE();` · `SHOW TABLES;` · `DESCRIBE table;`. `FROM database` and `database.table` reach other databases without moving.
- **Nothing runs until the semicolon.** `->` = waiting for the end · `'>` = a quote is open · `\c` = give up.
- **SQL versus client commands:** SHOW, USE, DESCRIBE work in every client; `\G` (vertical output), `\c`, `SOURCE`, `help`, `status`, `exit` only in the mysql client.
- **Run files** with `SOURCE path` at the prompt or `mysql -u root -p < file.sql` from the shell (batch mode: no boxes without -t; MySQL 9.x ignores client commands there unless you add --commands).
- **Errors to know:** 1049 unknown database · 1146 table not found (the message names the database it looked in) · 1064 cannot read the statement (often a reserved word such as current_time).

WORKED EXAMPLE: EXPLORING WITHOUT MOVING

You can explore without moving: in a fresh session there is no current database, and a full name `database.table` (or `FROM database`) reaches any table.

```
SELECT DATABASE() AS current_db;
```

```
+------------+
| current_db |
+------------+
| NULL       |
+------------+
```

Which tables hold the built-in help texts? Ask the mysql database without switching to it:

```
SHOW TABLES FROM mysql LIKE 'help%';
```

```
+-------------------------+
| Tables_in_mysql (help%) |
+-------------------------+
| help_category           |
| help_keyword            |
| help_relation           |
| help_topic              |
+-------------------------+
```

What does one of them look like? `SHOW COLUMNS FROM` is DESCRIBE by another name:

```
SHOW COLUMNS FROM mysql.help_category;
```

```
+--------------------+------------------+------+-----+---------+-------+
| Field              | Type             | Null | Key | Default | Extra |
+--------------------+------------------+------+-----+---------+-------+
| help_category_id   | smallint unsigned | NO   | PRI | NULL    |       |
| name               | char(64)         | NO   | UNI | NULL    |       |
| parent_category_id | smallint unsigned | YES  |     | NULL    |       |
| url                | text             | NO   |     | NULL    |       |
+--------------------+------------------+------+-----+---------+-------+
```

And a few of its rows (a SELECT with a full table name, filtered and sorted):

```
SELECT name
FROM mysql.help_category
WHERE name LIKE 'Data%'
ORDER BY name;
```

```
+-------------------+
| name              |
+-------------------+
| Data Definition   |
| Data Manipulation |
| Data Types        |
+-------------------+
```

`SELECT DATABASE();` still says NULL: you never left. The same full names work in phpMyAdmin's SQL tab.

GUIDED LAB

- SOURCE `labs/module-02/l1_3-mysql-cli-phpmyadmin/find_your_way_starter.sql`. The setup builds a café database; do not read it.
- TODO 1 together, TODOs 2 to 6 in pairs: replace each placeholder SELECT with one statement, save, SOURCE again.
- Write what you found:

QUESTION	MY ANSWER
Tables in kit_s2_cafe	
menu_items: primary key / required columns	
cafe_orders: the _at columns	
time_zone tables in mysql	
Menu item 3 (vertically)	
staff_notes: engine and collation	

- Finished early? `system_tour_starter.sql`, then `debug_lost.sql` in the same folder.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code at the mysql> prompt. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (A fresh session: nothing chosen yet.)

```
SELECT DATABASE() AS before_use;
USE mysql;
SELECT DATABASE() AS after_use;
```

P2

Predict the output, then run it. (One column of mysql.user.)

```
SHOW COLUMNS FROM mysql.user LIKE 'account_locked';
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Server Report Script (lesson page, "Project: A Server Report Script"). In class: Steps 1 to 3. At home: Steps 4 and 5.

- ☐ Step 1: who am I, and where (USER, CURRENT_USER, DATABASE)
- ☐ Step 2: tables per system database from information_schema
- ☐ Step 3: explore.sql saved, a header comment, every statement ends with ;
- ☐ Step 4: run with SOURCE, with `mysql -u root -p < explore.sql` and with -t; why no boxes without -t?
- ☐ Step 5: the same SQL in phpMyAdmin (or DESCRIBE mysql.user; in the client)

SELF-CHECK

Q1

Which of these does nothing in phpMyAdmin's SQL tab, because only the mysql command-line client understands it?

A) `SHOW DATABASES;`

B) `DESCRIBE user;`

C) `SOURCE ~/mysql-course/hello.sql`

D) `SELECT DATABASE();`

Q2

You press Enter and the prompt changes from `mysql>` to `'>`. What does it mean?

A) The statement only needs a semicolon

B) A single quote is still open: close it, then type ;

C) The server is still running the query

D) You were logged out

Q3

On MySQL 9.7, how many rows does this print?

```
SHOW TABLES FROM mysql LIKE 'time_zone%';
```

Q4

You type `mysql -u root -p secret123` (with a space after -p). What happens?

A) It logs in with the password secret123

B) `ERROR 1045 (28000): Access denied`

C) It logs in without asking for a password

D) It asks for the password, then fails with `ERROR 1049 (42000): Unknown database 'secret123'`

Q5

report.sql ends its query with `\G`. Which way of running it stops with Unknown command on MySQL 9.7?

A) `mysql -u root -p < report.sql`

B) `SOURCE report.sql` at the mysql> prompt

C) `mysql --commands -u root -p < report.sql`

D) Typing the query at the prompt

Q6

Which statement lists only the columns of mysql.user whose names contain lock?

A) `SHOW TABLES FROM mysql LIKE '%lock%';`

B) `SHOW COLUMNS FROM mysql.user LIKE '%lock%';`

C) `DESCRIBE mysql.user WHERE lock;`

D) `SELECT lock FROM mysql.user;`

JOURNAL

Make your own cheat sheet of the ten commands you used most today, in two columns: "SQL (works in any client)" and "mysql client only". Which one surprised you by being in the column it is in?

2.1 Creating Databases & Tables

BY THE END YOU CAN

- Create, select and drop a database, with the utf8mb4 character set and an explicit collation
- Write a CREATE TABLE statement with column names, data types and basic constraints
- Inspect a table with SHOW TABLES, DESCRIBE and SHOW CREATE TABLE
- Add, rename, change and remove columns with ALTER TABLE, without losing a constraint by accident
- Rename, empty and drop tables, and use IF EXISTS / IF NOT EXISTS to write scripts you can rerun
- Read and fix errors 1007, 1046, 1050, 1064 and 1146

KEY IDEAS

- **Server = cabinet, database = drawer, table = folder.** `CREATE DATABASE`, `USE`, `DROP DATABASE` (no undo).
- **Name the text settings:** `CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci`. Never utf8 (a three-byte subset, warning 3719).
- **Names:** lowercase, snake_case, plural tables (`books`), `id` for the key, `customer_id` for a link, no reserved words (`order`).
- **CREATE TABLE** lists each column's name, type and rules; commas between columns, none after the last. `IF NOT EXISTS` leaves an existing table untouched (Note 1050).
- **Inspect:** `DESCRIBE t;` for the columns, `SHOW CREATE TABLE t\G` for the full definition (engine, charset, collation).
- **ALTER TABLE** adds, renames, redefines and drops columns and keeps the rows. **MODIFY and CHANGE replace the whole definition:** repeat NOT NULL and DEFAULT, then DESCRIBE.
- **TRUNCATE** empties a table and resets AUTO_INCREMENT; **DROP** removes the table or database itself.
- **Build from a script** that starts with `DROP DATABASE IF EXISTS` and can run any number of times. Errors: 1007/1050 exists · 1046 no database selected · 1064 · 1146.

WORKED EXAMPLE: A BUILD SCRIPT FOR A RECIPE BOX

A build script for a recipe box, from nothing: the same shape as `create_bookstore.sql`, on another subject.

```
DROP DATABASE IF EXISTS kit_recipes;
CREATE DATABASE kit_recipes
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE kit_recipes;
CREATE TABLE recipes (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL UNIQUE,
    minutes     INT,
    vegetarian  BOOLEAN DEFAULT FALSE,
    added_on    DATE
);
SHOW TABLES;
```

```
Database changed
+-----------------------+
| Tables_in_kit_recipes |
+-----------------------+
| recipes               |
+-----------------------+
```

Check what MySQL stored. BOOLEAN became tinyint(1) and FALSE became 0:

```
DESCRIBE recipes;
```

```
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| id         | int          | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100) | NO   | UNI | NULL    |                |
| minutes    | int          | YES  |     | NULL    |                |
| vegetarian | tinyint(1)   | YES  |     | 0       |                |
| added_on   | date         | YES  |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
```

The full definition, vertically: the UNIQUE KEY MySQL built for name, the InnoDB engine, and the collation inherited from the database:

```
SHOW CREATE TABLE recipes\G
```

```
*************************** 1. row ***************************
       Table: recipes
Create Table: CREATE TABLE `recipes` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL,
  `minutes` int DEFAULT NULL,
  `vegetarian` tinyint(1) DEFAULT '0',
  `added_on` date DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

Clean up (the recipe box was practice):

```
DROP DATABASE kit_recipes;
```

GUIDED LAB

- Open `labs/module-02/l2_1-creating-databases-tables/events_build_starter.sql`. Step 1 builds the database for you.
- TODO 1: fill the three gaps in the rooms skeleton. TODO 2: write the readings table from the spec in the comment. TODO 3: turn on the two lines.
- SOURCE it, read the self-check, then SOURCE it again: a build script must work twice in a row.
- Done when the self-check lists readings (7 columns) and rooms (3), both utf8mb4_unicode_ci.
- Finished early? `change_request_starter.sql`, then `debug_build_script.sql`.

PREDICT, THEN RUN

Write your prediction first, then run the code in a practice database (`CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done).

P3

Predict the output, then run it. (TRUNCATE, then one more row.)

```
CREATE TABLE tickets (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    label VARCHAR(20) NOT NULL
);
INSERT INTO tickets (label) VALUES ('first'), ('second');
TRUNCATE TABLE tickets;
INSERT INTO tickets (label) VALUES ('third');
SELECT * FROM tickets;
```

P4

Predict the output, then run it. (MODIFY without NOT NULL.)

```
CREATE TABLE notes (
    id    INT PRIMARY KEY,
    body  VARCHAR(100) NOT NULL
);
ALTER TABLE notes MODIFY COLUMN body TEXT;
SHOW COLUMNS FROM notes LIKE 'body';
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Bookstore Build Script (lesson page, "Project: The Bookstore Build Script"). All five steps in class: Session 3 starts from it.

- ☐ Step 1: header, `DROP DATABASE IF EXISTS bookstore;`, utf8mb4 + utf8mb4_unicode_ci, USE
- ☐ Step 2: books (seven columns)
- ☐ Step 3: customers and categories, each with a comment
- ☐ Step 4: the information_schema self-check

- ☐ Step 5: run it twice; the self-check shows books, categories, customers both times

SELF-CHECK

Q7

What happens when this runs?

```
CREATE DATABASE kit_q7 CHARACTER SET utf8;
SHOW WARNINGS;
```

A) `ERROR 1064`: utf8 is not a character set

B) The database stores full Unicode, emoji included

C) Nothing: CHARACTER SET is ignored

D) The database is created with utf8mb3 (no emoji), and warning 3719 says to use utf8mb4

Q8

Which column does the last statement list first?

```
CREATE TABLE members (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(40) NOT NULL
);
ALTER TABLE members ADD COLUMN email VARCHAR(100) FIRST;
SHOW COLUMNS FROM members;
```

Q9

A script stops with `ERROR 1046 (3D000): No database selected`. What does it mean?

A) There is no current database: run USE first, or write database.table

B) The database name is misspelled

C) The table already exists

D) Your account may not create tables

Q10

Which name follows the course's naming conventions for a table of book reviews?

A) `BookReviews`

B) `tbl_review`

C) `book_reviews`

D) `bookReview`

Q11

In `shelf`, title was `VARCHAR(200) NOT NULL`. After the ALTER below, what is true?

```
CREATE TABLE shelf (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL
);
ALTER TABLE shelf CHANGE COLUMN title book_title VARCHAR(250);
SHOW COLUMNS FROM shelf;
```

A) The column is now book_title and is still required

B) The column is now book_title, VARCHAR(250), and allows NULL

C) ERROR: CHANGE cannot rename a column

D) title is kept and a new column book_title is added

Q12

Write the one statement that removes every row of the customers table, keeps the table and starts the ids at 1 again.

JOURNAL

Sketch, on paper, the tables you would create for something you know well (a recipe box, a game collection, a club's members). Give each table a plural name and three to five columns with a type you would guess for each. Which column would be the primary key? Keep the sketch: Lesson 2.2 checks your type choices.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Name two statements that work in phpMyAdmin's SQL tab and one command that only the mysql client understands.

E2

A script stops with `ERROR 1046 (3D000): No database selected`. Give two ways to fix it.

E3

title is `VARCHAR(200) NOT NULL`. You run `ALTER TABLE books MODIFY COLUMN title VARCHAR(250);`. What did you lose, and which statement shows it?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish explore.sql (Steps 4 and 5)
- ☐ create_bookstore.sql finished and run once more from a fresh session: SHOW TABLES lists books, categories, customers
- ☐ Lesson 1.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries (keep the 2.1 sketch for Lesson 2.2)
- ☐ Optional: `system_tour_starter.sql`, `change_request_starter.sql`, the two debug files
- ☐ Bring to Session 3: bookstore built on my laptop, create_bookstore.sql, my 2.1 sketch, this workbook and my journal

GLOSSARY

TERM	MEANING
client	A program that sends SQL to the server and shows the answer (`mysql`, phpMyAdmin, Workbench)

TERM	MEANING
client command	Something only the mysql program understands (`\G` , `\c` , SOURCE, help, exit); never sent to the server
prompt	`mysql>` ready · `->` statement not finished · `'>` quote open
current database	The one `USE` selected; `SELECT DATABASE();` shows it (NULL if none)
information_schema	Read-only system database that describes every database, table and column ("data about data")
batch mode	The client reading a file from the shell (`mysql < file.sql`): plain text output, no client commands in 9.x
character set	Which characters can be stored; always `utf8mb4`
collation	How text is compared and sorted; `_ci` = case-insensitive (`utf8mb4_unicode_ci`)
constraint	A rule on a column: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT (Lesson 2.2 adds CHECK)
ALTER TABLE	Changes a table's structure and keeps its rows (ADD, RENAME, MODIFY, CHANGE, DROP COLUMN)
TRUNCATE	Removes every row, keeps the table, resets AUTO_INCREMENT
build script	A .sql file that rebuilds a database from nothing and can run any number of times

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 3: Data Types, Constraints and CRUD

Your workbook for Session 3: a warm-up on Session 2, the key ideas of Lessons 2.2 and 2.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

`Lessons 2.2–2.3` `Session 3 · 3 hours` `Name: _____`

SESSION GOALS

- Choose a type for every column: integers, DECIMAL for money, VARCHAR, CHAR, DATE and DATETIME, BOOLEAN, ENUM.
- Add the rules that keep bad data out: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, named CHECK, and read the error each one raises.
- Add, read, change and remove rows with INSERT, SELECT, UPDATE and DELETE.
- Change many rows safely: preview with SELECT, read Rows matched and Changed, use safe-updates mode.
- Write an upsert, and load the bookstore's official data from a script you can rerun.

WARM-UP: SESSION 2

Five questions from Lessons 1.3 and 2.1. Four minutes, no notes.

W1

The prompt shows `->`. What do you type?

A) `exit`

B) `;` and Enter

C) Ctrl+C

D) `USE bookstore;`

W2

MySQL says `ERROR 1146 (42S02): Table 'mysql.books' doesn't exist`. What do you check first?

A) Reinstall MySQL

B) Whether books was dropped

C) Which database you are in: the message says mysql, not bookstore

D) The password

W3

Which of these works in phpMyAdmin's SQL tab?

A) `DESCRIBE books;`

B) `SOURCE create_bookstore.sql`

C) `\G`

D) `exit`

W4

What does SHOW COLUMNS print under Null?

```
CREATE TABLE w4 (id INT PRIMARY KEY, title VARCHAR(200) NOT NULL);
ALTER TABLE w4 MODIFY COLUMN title VARCHAR(250);
SHOW COLUMNS FROM w4 LIKE 'title';
```

A) `NO`: NOT NULL is kept

B) `YES`: MODIFY replaced the whole definition

C) Nothing: ERROR 1064

D) `NO`, with a warning

W5

Right after `DROP DATABASE scratch;` (you were inside scratch), `CREATE TABLE t (id INT);` fails. Why?

A) The table exists (1050)

B) A syntax error (1064)

C) Permission denied

D) No database selected (1046): USE one, or write database.table

LESSONS

2.2 Data Types & Constraints

BY THE END YOU CAN

- Pick an integer type and decide when UNSIGNED makes sense; explain why money is DECIMAL, never FLOAT
- Choose between CHAR, VARCHAR and TEXT, and between DATE, DATETIME and TIMESTAMP
- Use BOOLEAN and ENUM correctly, knowing what MySQL really stores
- Apply PRIMARY KEY, AUTO_INCREMENT, NOT NULL, UNIQUE, DEFAULT and named CHECK constraints
- Read the errors strict mode raises for bad values (1048, 1062, 1264, 1265, 1292, 1364, 1406, 3819)

KEY IDEAS

- **A type is a promise** MySQL keeps: it validates values, makes math and sorting work, and decides the size.
- **Strict mode** (the default) turns a value that does not fit into an error instead of a silent change. Keep it on.
- **Numbers:** INT is the default; TINYINT/SMALLINT for small ranges; UNSIGNED forbids negatives. **Money is DECIMAL(m, d), never FLOAT.**
- **Text:** VARCHAR(n) for almost everything, CHAR(n) for fixed-length codes, TEXT for long writing. Phone numbers and ZIP codes are text.
- **Dates** are 'YYYY-MM-DD'. DATETIME for moments; TIMESTAMP converts time zones but ends in 2038.
- **BOOLEAN is TINYINT(1):** test it with `WHERE col`, not `= TRUE`. **ENUM** is one value from a short, stable list.
- **Constraints:** PRIMARY KEY, AUTO_INCREMENT, NOT NULL, UNIQUE, DEFAULT, named CHECK. Errors: 1048/1364 required · 1062 duplicate · 1264 out of range · 1265 not on the list · 1292 bad date · 1406 too long · 3819 CHECK.

WORKED EXAMPLE: A PREORDER SLIP

A preorder slip, with a type and rules for every column: the email and ISBN are required, the number of copies is small and never negative, the deposit is money with a limit, the format is one of three.

```
CREATE TABLE kit_preorders (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    email      VARCHAR(100) NOT NULL,
    isbn       CHAR(13) NOT NULL,
    copies     TINYINT UNSIGNED NOT NULL DEFAULT 1,
    deposit    DECIMAL(6, 2) NOT NULL DEFAULT 0.00,
    format     ENUM('paperback', 'hardcover', 'ebook') NOT NULL DEFAULT 'paperback
    wanted_by  DATE,
    CONSTRAINT chk_preorder_deposit CHECK (deposit <= 50)
);
INSERT INTO kit_preorders (email, isbn) VALUES ('ana@example.com', '9780441013593'
INSERT INTO kit_preorders (email, isbn, copies, deposit, format, wanted_by)
VALUES ('ben@example.com', '9780547928227', 2, 10.00, 'hardcover', '2026-12-15');
```

Ana gave only the required values; the defaults filled in the rest:

```
SELECT id, email, copies, deposit, format, wanted_by
FROM kit_preorders
ORDER BY id;
```

```
+----+-----------------+--------+---------+-----------+------------+
| id | email           | copies | deposit | format    | wanted_by  |
+----+-----------------+--------+---------+-----------+------------+
|  1 | ana@example.com |      1 |    0.00 | paperback | NULL       |
|  2 | ben@example.com |      2 |   10.00 | hardcover | 2026-12-15 |
+----+-----------------+--------+---------+-----------+------------+
```

MySQL keeps every CHECK in information_schema, under the name you gave it:

```
SELECT CONSTRAINT_NAME, CHECK_CLAUSE
FROM information_schema.CHECK_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = DATABASE();
```

```
+----------------------+-------------------+
| CONSTRAINT_NAME      | CHECK_CLAUSE      |
+----------------------+-------------------+
| chk_preorder_deposit | (`deposit` <= 50) |
+----------------------+-------------------+
```

Clean up (the slip was practice):

```
DROP TABLE kit_preorders;
```

GUIDED LAB

- `USE bookstore;`, then SOURCE `labs/module-03/l2_2-data-types-constraints/types_matter_starter.sql` as it is.
- Which of the four answers are wrong, and why? Write it down before you change anything.
- TODOs 1 to 3: fix the three types (and convert the dates). Run it again.

QUESTION	AS SHIPPED	WITH THE RIGHT TYPES
Biggest delivery first		
Total cost		
Calendar order		
Last delivery		

- Finished early? `membership_rules_starter.sql`, then `debug_bad_rows.sql`.

PREDICT, THEN RUN

Write your prediction first, then run the code in a practice database (`CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done, then `USE bookstore;`).

P1

Predict the output, then run it. (Two spaces at the end, twice.)

```
CREATE TABLE codes (code CHAR(6), label VARCHAR(6));
INSERT INTO codes VALUES ('ab  ', 'ab  ');
SELECT CHAR_LENGTH(code)  AS code_len,
       CHAR_LENGTH(label) AS label_len
FROM codes;
```

P2

Predict the output, then run it. (255 and then 256.)

```
CREATE TABLE counts (n TINYINT UNSIGNED);
INSERT INTO counts VALUES (255);
INSERT INTO counts VALUES (256);
SELECT n FROM counts;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Staff Table (lesson page, "Project: The Staff Table"). In class: Steps 1 to 4 and Step 6's cleanup. At home: Step 5 and a rerun.

- ☐ Step 1: my types and rules on paper, before I look
- ☐ Steps 2 and 3: the table and three staff members
- ☐ Step 4: four bad rows refused: 3819, 1062, 1265, 1364 (my predictions: _____)
- ☐ Step 5: COUNT is still 3; I found the UNIQUE KEY and chk_salary in SHOW CREATE TABLE
- ☐ Step 6: demo tables dropped; SHOW TABLES lists books, categories, customers

SELF-CHECK

Q1

A column holds a discount as a whole percentage from 0 to 100. Which type is the smallest sensible choice?

A) `TINYINT UNSIGNED`

B) `INT`

C) `DECIMAL(5, 2)`

D) `VARCHAR(3)`

Q2

What does the SELECT print?

```
CREATE TABLE q2 (amount DECIMAL(5, 2));
INSERT INTO q2 VALUES (7.005);
SELECT amount FROM q2;
```

Q3

What happens when this runs?

```
CREATE TABLE q3 (due DATE);
INSERT INTO q3 VALUES ('2026-04-31');
```

A) 2026-05-01 is stored

B) 2026-04-30 is stored

C) `ERROR 1292 (22007): Incorrect date value: '2026-04-31' for column 'due' at row 1`

D) NULL is stored with a warning

Q4

A library stores the moment each long-term loan ends, some of them in 2045. Which type fits best?

A) `TIMESTAMP`

B) `DATE`

C) `YEAR`

D) `DATETIME`

Q5

email is `NOT NULL`. An INSERT gives it the value NULL explicitly. Which error?

A) `ERROR 1364`: Field 'email' doesn't have a default value

B) `ERROR 1048`: Column 'email' cannot be null

C) `ERROR 1062`: Duplicate entry

D) `ERROR 3819`: Check constraint is violated

Q6

In which order does the SELECT list the three sizes?

```
CREATE TABLE q6 (size ENUM('small', 'medium', 'large'));
INSERT INTO q6 VALUES ('large'), ('small'), ('medium');
SELECT size FROM q6 ORDER BY size;
```

JOURNAL

Take out the table sketch from Lesson 2.1's journal. Revise every column with what you know now: its exact type and size, whether it is NOT NULL, and one CHECK or UNIQUE rule it should have. Which of your first guesses changed, and why?

2.3 CRUD Operations

BY THE END YOU CAN

- Create rows with INSERT: one row, many rows, with defaults, and read back the new id
- Read rows with SELECT: chosen columns, aliases, calculated columns and DISTINCT
- Update and delete exactly the rows you mean, using the "SELECT first" habit and safe-updates mode
- Write an upsert with INSERT ... AS new ON DUPLICATE KEY UPDATE, and explain how it differs from REPLACE and INSERT IGNORE
- Read and fix errors 1054, 1062, 1136 and 1175

KEY IDEAS

- **CRUD:** INSERT creates, SELECT reads, UPDATE changes, DELETE removes. WHERE picks the rows; without it, every row.
- **INSERT** always with a column list; several value lists in one statement; `LAST_INSERT_ID()` is the id just given (the first one after a multi-row insert).
- **SELECT** names the columns it needs; `AS` renames; calculations never change the table; `DISTINCT` shows each value once.
- **Preview, change, compare:** a SELECT with the same WHERE first; then read `Rows matched` and `Changed`.
- **Safe-updates mode** (`SET SQL_SAFE_UPDATES = 1;` , `mysql --safe-updates`) refuses UPDATE/DELETE without a key in the WHERE: error 1175.
- **SET** separates assignments with commas, never AND. AUTO_INCREMENT ids are never reused.
- **Upsert:** `INSERT ... AS new ON DUPLICATE KEY UPDATE col = new.col` (1 inserted, 2 updated, 0 unchanged). REPLACE deletes and re-inserts; INSERT IGNORE skips.

WORKED EXAMPLE: A CHANGE IN THREE STEPS

A change to several rows in three steps: preview, change with the same WHERE, check. Four customers first:

```
INSERT INTO customers (first_name, last_name, email, city, created_at) VALUES
    ('Nina', 'Park', 'nina@example.com', 'Denver', '2026-03-01 09:00:00'),
    ('Omar', 'Haddad', 'omar@example.com', 'Boston', '2026-03-02 10:30:00'),
    ('Paula', 'Reyes', 'paula@example.com', 'Denver', '2026-03-03 12:00:00'),
    ('Quinn', 'Lee', 'quinn@example.com', NULL, '2026-03-04 15:45:00');
```

Step 1, the preview: who lives in Denver?

```
SELECT id, first_name, city
FROM customers
WHERE city = 'Denver'
ORDER BY id;
```

```
+----+------------+--------+
| id | first_name | city   |
+----+------------+--------+
|  1 | Nina       | Denver |
|  3 | Paula      | Denver |
+----+------------+--------+
```

Step 2, the same WHERE in the UPDATE. Read the second line: 2 matched, the same 2 the preview showed:

```
UPDATE customers
SET city = 'Aurora'
WHERE city = 'Denver';
```

```
Query OK, 2 rows affected
Rows matched: 2  Changed: 2  Warnings: 0
```

Step 3, look at the result. Quinn's NULL city was not touched: `NULL = 'Denver'` is not true.

```
SELECT id, first_name, city
FROM customers
ORDER BY id;
```

```
+----+------------+--------+
| id | first_name | city   |
+----+------------+--------+
|  1 | Nina       | Aurora |
|  2 | Omar       | Boston |
|  3 | Paula      | Aurora |
|  4 | Quinn      | NULL   |
+----+------------+--------+
```

Clean up:

```
TRUNCATE TABLE customers;
```

◆ GUIDED LAB

- `USE bookstore;`, then SOURCE `labs/module-03/l2_3-crud-operations/new_arrivals_starter.sql`. Steps 1 and 2 run as they are.
- TODOs 3 to 6: before you turn each one on, write what you expect (which rows, how many, which id).
- Done when your results match the "Expected when done" comment.
- Finished early? `category_feed_starter.sql`, then `debug_crud_slips.sql`.

PREDICT, THEN RUN

Write your prediction first, then run the code in a practice database (`CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done, then `USE bookstore;`).

P3

Predict the output, then run it. (Delete id 3, then one more row.)

```
CREATE TABLE notes (id INT AUTO_INCREMENT PRIMARY KEY, t VARCHAR(10));
INSERT INTO notes (t) VALUES ('a'), ('b'), ('c');
DELETE FROM notes WHERE id = 3;
INSERT INTO notes (t) VALUES ('d');
SELECT * FROM notes;
```

P4

Predict the output, then run it. (An upsert that adds.)

```
CREATE TABLE tags (name VARCHAR(20) PRIMARY KEY, uses INT NOT NULL);
INSERT INTO tags VALUES ('sale', 1);
INSERT INTO tags VALUES ('sale', 1) AS new
ON DUPLICATE KEY UPDATE uses = tags.uses + new.uses;
SELECT * FROM tags;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Stock the Bookstore (lesson page, "Project: Stock the Bookstore"). All five steps in class.

- ☐ Step 1: header, USE, three TRUNCATEs
- ☐ Steps 2 and 3: 10 books, 8 customers, 5 categories, every INSERT with a column list
- ☐ Step 4: books end with 10 Fahrenheit 451; 8 customers; 5 categories
- ☐ Step 5: run twice, same result

SELF-CHECK

Q7

Why should you avoid `INSERT INTO books VALUES (NULL, 'Emma', ...);` without a column list?

A) It is slower than listing the columns

B) NULL is not allowed in an id

C) It inserts the row twice

D) It depends on the table's exact column order: an added column breaks it (1136) or shifts values silently

Q8

What does the last SELECT print?

```
CREATE TABLE q8 (id INT AUTO_INCREMENT PRIMARY KEY, t VARCHAR(10));
INSERT INTO q8 (t) VALUES ('a');
INSERT INTO q8 (t) VALUES ('b'), ('c'), ('d');
SELECT LAST_INSERT_ID();
```

Q9

An UPDATE reports `Rows matched: 0 Changed: 0 Warnings: 0`. What is most likely?

A) The WHERE found no rows: often a typo in the value or the wrong column

B) It worked: every row was changed

C) The rows already had those values

D) Safe-updates mode refused it

Q10

After `SET SQL_SAFE_UPDATES = 1;`, which statement still runs? (id is the primary key.)

A) `UPDATE books SET price = 0;`

B) `DELETE FROM books WHERE price > 10;`

C) `UPDATE books SET price = 9.99 WHERE id = 2;`

D) `DELETE FROM books;`

Q11

A category name already exists. Which statement updates its description and keeps its id and every other column?

A) `REPLACE INTO categories (name, description) VALUES (...);`

B) `INSERT ... AS new ON DUPLICATE KEY UPDATE description = new.description;`

C) `INSERT IGNORE INTO categories (name, description) VALUES (...);`

D) `DELETE` the old row, then `INSERT` the new one

Q12

What does ROW_COUNT() print?

```
CREATE TABLE q12 (name VARCHAR(20) PRIMARY KEY, note VARCHAR(30));
INSERT INTO q12 VALUES ('a', 'x'), ('b', 'y');
INSERT INTO q12 VALUES ('a', 'x'), ('b', 'z'), ('c', 'w') AS new
ON DUPLICATE KEY UPDATE note = new.note;
SELECT ROW_COUNT() AS affected;
```

JOURNAL

Think of an app you use daily (a music player, a shopping site, a chat). Write down one real action for each CRUD letter in that app, and the WHERE clause the app would need for the update and the delete. What would happen to other users if that WHERE were missing?

EXIT TICKET

Three questions, no notes.

E1

Why is money stored as DECIMAL and never as FLOAT? One sentence and one example.

E2

An UPDATE says `Rows matched: 3 Changed: 2`. What happened to the third row?

E3

What do you run before every UPDATE or DELETE that touches more than one row, and why?

NOTES

HOMEWORK CHECKLIST

- ☐ employees.sql: Step 5, a rerun, Step 6's cleanup
- ☐ load_bookstore_data.sql finished and run once more (10 books, 8 customers, 5 categories)
- ☐ Lesson 2.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.3 Hands-on Exercises 1 to 4 (Exercise 5 is a stretch)

- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Optional: `membership_rules_starter.sql`, `category_feed_starter.sql`, the two debug files; a week of `--safe-updates`
- ☐ Bring to Session 4: bookstore loaded, my two scripts, this workbook, my notes (Checkpoint 1 on Modules 1 and 2, open notes)

GLOSSARY

TERM	MEANING
data type	What kind of value a column holds (INT, DECIMAL, VARCHAR, DATE...)
strict mode	`STRICT_TRANS_TABLES` in `sql_mode`: values that do not fit are errors, not silent changes
UNSIGNED	No negative numbers; the range shifts up
DECIMAL(m, d)	An exact number: m digits in all, d after the point; the type for money
ENUM	One value from a fixed list; sorts by list position
constraint	A rule MySQL enforces for every writer: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, CHECK
CRUD	Create, Read, Update, Delete: INSERT, SELECT, UPDATE, DELETE
Rows matched / Changed	How many rows the WHERE found / how many got a new value
LAST_INSERT_ID()	The AUTO_INCREMENT id your session's last INSERT received
ROW_COUNT()	How many rows the previous statement changed
safe-updates mode	Refuses UPDATE and DELETE without a key column in the WHERE (error 1175)
upsert	Insert a row, or update it if its key already exists (`ON DUPLICATE KEY UPDATE`)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 4: Filtering, Sorting and Limiting

Your workbook for Session 4: a warm-up on Session 3, the key ideas of Lessons 3.1 and 3.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, Checkpoint 1 and your exit ticket.

Lessons 3.1-3.2 | Session 4 · 3 hours | Name: _____

SESSION GOALS

- Filter rows with comparisons, AND/OR/NOT and parentheses, IN, BETWEEN and LIKE.
- Find missing values with IS NULL, and keep or drop the NULL rows on purpose.
- Sort by several columns, expressions and custom orders, with a tie-breaker that makes the order repeatable.
- Take the top N with LIMIT, page through results with OFFSET, and know when keyset pages are better.
- Show what you can do with Modules 1 and 2 in Checkpoint 1.

WARM-UP: SESSION 3

First load the Module 3 data: `SOURCE labs/_states/flat.sql`. Then five questions, four minutes, no notes.

W1

Which type stores a book's price?

A) `FLOAT`

B) `DECIMAL(8, 2)`

C) `VARCHAR(10)`

D) `INT`

W2

An INSERT gives the NOT NULL column email the value NULL. Which error?

A) `1364`

B) `1062`

C) `1048`

D) `3819`

W3

What does the second line say after this UPDATE?

```
CREATE TABLE w3 (id INT PRIMARY KEY, price DECIMAL(6, 2));
INSERT INTO w3 VALUES (1, 9.99);
UPDATE w3 SET price = 9.99 WHERE id = 1;
```

A) `Rows matched: 0 Changed: 0`

B) `Rows matched: 1 Changed: 1`

C) `ERROR 1062`

D) `Rows matched: 1 Changed: 0`

W4

What do you run right before an UPDATE that touches several rows?

A) A SELECT with exactly the same WHERE

B) `SHOW TABLES;`

C) `SET SQL_SAFE_UPDATES = 0;`

D) Nothing: UPDATE can be undone

W5

Which statement gives an existing category a NEW id?

A) `INSERT ... AS new ON DUPLICATE KEY UPDATE ...`

B) `INSERT IGNORE ...`

C) `REPLACE INTO ...`

D) `UPDATE ... WHERE id = 3`

LESSONS

3.1 WHERE Clauses, Operators & Filtering

BY THE END YOU CAN

- Filter rows by numbers, text and dates with the comparison operators (=, !=, <, >, <=, >=)
- Combine conditions with AND, OR and NOT, and use parentheses to say exactly what you mean
- Match lists, ranges and text patterns with IN, BETWEEN and LIKE
- Find missing values with IS NULL and explain why = NULL finds nothing
- Spot the silent filtering mistakes (missing NULL rows, AND/OR mix-ups) and fix error 1054
- Save a set of reusable searches in a .sql script

KEY IDEAS

- **WHERE tests every row;** only rows where the test is true are kept. False and unknown (NULL) are dropped.
- **Comparisons** `= != < > <= >=` work on numbers, dates ('YYYY-MM-DD') and text (case-insensitive with _ci collations).
- **AND narrows, OR widens, NOT flips.** AND is evaluated before OR: when you mix them, add parentheses.
- **IN** a list · **BETWEEN** an inclusive range · **LIKE** a pattern (% any characters, _ exactly one).
- **NULL is unknown:** test it with IS NULL / IS NOT NULL, never = NULL. `!=`, NOT IN and NOT LIKE silently drop NULL rows: add `OR col IS NULL` when you want them. Never a NULL inside NOT IN.
- **Date-and-time columns:** a half-open range, `>= start AND < next_start`.
- **Count first:** say how many rows you expect before you run a filter.

WORKED EXAMPLE: A FLYER LIST, BUILT AND BROKEN

A flyer for customers who joined in January and live in New York or Chicago, plus anyone whose city is unknown (the shop will ask them). First, every condition as a column, to see each row's answers:

```
SELECT first_name, city, created_at,
       created_at < '2026-02-01'          AS january,
       city IN ('New York', 'Chicago')    AS east_midwest
FROM customers
ORDER BY id;
```

```
+------------+-------------+---------------------+---------+--------------+
| first_name | city        | created_at          | january | east_midwest |
+------------+-------------+---------------------+---------+--------------+
| Alice      | New York    | 2026-01-04 10:15:00 |       1 |            1 |
| Bob        | Chicago     | 2026-01-06 14:02:00 |       1 |            1 |
| Carol      | Los Angeles | 2026-01-09 09:40:00 |       1 |            0 |
| Dave       | Houston     | 2026-01-12 16:25:00 |       1 |            0 |
| Eve        | NULL        | 2026-01-20 11:08:00 |       1 |         NULL |
| Frank      | New York    | 2026-01-27 13:30:00 |       1 |            1 |
| Grace      | Chicago     | 2026-02-02 08:55:00 |       0 |            1 |
| Hank       | NULL        | 2026-02-10 19:12:00 |       0 |         NULL |
+------------+-------------+---------------------+---------+--------------+
```

Now the filter, with the OR in parentheses:

```sql
SELECT first_name, city
FROM customers
WHERE created_at < '2026-02-01'
  AND (city IN ('New York', 'Chicago') OR city IS NULL)
ORDER BY first_name;
```

```
+------------+----------+
| first_name | city     |
+------------+----------+
| Alice      | New York |
| Bob        | Chicago  |
| Eve        | NULL     |
| Frank      | New York |
+------------+----------+
```

The same conditions without the parentheses. AND runs first, so `OR city IS NULL` stands on its own and lets Hank in, although he joined in February:

```sql
SELECT first_name, city
FROM customers
WHERE created_at < '2026-02-01'
  AND city IN ('New York', 'Chicago') OR city IS NULL
ORDER BY first_name;
```

```
+------------+----------+
| first_name | city     |
+------------+----------+
| Alice      | New York |
| Bob        | Chicago  |
| Eve        | NULL     |
| Frank      | New York |
| Hank       | NULL     |
+------------+----------+
```

GUIDED LAB

- `USE bookstore;` (Module 3 data), then SOURCE `labs/module-04/l3_1-where-clauses-operators-filtering/staff_requests_starter.sql`. Seven Empty sets.
- For each request: write how many rows you expect, then replace FALSE with the condition.

REQUEST	ROWS I EXPECT	ROWS I GOT
1		
2		
3		
4		
5		
6		
7		

- Finished early? `null_detective_starter.sql`, then `debug_filters.sql`.

PREDICT, THEN RUN

Write your prediction first (a row count at least), then run it inside bookstore with the Module 3 data.

P1

Predict the output, then run it. (Two cities left out of eight customers.)

```
SELECT COUNT(*) AS n
FROM customers
WHERE city NOT IN ('Chicago', 'Houston');
```

P2

Predict the output, then run it. (Two o's somewhere in the title.)

```
SELECT title
FROM books
WHERE title LIKE '%o%o%'
ORDER BY title;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Saved Searches (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 7.

- ☐ Steps 1 to 3: gift ideas, long reads, mid-century shelf
- ☐ Steps 4 to 6: title search, follow-up list (IS NULL), flyer (half-open range)
- ☐ Step 7: SOURCE the whole file

SELF-CHECK

Q1

Two of the 8 customers live in Chicago and two have no city. How many rows does this return?

```
SELECT * FROM customers WHERE city != 'Chicago';
```

A) 6

B) 4

C) 8

D) 0

Q2

On the Module 3 data, what does this print?

```
SELECT COUNT(*) AS n
FROM books
WHERE price BETWEEN 8.99 AND 9.99;
```

Q3

Which WHERE keeps every book NOT written by Tolkien or Orwell?

A) `WHERE author NOT IN ('J.R.R. Tolkien', 'George Orwell')`

B) `WHERE author != 'J.R.R. Tolkien' OR author != 'George Orwell'`

C) `WHERE NOT author = 'J.R.R. Tolkien' OR 'George Orwell'`

D) `WHERE author NOT LIKE 'Tolkien, Orwell'`

Q4

created_at is a DATETIME. Which WHERE finds the customers who signed up on January 27, 2026 (Frank, at 13:30)?

A) `WHERE created_at = '2026-01-27'`

B) `WHERE created_at BETWEEN '2026-01-27' AND '2026-01-27'`

C) `WHERE created_at >= '2026-01-27' AND created_at < '2026-01-28'`

D) `WHERE created_at LIKE 2026-01-27`

Q5

Which first names does this return, in order?

```
SELECT first_name
FROM customers
WHERE city = 'New York' OR city = 'Chicago' AND first_name LIKE 'G%'
ORDER BY first_name;
```

Q6

Which of these returns no rows at all?

A) `WHERE city IS NULL`

B) `WHERE city <=> NULL`

C) `WHERE city LIKE '%o%'`

D) `WHERE city NOT IN ('Reno', NULL)`

JOURNAL

Four of this lesson's mistakes return wrong rows with no error at all. Write down how you would notice each one if nobody told you, for example by counting the rows you expect before you run the query. What habit will you build so a silent mistake does not slip into a report?

3.2 Sorting & Limiting Results

BY THE END YOU CAN

- Sort results with ORDER BY on one or more columns or expressions, each ascending or descending
- Predict where NULL values land in a sort and move them to the end
- Take the top N rows with LIMIT, and add a tie-breaker so the result is always the same
- Split a result into pages with LIMIT ... OFFSET, and explain when keyset pagination is better
- Read and fix the errors MySQL raises for misplaced or malformed ORDER BY and LIMIT clauses

KEY IDEAS

- **A table has no order:** without ORDER BY, row order is not guaranteed.
- **ORDER BY** is ascending by default; DESC belongs only to the column before it. Extra columns break ties.
- **Sort by expressions, aliases and CASE ranks;** avoid column numbers in saved queries.
- **NULLs** sort first ascending, last descending; `ORDER BY col IS NULL, col` puts them last (MySQL has no NULLS LAST).
- **LIMIT n OFFSET m** skips m rows and keeps n; it runs last. Page N = OFFSET (N - 1) × size, computed before the query.
- **Every ORDER BY that feeds a LIMIT ends with a unique column.** Count pages with a separate COUNT(*).
- **Keyset pagination** (`WHERE id > last_seen ORDER BY id LIMIT n`) stays fast on huge tables.

WORKED EXAMPLE: PAGES OF RECENT BOOKS, TWO WAYS

"Recently published" in pages of three, newest first, with id as the tie-breaker. Page 1:

```
SELECT title, published
FROM books
ORDER BY published DESC, id
LIMIT 3;
```

```
+-----------------------+------------+
| title                 | published  |
+-----------------------+------------+
| Dune                  | 1965-08-01 |
| To Kill a Mockingbird | 1960-07-11 |
| Lord of the Rings     | 1954-07-29 |
+-----------------------+------------+
```

Page 2 the keyset way: the rows published before the last one we showed (Lord of the Rings, 1954-07-29):

```
SELECT title, published
FROM books
WHERE published < '1954-07-29'
ORDER BY published DESC, id
LIMIT 3;
```

```
+------------------------+------------+
| title                  | published  |
+------------------------+------------+
| Fahrenheit 451         | 1953-10-19 |
| The Catcher in the Rye | 1951-07-16 |
| 1984                   | 1949-06-08 |
+------------------------+------------+
```

And the OFFSET way. Same rows; the keyset version stays fast however deep the pages go:

```
SELECT title, published
FROM books
ORDER BY published DESC, id
LIMIT 3 OFFSET 3;
```

```
+------------------------+------------+
| title                  | published  |
+------------------------+------------+
| Fahrenheit 451         | 1953-10-19 |
| The Catcher in the Rye | 1951-07-16 |
| 1984                   | 1949-06-08 |
+------------------------+------------+
```

GUIDED LAB

- SOURCE `labs/module-04/l3_2-sorting-limiting-results/bestseller_board_starter.sql`. Every list is in id order.
- Replace each `ORDER BY id` with the order the comment asks for. Could two rows tie? Add a tie-breaker.
- Finished early? `paging_starter.sql`, then `debug_sorting.sql`.

PREDICT, THEN RUN

Write your prediction first (a row count at least), then run it inside bookstore with the Module 3 data.

P3

Predict the output, then run it. (Z to A, then three rows.)

```
SELECT first_name, city
FROM customers
ORDER BY city DESC, first_name
LIMIT 3;
```

P4

Predict the output, then run it. (Shortest first, skip eight.)

```
SELECT title, pages
FROM books
ORDER BY pages
LIMIT 2 OFFSET 8;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Storefront Queries (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Steps 1 and 2: new and notable (with a tie-breaker), bargain shelf
- ☐ Steps 3 and 4: catalog pages 1 to 3 and the page count
- ☐ Step 5: customer directory, missing cities last
- ☐ Step 6: run it; every LIMIT has an ORDER BY that cannot tie

SELF-CHECK

Q7

What does `ORDER BY in_stock, price DESC` do?

A) in_stock ascending (0 first), then price descending within each group

B) Both columns descending

C) `ERROR 1064`: DESC must come after every column

D) price first, then in_stock

Q8

What does this print?

```
SELECT title
FROM books
ORDER BY pages DESC
LIMIT 1 OFFSET 2;
```

Q9

What happens with `'price'` in quotes: `ORDER BY 'price' DESC LIMIT 3`?

A) The three most expensive books, as intended

B) `ERROR 1054`: Unknown column

C) No error, but the rows come in no particular order: 'price' is a fixed piece of text

D) The rows are sorted alphabetically by title

Q10

Where do Eve and Hank (no city) appear with `ORDER BY city DESC, first_name`?

A) First

B) They are left out

C) `ERROR 1064`

D) Last

Q11

A list sorted by id shows 20 rows per page, and page 1 ended at id 57. Which query gets the next page fast, even on a huge table?

A) `... ORDER BY id LIMIT 20 OFFSET 57`

B) `... WHERE id > 57 ORDER BY id LIMIT 20`

C) `... WHERE id >= 57 LIMIT 20`

D) `... ORDER BY id LIMIT 57, 20`

Q12

With 3 books per page, how many pages? What does this print?

```
SELECT CEILING(COUNT(*) / 3) AS pages FROM books;
```

JOURNAL

Think of a website or app you use every day (a shop, a news feed, your email). Describe one of its lists as SQL in plain words: what is it sorted by, what breaks ties, how many rows per page, and does it look like OFFSET pages or "load more" keyset pages? What would go wrong for users if it had no tie-breaker?

CHECKPOINT 1: MODULES 1 AND 2

30 minutes, open notes (this workbook, your scripts, the lesson pages), on your own. Parts A to C go into `labs/module-04/checkpoint-1/checkpoint_starter.sql`; SOURCE it after each part. Part D is answered on this page.

The task. The bookstore is opening a used-book trade-in desk: people bring books, the desk makes an offer, and pays later.

PART A · BUILD (RERUNNABLE)

- The file already drops and creates `kit_cp1_tradein` with utf8mb4 and utf8mb4_unicode_ci, and USEs it.
- **sellers:** an id MySQL numbers; a name (up to 80 characters, required); an email (up to 100, required, never shared by two sellers); a city (up to 50, may be unknown).
- **trade_ins:** an id MySQL numbers; `seller_id` (a whole number, required); a title (up to 200, required); `book_condition`: like new, good or worn, good unless given; an offer (money, required, from 0.50 to 20.00, enforced by a CHECK named `chk_offer`); `received` (a date, required); `paid` (yes/no, no unless given).

PART B · FILL AND CHANGE

NAME	EMAIL	CITY
Mo Rahman	mo@example.com	Reno
Ivy Chen	ivy@example.com	(unknown)
Sam Okoro	sam@example.com	Sparks

SELLER_ID	TITLE	CONDITION	OFFER	RECEIVED
1	Dune	good	4.00	2026-05-01
1	Emma	worn	1.50	2026-05-01
2	The Hobbit	like new	6.00	2026-05-03
3	1984	good	3.50	2026-05-04
3	Dracula	worn	1.00	2026-05-04

1. B1 and B2: insert the sellers and the trade-ins above (one INSERT each; leave `paid` out).
2. B3: Dune's offer (trade-in 1) goes up by one dollar.
3. B4: Mo (seller 1) is paid for all his books: preview, update, then show how many rows changed.
4. B5: Dracula (trade-in 5) was turned down: remove it.

5. B6: Ivy sends her details again (Ivy Chen, ivy@example.com, Henderson): ONE statement that inserts her, or updates her city if her email is already there.

PART C · READ

1. C1: every trade-in by id: id, title, condition, offer, a column `store_credit` (the offer plus 10%, rounded to cents) and paid.
2. C2: the conditions that occur, each once.
3. C3: the sellers by id: id, name, city.

PART D · ON PAPER: WHAT DOES MYSQL ANSWER, AND WHY?

Each statement below is run after your finished Parts A and B. Write the error number, the rule it protects, and a fix.

D1

The desk offers 25.00 for a rare copy of Emma.

```
INSERT INTO trade_ins (seller_id, title, offer, received)
VALUES (1, 'Emma', 25.00, '2026-05-02');
```

D2

Mo comes back with a new name on his card, same email.

```
INSERT INTO sellers (name, email)
VALUES ('Mo R.', 'mo@example.com');
```

D3

Ivy's Emma is in "mint" condition.

```
INSERT INTO trade_ins (seller_id, title, book_condition, offer, received)
VALUES (2, 'Emma', 'mint', 3.00, '2026-05-02');
```

EXIT TICKET

Three questions, no notes.

E1

Why does `WHERE city != 'Chicago'` skip Eve and Hank, and how do you include them?

E2

Write the WHERE clause for: books from $9 to $11, both ends included, that are not by Tolkien.

E3

Why should every ORDER BY that feeds a LIMIT end with a unique column?

NOTES

HOMEWORK CHECKLIST

- ☐ saved_searches.sql Steps 4 to 7
- ☐ storefront.sql Steps 3 to 6
- ☐ Lesson 3.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 3.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes; both journal entries
- ☐ Optional: `null_detective_starter.sql`, `paging_starter.sql`, the two debug files
- ☐ Bring to Session 5: bookstore with the Module 3 data, my two scripts, this workbook

GLOSSARY

TERM	MEANING
WHERE	A test run on every row; only rows where it is true are kept
three-valued logic	A condition is true, false or unknown (NULL); unknown is dropped
IN / BETWEEN / LIKE	A list / an inclusive range / a pattern (% any characters, _ one)
IS NULL	The only correct test for a missing value
<=>	MySQL's NULL-safe equals: two NULLs count as equal
half-open range	`>= start AND < next_start`: never misses or double-counts a timestamp
precedence	The order operators are evaluated: comparisons, NOT, AND, then OR
sort key / tie-breaker	A column in ORDER BY / an extra, unique one that decides ties
LIMIT / OFFSET	How many rows to keep / how many to skip first
keyset pagination	Ask for the rows after the last one seen (`WHERE id > 57`) instead of skipping

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 5: Aggregates and Subqueries

Your workbook for Session 5: the warm-up, the key ideas of Lessons 3.3 and 3.4, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, the error clinic and your exit ticket.

Lessons 3.3–3.4 | Session 5 · 3 hours | Name: _____

SESSION GOALS

- Summarize a table in one row with COUNT, SUM, AVG, MIN and MAX, and predict what each does with NULL.
- Make one summary row per group with GROUP BY, keep or drop whole groups with HAVING, and sort the result.
- Read and fix errors 1140, 1055 and 1111 (and 1064 when HAVING comes too early).
- Answer two-step questions in one statement with a subquery: above average, the row behind the maximum, a list for IN.
- Spot the NOT IN / NULL trap, and write correlated subqueries, NOT EXISTS checks and derived tables.

WARM-UP: LESSONS 3.1 AND 3.2

ANSWER WITHOUT RUNNING ANYTHING, THEN CHECK ON THE PROJECTOR

W1

Who comes back, and why?

```
SELECT first_name, city, id
FROM customers
WHERE city = 'Chicago' OR city = 'Houston' AND id > 5
ORDER BY id;
```

W2

Two customers have no city. Why does this find nobody? What would you write instead?

```
SELECT first_name FROM customers WHERE city = NULL;
```

W3

In which order must you write `WHERE`, `ORDER BY` and `LIMIT`?

W4

Which three books does this show?

```
SELECT title, price
FROM books
ORDER BY price DESC, title
LIMIT 3 OFFSET 2;
```

W5

Predict: how many customers live outside Chicago, according to this query?

```
SELECT first_name
FROM customers
WHERE city NOT IN ('Chicago', NULL);
```

LESSONS

3.3 Aggregate Functions & GROUP BY

BY THE END YOU CAN

- Summarize a table with COUNT, SUM, AVG, MIN and MAX, and predict how each one treats NULL
- Produce one summary row per group with GROUP BY, sorted with ORDER BY
- Keep or drop whole groups with HAVING, and explain when to use WHERE instead
- Read and fix the errors MySQL raises for broken aggregate queries (1140, 1055, 1111)
- Build a small inventory report as a reusable .sql script

KEY IDEAS

- **An aggregate turns many rows into one value:** COUNT, SUM, AVG, MIN, MAX.
- `COUNT(*)` **counts rows; COUNT(col) counts filled-in values; COUNT(DISTINCT col) counts different values.**
- **Every aggregate skips NULL** (except `COUNT(*)`). Over no rows, COUNT gives 0 and the others NULL.
- **AVG of a DECIMAL prints extra decimals:** show money with `ROUND(AVG(price), 2)`.
- **MIN and MAX give the value, not the row.** To see the row, sort and LIMIT (or, after the break, a subquery).
- **GROUP BY makes piles; the aggregate runs once per pile.** Every selected column is grouped or aggregated (1140, 1055).
- **WHERE picks rows before the piles; HAVING picks piles after.** An aggregate in WHERE is 1111.
- **GROUP BY does not sort** on MySQL 8.0 and later: always add ORDER BY.

WORKED EXAMPLE: WHERE REMOVES ROWS, HAVING REMOVES PILES

Three small summaries that show which clause does which job. First, one pile per author, kept only when the pile has more than one book, with the first and latest publication date of each kept pile:

```
SELECT author,
       COUNT(*)       AS books,
       MIN(published) AS first_book,
       MAX(published) AS latest_book
FROM books
GROUP BY author
HAVING COUNT(*) > 1;
```

```
+----------------+-------+------------+-------------+
| author         | books | first_book | latest_book |
+----------------+-------+------------+-------------+
| J.R.R. Tolkien |     2 | 1937-09-21 | 1954-07-29  |
+----------------+-------+------------+-------------+
```

Tolkien is the only author with two books. Now a condition on single rows: WHERE keeps the books over $10 BEFORE they are piled up, so the out-of-stock pile never forms (both out-of-stock books cost 8.99):

```sql
SELECT in_stock, COUNT(*) AS books_over_10
FROM books
WHERE price > 10
GROUP BY in_stock
ORDER BY in_stock DESC;
```

```
+----------+---------------+
| in_stock | books_over_10 |
+----------+---------------+
|        1 |             5 |
+----------+---------------+
```

And a condition on whole piles: every book goes into a pile, then HAVING drops the pile that is too small:

```sql
SELECT in_stock, COUNT(*) AS books
FROM books
GROUP BY in_stock
HAVING COUNT(*) > 5
ORDER BY in_stock DESC;
```

```
+----------+-------+
| in_stock | books |
+----------+-------+
|        1 |     8 |
+----------+-------+
```

Same table, two different missing rows, for two different reasons: WHERE removed rows, HAVING removed a pile.

GUIDED LAB

- SOURCE `labs/module-05/l3_3-aggregate-functions/customer_summary_starter.sql`. Every query runs; the placeholders show 0, NULL or a TODO line.
- TODO 1 together, TODOs 2 to 4 in pairs. Save, SOURCE again, compare with the "Expected when done"

comment at the end of the file.

- For TODO 4 write one sentence: which line picks the rows, and which line makes the piles?

- Finished early? `page_report_starter.sql`, then `debug_group_by.sql` in the same folder.
- Data look wrong (11 books, no author column)? `SOURCE labs/_states/flat.sql` and run again.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code in the flat bookstore. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Customers 5 to 8: Eve and Hank have no city.)

```
SELECT COUNT(*), COUNT(city), COUNT(DISTINCT city)
FROM customers
WHERE id > 4;
```

P2

Predict the output, then run it. (The out-of-stock books have 180 and 234 pages.)

```
SELECT in_stock, MAX(pages) AS longest
FROM books
GROUP BY in_stock
HAVING MAX(pages) > 300
ORDER BY in_stock;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Inventory Report (lesson page, "Project: The Inventory Report"): `inventory_report.sql`. In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: `USE bookstore;` and the headline row (10 titles, 8 in stock, 3824 pages, 10.99)
- ☐ Step 2: stock value by status (In stock / Out of stock)
- ☐ Step 3: price bands with CASE, sorted by MIN(price)
- ☐ Step 4: authors for the display table (HAVING with OR): exactly three
- ☐ Step 5: customers by city with `(no city)`
- ☐ Step 6: SOURCE runs the whole file without an error

SELF-CHECK

Q1

Grace (Chicago) and Hank (no city) are the only customers who signed up in February. What does this print?

```
SELECT COUNT(*), COUNT(city)
FROM customers
WHERE created_at >= '2026-02-01';
```

A) `2` and `2`

B) `2` and `1`

C) `1` and `1`

D) `8` and `6`

Q2

You want only the authors whose books average more than $10. Where does that condition go?

A) In `WHERE`, before `GROUP BY`

B) In `ORDER BY`

C) In `HAVING`, after `GROUP BY`

D) In the `SELECT` list, as `AVG(price) > 10`

Q3

The flat bookstore has no books by Isaac Asimov. What does this print?

```
SELECT COUNT(*) AS n, MAX(price) AS top
FROM books
WHERE author = 'Isaac Asimov';
```

Q4

What happens when this runs on MySQL 9.7?

```
SELECT city, first_name
FROM customers
GROUP BY city;
```

A) `ERROR 1055 (42000)`: first_name is not in GROUP BY and not aggregated

B) One row per city, each with one customer's first name

C) `ERROR 1140`: an aggregated query without GROUP BY

D) One row per customer, sorted by city

Q5

A colleague says: "I don't need `ORDER BY` after `GROUP BY city`: the cities always come out sorted." On MySQL 9.7, is that right?

A) Yes: GROUP BY sorts the groups by the grouped column

B) Yes, but only when the column has an index

C) Only on Windows servers

D) No: since MySQL 8.0 the order of groups is not guaranteed; add ORDER BY

Q6

What does this print? (Under $10: three books in stock and two out of stock.)

```
SELECT in_stock, COUNT(*) AS books
FROM books
WHERE price < 10
GROUP BY in_stock
ORDER BY in_stock;
```

JOURNAL

Explain the difference between WHERE and HAVING with an example from everyday life (sorting laundry, a class's test scores, a team's results). Which step is "pick the items", which is "make the piles", and which is "keep only some piles"?

3.4 Subqueries & Nested Queries

BY THE END YOU CAN

- Compare every row with an aggregate using a scalar subquery ("above average", "equal to the maximum")
- Filter with IN, NOT IN, ANY and ALL subqueries, and keep NULLs from emptying a NOT IN
- Write correlated subqueries and EXISTS / NOT EXISTS checks that compare a row with related rows
- Build derived tables in FROM and computed columns in SELECT
- Read and fix the subquery errors 1242, 1241, 1248 and 1235

KEY IDEAS

- **A subquery is a SELECT in parentheses inside another statement.** The inner query answers first; read from the inside out.
- **Scalar subquery = one value:** works with =, >, arithmetic and in the SELECT list. `=` with several rows is 1242.
- **A one-column list goes with IN, ANY, ALL.** Two columns in the list is 1241.
- **One NULL in a NOT IN list empties the result.** Add `IS NOT NULL` inside, or use NOT EXISTS.
- **Correlated = refers to the outer row.** Alias both copies of the table and qualify every column inside.
- **EXISTS asks "is there at least one row?"** `SELECT 1` inside says the columns do not matter.
- **A derived table (subquery in FROM) needs an alias** (1248) and allows a summary of a summary.
- **No LIMIT inside an IN subquery** (1235): wrap it in a derived table.

WORKED EXAMPLE: A CORRELATED SUBQUERY IN TWO RUNS

Build a correlated subquery in two runs: the latest customer to sign up in each city. Run 1, the inner query for ONE city, with the city typed in:

```
SELECT MAX(created_at) FROM customers WHERE city = 'Chicago';
```

```
+---------------------+
| MAX(created_at)     |
+---------------------+
| 2026-02-02 08:55:00 |
+---------------------+
```

It works for Chicago. Run 2: replace the typed city with the outer row's city, and give both copies of the table an alias so MySQL knows which one each column belongs to:

```
SELECT c.first_name, c.city, c.created_at
FROM customers AS c
WHERE c.created_at = (
    SELECT MAX(c2.created_at)
    FROM customers AS c2
    WHERE c2.city = c.city
)
ORDER BY c.city;
```

```
+------------+-------------+---------------------+
| first_name | city        | created_at          |
+------------+-------------+---------------------+
| Grace      | Chicago     | 2026-02-02 08:55:00 |
| Dave       | Houston     | 2026-01-12 16:25:00 |
| Carol      | Los Angeles | 2026-01-09 09:40:00 |
| Frank      | New York    | 2026-01-27 13:30:00 |
+------------+-------------+---------------------+
```

For every customer c, the inner query asks "what is the latest signup in c's city?", and the row is kept if it is c's own date. Grace is the latest in Chicago; Frank in New York. Eve and Hank never appear: their city is NULL, and NULL = NULL is not true.

GUIDED LAB

- `labs/module-05/l3_4-subqueries/page_checks_starter.sql`: replace each hand-typed placeholder with a subquery, so no number in the file is typed by hand.
- `city_neighbors_starter.sql`: correlated subqueries, a derived table, and a NOT IN that prints nothing until you find the NULL.
- Build each subquery inner query first: run it on its own for one example value, then connect it to the outer row.
- Finished early? `debug_subqueries.sql`: four broken subqueries, one per shape.

SHAPE THE BLANK EXPECTS	WHERE	ERROR WHEN IT GETS THE WRONG SHAPE
one value	after `=`, `>`, `<`, in the SELECT list	1242 (more than one row)
one column, any rows	after `IN`, `ANY`, `ALL`	1241 (more than one column)
a table with a name	in FROM	1248 (no alias)

PREDICT, THEN RUN

Write your prediction on the line first, then run the code in the flat bookstore. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (The average book has 382.4 pages.)

```
SELECT COUNT(*) AS long_books
FROM books
WHERE pages > (SELECT AVG(pages) FROM books);
```

P4

Predict the output, then run it. (Customer 5 is Eve, who has no city.)

```
SELECT first_name
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id = 5);
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Price Review (lesson page, "Project: The Price Review"): `price_review.sql`. In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: books above the store average, with the average and the difference
- ☐ Step 2: each author's entry-level book (correlated; inner query first for Tolkien)
- ☐ Step 3: authors fully in stock with NOT EXISTS (seven)
- ☐ Step 4: the top price tier through a derived table
- ☐ Step 5: the headline numbers (10.71 and 16.4)
- ☐ Step 6: run it, change a price inside a transaction, run it again, ROLLBACK

SELF-CHECK

Q7

The average price is 10.99. What does this print?

```
SELECT COUNT(*)
FROM books
WHERE price < (SELECT AVG(price) FROM books);
```

A) `4`

B) `6`

C) `7`

D) `10`

Q8

What happens when this runs?

```
SELECT title
FROM books
WHERE price = (SELECT price FROM books
               WHERE author = 'J.R.R. Tolkien');
```

A) `ERROR 1242 (21000): Subquery returns more than 1 row`

B) It returns Lord of the Rings

C) `ERROR 1241 (21000): Operand should contain 1 column(s)`

D) `ERROR 1248 (42000): Every derived table must have its own alias`

Q9

Customer 5 (Eve) has no city; customer 6 (Frank) lives in New York. What does this print?

```
SELECT COUNT(*)
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id IN (5, 6));
```

Q10

You need the customers whom no other customer shares a city with, and city can be NULL. Which approach gives a predictable answer with no NULL trap?

A) `city NOT IN (SELECT city FROM customers ...)`

B) `city != ALL (SELECT city ...)`

C) `NOT EXISTS (SELECT 1 FROM customers AS other WHERE other.city = c.city AND other.id != c.id)`

D) `city NOT LIKE (SELECT city ...)`

Q11

What does this print? (Two customers have no city.)

```
SELECT MAX(n) AS biggest_group
FROM (
    SELECT city, COUNT(*) AS n
    FROM customers
    GROUP BY city
) AS per_city;
```

Q12

This was meant to list the books priced above THEIR OWN author's average. Why does it return three books instead of one?

```
SELECT title, author, price
FROM books
WHERE price > (
    SELECT AVG(price)
    FROM books
    WHERE author = author
)
ORDER BY price;
```

A) The subquery runs once per author, but AVG ignores the out-of-stock books

B) MySQL treats the second author as a string literal

C) HAVING is missing from the subquery

D) Both author names refer to the inner table, so author = author is always true and the subquery returns the store average

◆ JOURNAL

Pick the correlated subquery from Step 2 of the project and explain it in plain words to someone who has never seen SQL, as if you were the database walking through the books one at a time. Then write down the strategy you used to build it (inner query first? outer first?) and whether it worked for you.

ERROR CLINIC

Your pair's file: ☐ `debug_group_by.sql` ☐ `debug_subqueries.sql`. For each ERROR line: the number, what MySQL could not decide, your fix.

ERROR NUMBER	WHAT MYSQL COULD NOT DECIDE	MY FIX

EXIT TICKET

Three questions, no notes. Hand them in or keep them: they tell you and your instructor what to revisit.

E1

Write one condition that belongs in WHERE and one that belongs in HAVING, for the books table.

E2

A NOT IN subquery returns `('Chicago', NULL)`. What does the outer query return, and what are two fixes?

E3

In your own words: what makes a subquery correlated, and why does it need aliases?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the inventory report (Steps 3 to 6) and the price review (Steps 3 to 6); both run with SOURCE, no errors
- ☐ Lesson 3.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 3.4 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages

- ☐ Both journal entries
- ☐ Labs not finished in class: `page_report_starter.sql`, `city_neighbors_starter.sql`, the debug handouts
- ☐ Save `bookstore.sql` in my course folder (do not run it yet); keep the flat bookstore loaded
- ☐ Bring to Session 6: the flat bookstore, `bookstore.sql`, my two project scripts, this workbook and my journal

GLOSSARY

TERM	MEANING
aggregate function	COUNT, SUM, AVG, MIN, MAX: many rows in, one value out
GROUP BY	Sorts rows into piles that share a value; the aggregates run once per pile
HAVING	Keeps or drops whole groups, after grouping (WHERE filters rows before)
ONLY_FULL_GROUP_BY	MySQL's default rule: every selected column is grouped or aggregated (errors 1140, 1055)
COALESCE(a, b)	The first value that is not NULL: a label for the NULL group, 0 instead of a NULL sum
subquery	A SELECT in parentheses inside another statement (the inner query)
scalar subquery	A subquery that returns exactly one value
correlated subquery	A subquery that refers to the outer query's current row; re-evaluated per row
EXISTS / NOT EXISTS	True when the subquery returns at least one row / no row
derived table	A subquery in FROM, read like a table; it must have an alias
ANY / ALL	Compare with a list: beats at least one item / beats every item

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 6: Relationships and Foreign Keys

Your workbook for Session 6: the warm-up, the key ideas of Lessons 4.1 and 4.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, the relationship map and your exit ticket.

Lessons 4.1–4.2 | Session 6 · 3 hours | Name: _____

SESSION GOALS

- Name the anomalies of one big table and explain why each fact belongs in one table, once.
- Place the foreign key for one-to-many, one-to-one and many-to-many relationships, and build a junction table.
- Move a repeated text column into its own table safely: add, fill, check, lock in.
- Predict errors 1452 and 1451, and choose RESTRICT, CASCADE or SET NULL for a key, with a reason.
- Add a key to a table that already holds data, and read every rule from information_schema.

WARM-UP: LESSONS 3.3 AND 3.4

ANSWER WITHOUT RUNNING ANYTHING, THEN CHECK ON THE PROJECTOR

W1

Three numbers: predict them. (Customers 1 to 5 signed up before 25 January; Eve has no city.)

```
SELECT COUNT(*), COUNT(city), COUNT(DISTINCT city)
FROM customers
WHERE created_at < '2026-01-25';
```

W2

`price > 10` and `COUNT(*) > 1`: which goes in WHERE, and which in HAVING?

W3

Which book is it?

```
SELECT title
FROM books
WHERE pages = (SELECT MAX(pages) FROM books);
```

W4

How many customers? (Customer 7 is Grace in Chicago, 8 is Hank with no city.)

```
SELECT COUNT(*)
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id > 6);
```

W5

What is wrong, and what is the error number?

```
SELECT author, title, MAX(price)
FROM books
GROUP BY author;
```

LESSONS

4.1 Table Relationships

BY THE END YOU CAN

- Explain the update, insert and delete problems of a flat table, using the bookstore's own data
- Tell one-to-many, one-to-one and many-to-many relationships apart, and say where the foreign key goes in each
- Move a repeated text column into its own table with CREATE TABLE, ALTER TABLE and UPDATE ... JOIN
- Build a junction table with a composite primary key, with and without extra columns
- Read a crow's foot ER diagram of the finished bookstore database

KEY IDEAS

- **Store each fact once**, in the table it describes; other tables point at it by id. Repeated text causes update, insert and delete anomalies.
- **Primary key** identifies a row; **foreign key** holds another table's primary key. The table with the id is the parent, the one with the pointer the child.
- **Can one A have many Bs?** Yes: the foreign key goes in B (the many side). Two yeses: a junction table.
- **One-to-one** = a foreign key that is also UNIQUE (or is the primary key).
- **Many-to-many** = a junction table: two foreign keys and a composite primary key; it may hold facts about the link (quantity, price_at_time, joined_on).
- **Migrate safely:** add the column, fill it (`UPDATE ... JOIN`), COUNT the NULLs, then NOT NULL, drop the old column, declare the key.
- **Create parents first, children second; drop in the opposite order** (1824 for a missing parent table).

WORKED EXAMPLE: A MANY-TO-MANY FROM SCRATCH

A small many-to-many from scratch, in a practice database of its own: members, clubs, and the memberships that link them (with one fact about each link, the date joined).

```
DROP DATABASE IF EXISTS kit_clubs;
CREATE DATABASE kit_clubs;
USE kit_clubs;
CREATE TABLE members (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);
CREATE TABLE clubs (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE memberships (
    member_id INT NOT NULL,
    club_id   INT NOT NULL,
    joined_on DATE NOT NULL,
    PRIMARY KEY (member_id, club_id),
    CONSTRAINT fk_ms_member FOREIGN KEY (member_id) REFERENCES members(id) ON DELE
    CONSTRAINT fk_ms_club   FOREIGN KEY (club_id)   REFERENCES clubs(id)   ON DELE
);
INSERT INTO members (name) VALUES ('Ana'), ('Ben'), ('Cleo');
INSERT INTO clubs (name) VALUES ('Chess'), ('Hiking'), ('Book club');
INSERT INTO memberships VALUES
    (1, 1, '2026-01-10'), (1, 3, '2026-02-01'),
    (2, 2, '2026-01-15'), (3, 2, '2026-03-05'), (3, 3, '2026-03-05');
```

```
Database changed
```

Which clubs is Cleo (member 3) in? Follow the links with a subquery:

```
SELECT name
FROM clubs
WHERE id IN (SELECT club_id FROM memberships WHERE member_id = 3)
ORDER BY name;
```

```
+-----------+
| name      |
+-----------+
| Book club |
| Hiking    |
+-----------+
```

How many members does each club have? Count the links per club:

```
SELECT
    name,
    (SELECT COUNT(*) FROM memberships AS ms WHERE ms.club_id = clubs.id) AS member
FROM clubs
ORDER BY members DESC, name;
```

```
+-----------+---------+
| name      | members |
+-----------+---------+
| Book club |       2 |
| Hiking    |       2 |
| Chess     |       1 |
+-----------+---------+
```

Ana is already in Chess. The composite primary key stores each pair once:

```
INSERT INTO memberships VALUES (1, 1, '2026-04-01');
```

```
ERROR 1062 (23000): Duplicate entry '1-1' for key 'memberships.PRIMARY'
```

Clean up:

```
DROP DATABASE kit_clubs;
```

Two one-to-many relationships meeting in the middle: one member, many memberships; one club, many memberships. The junction row is also the right home for `joined_on`, which belongs to neither the member nor the club.

GUIDED LAB

- SOURCE `labs/module-06/l4_1-table-relationships/publisher_split_starter.sql`. Step 1 shows the problems in the flat editions table.
- Turn on Steps 2 to 6 one at a time (remove the `--` from the statement lines, fill the `___` gaps), SOURCE again after each.
- Step 4 is the check before anything is dropped. Write down what it found, and why:

- Finished early (after loading `bookstore.sql`)? `navigate_links_starter.sql`, then

`debug_relationships.sql`.

PREDICT, THEN RUN

Write your prediction first, then run it in the relational bookstore (`bookstore.sql` loaded).

P1

Predict the output, then run it. (Animal Farm (11) is already Fiction and Classic (3).)

```
INSERT INTO book_categories (book_id, category_id) VALUES (11, 3);
```

P2

Predict the output, then run it. (Ten authors; Isaac Asimov has no books.)

```
SELECT COUNT(*) AS authors_without_books
FROM authors
WHERE id NOT IN (SELECT author_id FROM books);
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Orders, Order Items and Staff (lesson page, "Project: Orders, Order Items and Staff"). In class: load `bookstore.sql`, check the counts, read the design. At home: the rest.

- ☐ Loaded `bookstore.sql`; Step 5's counts are 10, 11, 5, 25, 8, 12, 18, 10 and I see eight tables
- ☐ I can say why orders has no total column, and why order_items stores price_at_time
- ☐ I can say why order_items has its own id while book_categories has a composite key
- ☐ Step 6: compared the CREATE TABLEs in `bookstore.sql` with my notes
- ☐ (optional) Steps 1 to 4 typed into a practice copy

SELF-CHECK

Q1

A customer can have many delivery addresses; each address belongs to exactly one customer. Where does the foreign key go?

A) `customers.address_id`

B) `addresses.customer_id`

C) A junction table `customer_addresses`

D) In both tables, one pointing at the other

Q2

Which of these needs a junction table?

A) One author writes many books

B) Each employee has at most one manager

C) Students and the courses they take

D) Each order belongs to one customer

Q3

In the finished bookstore, what does this print?

```
SELECT book_id, COUNT(*) AS categories
FROM book_categories
GROUP BY book_id
HAVING COUNT(*) = 3
ORDER BY book_id;
```

Q4

The flat books table types "J.R.R. Tolkien" in two rows. A staff member edits the spelling in one of them only. What is this called?

A) An update anomaly: one fact stored twice now disagrees with itself

B) An insert anomaly

C) A delete anomaly

D) Error 1062, a duplicate entry

Q5

A colleague builds `customer_profiles` with `customer_id INT NOT NULL` and a foreign key, but leaves out `UNIQUE`. What has the relationship become?

A) Many-to-many

B) Still one-to-one: the foreign key enforces it

C) Nothing: MySQL refuses the CREATE TABLE

D) One-to-many: a customer can now have several profiles

Q6

The bookstore's table is called books. What happens?

```
CREATE TABLE kit_reviews (
    id      INT PRIMARY KEY,
    book_id INT NOT NULL,
    FOREIGN KEY (book_id) REFERENCES book(id)
);
```

JOURNAL

Pick something you track in real life (a book club, a sports league, a recipe collection). Name its things (the future tables) and draw the lines between them. Which relationships are one-to-many, and which need a

junction table? What extra facts would that junction table hold?

4.2 Foreign Keys & Referential Integrity

BY THE END YOU CAN

- Explain referential integrity and predict when MySQL raises errors 1452 and 1451
- Choose RESTRICT, CASCADE or SET NULL for ON DELETE and ON UPDATE, and justify the choice
- Add, drop and change a named foreign key on an existing table, cleaning up orphan rows first
- List a database's foreign keys and their rules from information_schema
- Diagnose the errors raised by a badly formed key (3780, 6125, 1830)

KEY IDEAS

- **Referential integrity:** every foreign key value points at a row that exists. A declared key makes orphans impossible.
- **1452** = a child row whose parent is missing ("add or update a child row"). **1451** = a parent that children still point at ("delete or update a parent row").
- **RESTRICT** refuses · **CASCADE** deletes or updates the children too · **SET NULL** keeps them and clears the link (the column must allow NULL: 1830).
- **No rule written = NO ACTION**, which MySQL enforces exactly like RESTRICT.
- **Two questions:** can the child exist without its parent (no: CASCADE)? Must it keep naming it (yes: RESTRICT; no: SET NULL)? When in doubt, RESTRICT.
- **Name your keys** (`CONSTRAINT fk_child_parent`) and use the table-level form.
- **A key on existing data:** find the orphans (`NOT IN (SELECT id FROM parent)`), fix them truthfully, then ADD CONSTRAINT. To change a rule: DROP FOREIGN KEY, then ADD again.
- **Test destructive changes inside START TRANSACTION ... ROLLBACK.**

WORKED EXAMPLE: RESTRICT AND CASCADE IN ONE TRANSACTION

RESTRICT and CASCADE working together, inside one transaction so nothing is lost. Eve (5) has one order with two lines. Try to delete her:

```
START TRANSACTION;
DELETE FROM customers WHERE id = 5;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constrain
```

fk_orders_customer RESTRICTs it: her order still names her. Deal with the children first: delete her order (its lines cascade), then her, then undo everything:

```
SELECT COUNT(*) AS eves_items
FROM order_items
WHERE order_id IN (SELECT id FROM orders WHERE customer_id = 5);
DELETE FROM orders WHERE customer_id = 5;
SELECT COUNT(*) AS items_left FROM order_items;
DELETE FROM customers WHERE id = 5;
SELECT COUNT(*) AS customers_left FROM customers;
ROLLBACK;
```

```
+------------+
| eves_items |
+------------+
|          2 |
+------------+
+------------+
| items_left |
+------------+
|         16 |
+------------+
+----------------+
| customers_left |
+----------------+
|              7 |
+----------------+
```

RESTRICT made the decision explicit ("delete her orders too?"); CASCADE removed the order lines that mean nothing without their order. In a real store you would not do this at all: you would keep the sales history and mark the customer inactive.

GUIDED LAB

- Open `labs/module-06/l4_2-foreign-keys/delete_rules_starter.sql`. For each of the five tests, write your prediction in the table below BEFORE you turn on its line.
- Turn on one TODO line at a time, SOURCE, compare. Every test ends with ROLLBACK: the last row of the output must be the same every time.
- Finished early? `orphan_audit_starter.sql`, then `debug_fk_errors.sql`.

TEST	RULE THAT FIRES	MY PREDICTION	WHAT HAPPENED
1 delete category 5			
2 delete employee 6			
3 delete customer 8			

TEST	RULE THAT FIRES	MY PREDICTION	WHAT HAPPENED
4 renumber customer 7			
5 delete order 9			

PREDICT, THEN RUN

Write your prediction first, then run it in a freshly loaded `bookstore.sql`.

P3

Predict the output, then run it. (Fantasy (4) holds two books.)

```
START TRANSACTION;
DELETE FROM categories WHERE id = 4;
SELECT COUNT(*) AS links FROM book_categories;
ROLLBACK;
```

P4

Predict the output, then run it. (Bob (2) has two orders.)

```
DELETE FROM customers WHERE id = 2;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Book Reviews with the Right Rules (lesson page, "Project: Book Reviews with the Right Rules"). In class: Steps 1 to 4. At home: Steps 5 and 6.

- ☐ Step 1: review → book is _____ because _____; review → customer is _____ because _____
- ☐ Step 2: table created, four reviews
- ☐ Step 3: book 42 refused with error _____

- ☐ Step 4: deleting book 4 removed its reviews; ROLLBACK brought them back
- ☐ Step 5: deleting Carol kept her reviews with a NULL customer
- ☐ Step 6: both rules listed from information_schema; table dropped

SELF-CHECK

Q7

The bookstore has customers 1 to 8. What happens?

```
INSERT INTO orders (customer_id, order_date)
VALUES (9, '2026-07-01');
```

A) `ERROR 1451 (23000): Cannot delete or update a parent row`

B) `ERROR 1452 (23000): Cannot add or update a child row`

C) `ERROR 1062 (23000): Duplicate entry`

D) Order 13 is created for customer 9

Q8

The bookstore has 18 order lines; order 5 has two of them. What does the SELECT print?

```
START TRANSACTION;
DELETE FROM orders WHERE id = 5;
SELECT COUNT(*) FROM order_items;
ROLLBACK;
```

A) `16`

B) `18`

C) `ERROR 1451 (23000): Cannot delete or update a parent row`

D) `0`

Q9

James Okafor (8) manages Nina Kowalski (9). What does the SELECT print?

```
START TRANSACTION;
DELETE FROM employees WHERE id = 8;
SELECT manager_id FROM employees WHERE id = 9;
ROLLBACK;
```

Q10

A music app deletes a playlist. Its rows in playlist_songs (the links between the playlist and its songs) should...

A) block the delete: ON DELETE RESTRICT

B) keep the links with playlist_id NULL: ON DELETE SET NULL

C) go with it: ON DELETE CASCADE

D) stay unchanged: leave the rule out (NO ACTION)

Q11

The accountant wants `fk_items_order` changed from CASCADE to RESTRICT. How do you do it in MySQL?

A) `ALTER TABLE order_items ALTER FOREIGN KEY fk_items_order ON DELETE RESTRICT;`

B) `ALTER TABLE order_items MODIFY CONSTRAINT fk_items_order ON DELETE RESTRICT;`

C) `SET FOREIGN_KEY_CHECKS = 0;` then delete as usual

D) `DROP FOREIGN KEY fk_items_order`, then `ADD CONSTRAINT fk_items_order ... ON DELETE RESTRICT`

Q12

What does this print?

```
SELECT DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'bookstore'
  AND CONSTRAINT_NAME = 'fk_employees_manager';
```

JOURNAL

Think of an app you use daily (a music service, a school portal, a social network). When you delete something there (a playlist, an account, a post), what happens to the things attached to it? Which of those behaviors look like CASCADE, SET NULL or RESTRICT, and would you have chosen differently?

RELATIONSHIP MAP

Draw the eight tables of the bookstore as boxes. Draw each foreign key as a crow's foot line (`||` exactly one, `o|` zero or one, `|{` one or many, `o{` zero or many). Label each line with its ON DELETE rule. Then check with the query below.

```
SELECT TABLE_NAME, CONSTRAINT_NAME, DELETE_RULE, UPDATE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'bookstore'
ORDER BY TABLE_NAME, CONSTRAINT_NAME;
```

EXIT TICKET

Three questions, no notes.

E1

For one-to-many, one-to-one and many-to-many: where does the foreign key go?

E2

You get `ERROR 1452`. Which table were you changing (parent or child), and what is missing? And `ERROR 1451`?

E3

Choose an ON DELETE rule for `order_items.book_id → books` and justify it in one sentence.

NOTES

HOMEWORK CHECKLIST

- ☐ 4.1 project: compare my design with `bookstore.sql` (Steps 1 to 4 typed in a practice copy: optional)
- ☐ 4.2 project Steps 5 and 6
- ☐ Lesson 4.1 and 4.2 Hands-on Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes; both journal entries

- ☐ Labs not finished in class: `navigate_links_starter.sql`, `orphan_audit_starter.sql`, the debug handouts
- ☐ Part 1 assessment preparation: reread my project scripts (Sessions 3 to 6), redo one exercise per lesson from Modules 2 to 4
- ☐ Bring to Session 7: a charged laptop with `bookstore.sql` loaded, my course folder and scripts, this workbook with the relationship map

GLOSSARY

TERM	MEANING
anomaly (update, insert, delete)	A problem caused by storing one fact in many rows: copies disagree, nothing can be stored, or a fact is lost
parent / child table	The table whose id is referenced / the table that holds the foreign key
one-to-many (1:M)	One parent row, many child rows; the key goes on the many side
one-to-one (1:1)	At most one child per parent: a foreign key that is also UNIQUE
many-to-many (M:M)	Both sides have many partners: a junction table with two foreign keys
junction table	A table whose rows are links (book_categories, order_items), with a composite or own primary key
composite primary key	A primary key of two or more columns: the combination must be unique
referential integrity	Every foreign key value points at an existing row
orphan	A child row whose parent does not exist
RESTRICT / NO ACTION	Refuse to delete or change a parent that children point at (1451)
CASCADE	Delete (or renumber) the children along with the parent
SET NULL	Keep the children, set their foreign key to NULL (the column must allow NULL)
information_schema	The built-in database that describes all others (REFERENTIAL_CONSTRAINTS lists keys and rules)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 7: Joins and the Part 1 Assessment

Your workbook for Session 7: the warm-up, the key ideas of Lesson 4.3, a worked example, the guided lab, predict-then-run practice, the 4.3 self-check, how the Part 1 assessment works, the Part 1 review and your exit ticket.

Lesson 4.3 · Part 1 assessment | Session 7 · 3 hours | Name: _____

SESSION GOALS

- Combine matching rows of two or more tables with JOIN, using aliases.
- Keep unmatched rows with LEFT JOIN, and find rows with no match (the anti-join).
- Join a table to itself, and combine joins with GROUP BY without counting the wrong thing.
- Show what you can do with Part 1 in a 60-minute, open-book assessment on a new database.

WARM-UP: LESSONS 4.1 AND 4.2

ANSWER WITHOUT RUNNING ANYTHING, THEN CHECK ON THE PROJECTOR

W1

One order has many lines. Where does the foreign key go?

W2

Predict the count. (Order 3 has one line; fk_items_order is ON DELETE CASCADE.)

```
START TRANSACTION;
DELETE FROM orders WHERE id = 3;
SELECT COUNT(*) AS items FROM order_items;
ROLLBACK;
```

W3

What happens, and why?

```
DELETE FROM authors WHERE id = 6;
```

W4

A key is declared with no ON DELETE. What does MySQL do when a parent with children is deleted?

W5

How many links?

```
SELECT COUNT(*) AS links
FROM book_categories
WHERE book_id IN (SELECT id FROM books WHERE author_id = 6);
```

LESSON

4.3 Joins

BY THE END YOU CAN

- Combine matching rows from two or more tables with INNER JOIN ... ON, using table aliases
- Keep unmatched rows with LEFT JOIN, and find rows with no match using the anti-join pattern
- Explain what RIGHT JOIN and CROSS JOIN return, and when (rarely) to use them
- Join a table to itself to read a hierarchy such as employees and managers
- Combine joins with GROUP BY without the two silent traps: COUNT(*) on outer joins and row fan-out

KEY IDEAS

- **JOIN ... ON** (INNER) returns one row per matching pair; a row without a partner disappears.
- **LEFT JOIN** keeps every row of the left table (the one before JOIN) and fills the right side with NULL when nothing matches.
- **Anti-join:** `LEFT JOIN right ... WHERE right.id IS NULL` finds the rows with no match (or NOT EXISTS).
- **On a LEFT JOIN, conditions on the right table go in ON;** in WHERE they turn it into an inner join.
- **A JOIN without ON is a cross join** (every combination): check the ON when a join returns too many rows.
- **Self-join:** one table, two aliases, two roles (employee and manager).
- **What does one row of this join stand for?** Ask before COUNT or SUM: use `COUNT(right.id)`, `COUNT(DISTINCT ...)`, COALESCE.
- **Qualify columns** that exist in both tables (1052); after an alias, use only the alias (1054).

WORKED EXAMPLE: GUESTS AND RSVPS

A guest list and the RSVPs, small enough to check by eye. Four guests; Ana replied twice, Cleo once, Ben and Dev never.

```
DROP DATABASE IF EXISTS kit_rsvp;
CREATE DATABASE kit_rsvp;
USE kit_rsvp;
CREATE TABLE guests (id INT PRIMARY KEY, name VARCHAR(30) NOT NULL);
CREATE TABLE rsvps (
    id       INT PRIMARY KEY,
    guest_id INT NOT NULL,
    answer   VARCHAR(5) NOT NULL,
    CONSTRAINT fk_rsvps_guest FOREIGN KEY (guest_id) REFERENCES guests(id)
);
INSERT INTO guests VALUES (1, 'Ana'), (2, 'Ben'), (3, 'Cleo'), (4, 'Dev');
INSERT INTO rsvps VALUES (1, 1, 'yes'), (2, 3, 'no'), (3, 1, 'yes');
```

```
Database changed
```

JOIN keeps only the pairs (guest + reply): Ben and Dev are not in it, and Ana appears once per reply.

```
SELECT g.name, r.answer
FROM guests AS g
JOIN rsvps AS r ON r.guest_id = g.id
ORDER BY g.id, r.id;
```

```
+------+--------+
| name | answer |
+------+--------+
| Ana  | yes    |
| Ana  | yes    |
| Cleo | no     |
+------+--------+
```

LEFT JOIN keeps every guest; where there is no reply it writes NULL:

```
SELECT g.name, r.answer
FROM guests AS g
LEFT JOIN rsvps AS r ON r.guest_id = g.id
ORDER BY g.id, r.id;
```

```
+------+--------+
| name | answer |
+------+--------+
| Ana  | yes    |
| Ana  | yes    |
| Ben  | NULL   |
| Cleo | no     |
| Dev  | NULL   |
+------+--------+
```

The anti-join keeps only the guests with no reply (the right side's id is NULL):

```sql
SELECT g.name
FROM guests AS g
LEFT JOIN rsvps AS r ON r.guest_id = g.id
WHERE r.id IS NULL;
```

```
+------+
| name |
+------+
| Ben  |
| Dev  |
+------+
```

And the counting trap: COUNT(*) counts Ben's and Dev's padded rows; COUNT(r.id) does not:

```sql
SELECT g.name, COUNT(*) AS rows_counted, COUNT(r.id) AS replies
FROM guests AS g
LEFT JOIN rsvps AS r ON r.guest_id = g.id
GROUP BY g.id, g.name
ORDER BY g.id;
```

```
+------+--------------+---------+
| name | rows_counted | replies |
+------+--------------+---------+
| Ana  |            2 |       2 |
| Ben  |            1 |       0 |
| Cleo |            1 |       1 |
| Dev  |            1 |       0 |
+------+--------------+---------+
```

Clean up:

```
DROP DATABASE kit_rsvp;
```

GUIDED LAB

- SOURCE `labs/module-07/l4_3-joins/catalog_joins_starter.sql`. Every query runs; each is missing its join.
- TODO 1 together, TODOs 2 to 4 in pairs. Before each one, say what one row of your join stands for.
- TODO 3 answers a Session 6 question with a join instead of NOT EXISTS. Which do you find easier to read?

- Finished early? `city_sales_starter.sql`, then `debug_joins.sql`.

PREDICT, THEN RUN

Write your prediction first, then run it in a freshly loaded `bookstore.sql`.

P1

Predict the output, then run it. (8 customers, 12 orders; Carol and Hank never ordered.)

```
SELECT COUNT(*) AS rows_returned
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id;
```

P2

Predict the output, then run it. (Carol (3) has no orders; Frank (6) has two.)

```
SELECT c.first_name, COUNT(*) AS n
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE c.id IN (3, 6)
GROUP BY c.id, c.first_name
ORDER BY c.id;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Sales Report (lesson page, "Project: The Sales Report"): `sales_report.sql`. In class: Step 1. At home: Steps 2 to 6.

- ☐ Step 1: one line per order (12 rows; order 1 totals 34.97)
- ☐ Step 2: revenue by month (the six months add up to 237.29, the sum of Step 1's totals)
- ☐ Step 3: the five best sellers
- ☐ Step 4: customers to welcome (LEFT JOIN ... IS NULL): Hank and Carol
- ☐ Step 5: copies sold per category
- ☐ Step 6: active staff and their managers; SOURCE runs the whole file

SELF-CHECK

Q1

Eight customers placed 12 orders; Carol and Hank placed none. How many rows does this return?

```
SELECT c.first_name, o.id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id;
```

A) `8`

B) `12`

C) `14`

D) `96`

Q2

Which query lists the books that have never been sold?

A) `FROM books b LEFT JOIN order_items oi ON oi.book_id = b.id WHERE oi.id IS NULL`

B) `FROM books b JOIN order_items oi ON oi.book_id = b.id WHERE oi.id IS NULL`

C) `FROM books b LEFT JOIN order_items oi ON oi.book_id = b.id WHERE b.id IS NULL`

D) `FROM order_items oi LEFT JOIN books b ON oi.book_id = b.id WHERE b.id IS NULL`

Q3

Alice (customer 1) placed three orders. What does this print?

```
SELECT COUNT(*) AS rows_returned
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.id
WHERE o.customer_id = 1;
```

Q4

What happens when this runs?

```
SELECT id, order_date, first_name
FROM orders
JOIN customers ON orders.customer_id = customers.id;
```

A) It returns 12 rows, using the order's id

B) `ERROR 1052 (23000): Column 'id' in field list is ambiguous`

C) `ERROR 1054 (42S22): Unknown column 'id' in 'field list'`

D) It returns 96 rows: a cross join

Q5

Carol (3) has no orders; Dave (4) has two. What does this print?

```
SELECT c.first_name, COUNT(o.id) AS orders
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE c.id IN (3, 4)
GROUP BY c.id, c.first_name
ORDER BY c.id;
```

Q6

This was meant to list every author with their in-stock books. Isaac Asimov (no books) and the authors of the two out-of-stock books are missing. Why?

```
SELECT a.name, b.title
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.id
WHERE b.in_stock = TRUE
ORDER BY a.id, b.title;
```

A) LEFT JOIN cannot be combined with WHERE

B) in_stock is a BOOLEAN, so TRUE must be written as 1

C) The ORDER BY removes the NULL rows

D) The WHERE runs after the join: for the padded rows b.in_stock is NULL, so the LEFT JOIN behaves like an inner join

JOURNAL

Explain the difference between JOIN and LEFT JOIN to a friend who has never written SQL, using a guest list and a list of RSVPs. Then write down, in your own words, what "one row of this join stands for" means and why it matters before you use SUM or COUNT.

PART 1 ASSESSMENT: HOW IT WORKS

THE RULES

- **60 minutes, open book:** the lesson pages, your notes, this workbook and your own scripts are allowed. Other people and AI assistants are not.
- **The file is the brief:** `labs/module-07/part1-assessment/part1_assessment_starter.sql`. It builds the Riverside Cooking School database (`kit_riverside`): instructors, classes and students. You build the enrollments table (Part A) and answer eight questions (Part B), writing every answer in the file under its TODO.
- **Run it often:** save, then `SOURCE labs/module-07/part1-assessment/part1_assessment_starter.sql`. Every run rebuilds the school from scratch, so you cannot break anything.
- **Joins or subqueries:** either is fine where both work. Every query that returns several rows has an ORDER BY. Do not type a number the database can compute.
- **Hand in** at the end: the file, renamed `part1_assessment_<yourname>.sql`, through the channel on the board. It must run from start to finish.

BEFORE YOU START

- ☐ My laptop is charged or plugged in; `mysql -u root -p` works from my course folder
- ☐ The starter runs as handed out: three tables built, TODO lines, no ERROR
- ☐ The lesson pages I may want are open in tabs (2.2 types and constraints, 4.2 foreign keys, 3.3 GROUP BY, 4.3 joins)
- ☐ I know the plan: Part A in about 20 minutes, Part B in about 35, a final run in the last 5

IF YOU GET STUCK

- Read the FIRST error line: number, then reason. The lesson's Common Mistakes sections list most of them.
- If A1 will not work after 15 minutes, make a minimal table (`student_id INT, class_id INT, enrolled_on DATE, rating INT`), turn on the A2 INSERT and move on: Part B needs the rows more than the rules.
- Skip a Part B question and come back: each one is scored on its own.

PART 1 REVIEW

SELF-CHECK: ONE QUESTION PER MODULE

Six review questions across Modules 2 to 4 (the 4.3 items are above). Do them after the assessment or at home; the matching lesson is named in each.

Q7

Review (Lesson 2.2): which column type fits a book's price?

A) `DECIMAL(8, 2)`

B) `FLOAT`

C) `VARCHAR(10)`

D) `INT`

Q8

Review (Lessons 2.2 and 2.3): what happens?

```
INSERT INTO customers (first_name, last_name, email)
VALUES ('Ivy', 'Lane', 'alice@example.com');
```

Q9

Review (Lesson 3.1): how many books does this count?

```
SELECT COUNT(*) FROM books WHERE title LIKE 'The %';
```

A) `2`

B) `3`

C) `4`

D) `11`

Q10

Review (Lessons 3.2 and 3.3): in which order must these clauses be written?

A) SELECT, FROM, GROUP BY, WHERE, HAVING, ORDER BY, LIMIT

B) SELECT, FROM, WHERE, ORDER BY, GROUP BY, HAVING, LIMIT

C) SELECT, FROM, WHERE, HAVING, GROUP BY, ORDER BY, LIMIT

D) SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT

Q11

Review (Lesson 3.4): what does this print?

```
SELECT title
FROM books
WHERE price = (SELECT MIN(price) FROM books);
```

Q12

Review (Lessons 4.1 and 4.2): what is the bookstore's order_items table?

A) A one-to-one extension of orders

B) A copy of books with a quantity column

C) A junction table linking orders and books (many-to-many), carrying facts about each link: quantity and price_at_time

D) A self-referencing table

THE MAP OF PART 1

MODULE	WHAT IT GAVE YOU	LESSONS	CONFIDENT? (YES / NOT YET)
1 Getting Started	a server, a client, a database; errors you can read	1.1, 1.2, 1.3	

MODULE	WHAT IT GAVE YOU	LESSONS	CONFIDENT? (YES / NOT YET)
2 Tables & Data Types	tables with types and constraints; INSERT, SELECT, UPDATE, DELETE	2.1, 2.2, 2.3	
3 Querying Data	WHERE, ORDER BY, LIMIT; COUNT, SUM, AVG, GROUP BY, HAVING; subqueries	3.1, 3.2, 3.3, 3.4	
4 Relationships & Joins	keys and relationships; foreign key rules; JOIN and LEFT JOIN	4.1, 4.2, 4.3	

EXIT TICKET

Three questions; hand it in with your assessment file.

E1

In one sentence each, with the customers and orders tables: what does JOIN return, and what does LEFT JOIN add?

E2

Which assessment task was hardest for you, and what would you look up first?

E3

Name one thing from Part 1 you will use in a project or at work, and one you want to practice.

NOTES

HOMEWORK CHECKLIST

- ☐ `sales_report.sql` Steps 2 to 6 (optional practice if the course ends with Part 1)
- ☐ Lesson 4.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); the 4.3 Quick Quiz; the 4.3 journal
- ☐ The Part 1 review self-check (Q7 to Q12) and the map of Part 1: one "not yet" line, reread
- ☐ Labs not finished in class: `catalog_joins_starter.sql`, `city_sales_starter.sql`, `debug_joins.sql`
- ☐ If Part 2 follows: reload `bookstore.sql`; bring `sales_report.sql`, the course folder, this workbook and my journal to Session 8

GLOSSARY

TERM	MEANING
join	Pairs the rows of two tables that satisfy a condition (ON) and returns each pair as one wider row
INNER JOIN (JOIN)	Only the rows with a match on both sides
LEFT JOIN	Every row of the left table; NULL in the right table's columns where nothing matched
anti-join	LEFT JOIN ... WHERE right.id IS NULL: the left rows with no match
CROSS JOIN	Every combination of rows (also what a JOIN without ON does)
self-join	A table joined to itself under two aliases (employee and manager)
table alias	A short name for a table in one query (`books AS b`)
fan-out	Rows multiplied by a join (one row per order line, not per order): count with care
ambiguous column (1052)	A column name that exists in two joined tables, used without its table

MYSQL FOUNDATIONS · PARTICIPANT WORKBOOK

Part 2: Beyond the Basics

Sessions 8 to 14. Modules 5 to 8: CTEs and window functions, indexes, views and design, stored programs, transactions, JSON, users and backups, ending with the final project.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 8: CTEs and Window Functions

Your workbook for Session 8, the start of Part 2: the Part 2 environment check and a Part 1 refresher, the key ideas of Lessons 5.1 and 5.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, the debug clinic and your exit ticket.

Lessons 5.1–5.2 | Session 8 · 3 hours | Name: _____

SESSION GOALS

- Confirm your machine is ready for Part 2: the full bookstore, MySQL 9.7, READY for Part 2.
- Rewrite a nested query as named steps with WITH, chain several CTEs, and read one CTE twice.
- Write recursive CTEs: generate series, fill gaps in a report, walk the staff hierarchy down and up.
- Put every row next to a total, an average, a rank or its neighbor with window functions.
- Return the top N rows per group, and explain why WHERE cannot filter a window (3593).

START HERE: READY FOR PART 2?

ENVIRONMENT CHECK

- ☐ In the mysql client, from my course folder: `SOURCE bookstore.sql` prints eight row counts (11 books, 8 customers, 12 orders, 10 employees ...)
- ☐ `SOURCE labs/module-08/l5_0-part2-start/check_environment.sql` ends with `READY for Part 2` (or `READY WITH NOTES` on 8.4 or MariaDB)
- ☐ Starting with Part 2? I ran `refresher_starter.sql` and its four answers match the "Expected when done" comment

Part 2 starts every lesson from a fresh copy of the course database. When a result does not match the lesson, the first fix is always the same: `SOURCE bookstore.sql` and run it again.

PART 1 REFRESHER: ANSWER FROM MEMORY

Five questions on the Part 1 skills today's lessons build on. Write a short answer for each; the class goes through them together.

W1

Which customers have never placed an order? Say the query shape, then the names.

W2

employees has a manager_id. How do you show each employee next to their manager's name? Who is Sofia's manager?

W3

SELECT a.name, COUNT(*) FROM authors a LEFT JOIN books b ON b.author_id = a.id GROUP BY a.id, a.name: what does it say for Isaac Asimov, who has no books, and how do you fix it?

W4

Books per author and copies sold in one query: Orwell shows 5 books, but he wrote 2. What happened?

W5

A subquery in FROM (a derived table) fails with error 1248. What is missing?

Missed two or more? `labs/module-08/l5_0-part2-start/refresher_starter.sql` practices the same moves; Lesson 4.3's Common Mistakes 3 to 5 explain W3 and W4.

LESSONS

5.1 CTEs & Recursive Queries

BY THE END YOU CAN

- Rewrite a nested subquery as a WITH query and name each step
- Chain several CTEs in one statement, letting later steps read earlier ones
- Choose between a CTE, a derived table and a view for a given job
- Write a recursive CTE with an anchor, a recursive member and a stop condition, and generate number and month series with it
- Walk the employee hierarchy down (an org chart) and up (a chain of command)
- Read and fix errors 1146, 3636 and 1406 from broken CTEs

KEY IDEAS

- **A CTE** (`WITH name AS (...)`) is a named result that lives for ONE statement and is read like a table: select, filter, join, group it.
- **Several CTEs, one WITH,** separated by commas; each can read the ones before it, and each can be read more than once.
- **CTE, derived table or view?** Derived table: a small one-off step. CTE: multi-step questions and hierarchies. View: a question the whole team asks every day (Session 9).
- **A recursive CTE** = anchor (runs once) + `UNION ALL` + recursive member (runs per round, reading only the previous round's rows) with a stop condition.
- **The anchor fixes every column's type and width:** `CAST(... AS CHAR(200))` for paths and lists, or error 1406.
- **Down a hierarchy:** `e.manager_id = cte.id`. **Up:** `e.id = cte.manager_id`.
- `cte_max_recursion_depth` (1000) stops runaway recursion with error 3636; MariaDB stops quietly with a warning.

WORKED EXAMPLE: COUNTING COUNTS, AND WEEKS WITH NO ORDERS

A CTE can count, then the main query can count the counts. How many authors have 0, 1 or 2 books? The LEFT JOIN keeps Isaac Asimov, who has none:

```
WITH book_counts AS (
    SELECT a.name AS author, COUNT(b.id) AS books
    FROM authors AS a
    LEFT JOIN books AS b ON b.author_id = a.id
    GROUP BY a.id, a.name
)
SELECT books, COUNT(*) AS authors
FROM book_counts
GROUP BY books
ORDER BY books;
```

```
+-------+---------+
| books | authors |
+-------+---------+
|     0 |       1 |
|     1 |       7 |
|     2 |       2 |
+-------+---------+
```

A recursive series does not have to count by one. Here it makes the four Sundays of February 2026 (every 7 days), and a LEFT JOIN hangs the orders on each week, so the empty last week still shows:

```
WITH RECURSIVE weeks AS (
    SELECT DATE('2026-02-01') AS week_start
    UNION ALL
    SELECT week_start + INTERVAL 7 DAY
    FROM weeks
    WHERE week_start < '2026-02-22'
)
SELECT w.week_start, COUNT(o.id) AS orders
FROM weeks AS w
LEFT JOIN orders AS o
       ON o.order_date >= w.week_start
      AND o.order_date <  w.week_start + INTERVAL 7 DAY
GROUP BY w.week_start
ORDER BY w.week_start;
```

```
+------------+--------+
| week_start | orders |
+------------+--------+
| 2026-02-01 |      1 |
| 2026-02-08 |      1 |
| 2026-02-15 |      1 |
| 2026-02-22 |      0 |
+------------+--------+
```

The stop condition reads the row the round starts from: the round that reads February 22 fails `week_start < '2026-02-22'`, so no March 1 row is made.

GUIDED LAB

- SOURCE `labs/module-08/l5_1-ctes/staff_tree_starter.sql`. You see one month, a tree where every division says `(owner)`, and a TODO line.
- TODO 1 together: make the month series run to 12. Before you edit: which round makes month 12, and which round stops?
- TODO 2 in pairs: level-2 managers start a division; everyone below inherits it. Edit the recursive member, not the anchor.
- Done when your results match the "Expected when done" comment. Finished early? TODO 3, then `category_steps_starter.sql`.

PREDICT, THEN RUN

In the freshly loaded bookstore (these only read). Write your prediction first, then run it. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (A countdown in steps of 3.)

```
WITH RECURSIVE countdown AS (
    SELECT 10 AS n
    UNION ALL
    SELECT n - 3 FROM countdown WHERE n > 3
)
SELECT n FROM countdown;
```

P2

Predict the output, then run it. (Two customers have no city.)

```
WITH by_city AS (
    SELECT city, COUNT(*) AS customers
    FROM customers
    GROUP BY city
)
SELECT COUNT(*) AS city_groups, SUM(customers) AS customers
FROM by_city;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Staff and Sales Briefing (lesson page, "Project"), `org_report.sql`. In class: Steps 1 and 3. At home: Steps 2, 4 and 5.

- ☐ Step 1: `USE bookstore;` and the indented org chart with each person's level (10 people, 4 levels)
- ☐ Step 2: team size and payroll for every manager (Maria's team of 9 at the top)
- ☐ Step 3: the chain of command for Nina (id 9); with id 7: Sofia, Marcus, Daniel, Maria
- ☐ Step 4: the sales calendar, January to July, July says "no sales yet"

- ☐ Step 5: SOURCE runs the whole file without an error

SELF-CHECK

Q1

A statement starts `WITH monthly AS (...)`. How long does `monthly` exist?

A) Until you disconnect

B) Only while that one statement runs

C) Until you `DROP` it

D) Until the next `COMMIT`

Q2

What does this print?

```
WITH RECURSIVE s AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n * 2 FROM s WHERE n < 20
)
SELECT GROUP_CONCAT(n ORDER BY n) AS series FROM s;
```

Q3

A CTE named `org` joins to itself, but the statement starts `WITH org AS (` without RECURSIVE. What happens?

A) `ERROR 1146 (42S02): Table 'bookstore.org' doesn't exist`

B) `ERROR 3636 (HY000): Recursive query aborted after 1001 iterations`

C) It runs, but only the anchor's rows come back

D) `ERROR 1406 (22001): Data too long for column`

Q4

A recursive CTE named `chain` starts from one employee. Which condition in the recursive member walks UP to the owner?

A) `JOIN chain c ON e.manager_id = c.id`

B) `JOIN chain c ON e.id = c.id`

C) `JOIN chain c ON e.id = c.manager_id`

D) `JOIN chain c ON e.manager_id = c.manager_id`

Q5

The bookstore has 10 employees. This query prints 30. What is wrong?

```
WITH RECURSIVE org AS (
    SELECT id, 1 AS level FROM employees
    UNION ALL
    SELECT e.id, o.level + 1
    FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT COUNT(*) AS rows_back FROM org;
```

A) RECURSIVE is missing

B) `UNION ALL` should be `UNION`

C) The recursion limit is too low

D) The anchor has no WHERE, so every employee starts a walk of their own

Q6

In one sentence: why does a monthly report LEFT JOIN a generated calendar of months to the orders, instead of grouping the orders by month directly?

JOURNAL

Think of a chain in your own life that has no fixed length: a family tree, folders inside folders, a reply to a reply. Describe it as a recursive CTE in plain words: the anchor, what each round finds from the round before, and what makes it stop.

5.2 Window Functions

BY THE END YOU CAN

- Explain how OVER() differs from GROUP BY, and split a window with PARTITION BY
- Number and rank rows with ROW_NUMBER, RANK and DENSE_RANK, and pick the right one when there are ties
- Build running totals and moving averages, and control them with a frame (ROWS BETWEEN ...)
- Compare each row with the one before or after it using LAG and LEAD
- Return the top N rows per group by filtering a window result in a CTE, and explain why WHERE cannot do it (error 3593)

KEY IDEAS

- **A window function** computes across a set of rows (the window) and returns a value for EVERY row: `OVER` turns an aggregate into one.
- **PARTITION BY** splits the window per group without collapsing rows; **ORDER BY** inside OVER orders each window; the **frame** (`ROWS BETWEEN ...`) says which neighbors count.
- **ROW_NUMBER** never ties; **RANK** ties and skips (1, 2, 2, 4); **DENSE_RANK** ties without gaps (1, 2, 2, 3).
- **Running total:** `SUM(x) OVER (ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. The default frame is RANGE, which lumps ties together.
- **LAG / LEAD** read the previous / next row, with an optional offset and default.
- **Windows run after WHERE, GROUP BY and HAVING:** filter on them in an outer query over a CTE (error 3593 otherwise). Top N per group = ROW_NUMBER in a CTE, `WHERE pos <= N` outside.
- **Reserved words:** `rank`, `row_number`, `window`, `groups`: name columns `rank_num`, `pos`.

WORKED EXAMPLE: TWO WINDOWS PER CUSTOMER, AND THE FIRST VALUE

Two windows over the same partition: which order is this for the customer (ROW_NUMBER), and how many orders does the customer have in all (COUNT(*) OVER)? No GROUP BY, so every order stays:

```
SELECT c.first_name,
       o.id AS order_id,
       o.order_date,
       ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY o.order_date) AS nth
       COUNT(*)     OVER (PARTITION BY o.customer_id)                      AS of_
FROM orders AS o
JOIN customers AS c ON c.id = o.customer_id
WHERE c.first_name IN ('Alice', 'Bob')
ORDER BY c.first_name, o.order_date;
```

```
+------------+----------+------------+-----------+----------+
| first_name | order_id | order_date | nth_order | of_total |
+------------+----------+------------+-----------+----------+
| Alice      |        1 | 2026-01-15 |         1 |        3 |
| Alice      |        2 | 2026-02-20 |         2 |        3 |
| Alice      |        8 | 2026-04-05 |         3 |        3 |
| Bob        |        3 | 2026-03-10 |         1 |        2 |
| Bob        |        7 | 2026-03-28 |         2 |        2 |
+------------+----------+------------+-----------+----------+
```

FIRST_VALUE reads the first row of the window. Ordered by price within each author, that is the author's cheapest book, shown on every row of that author (LAST_VALUE would need a frame to the end of the window: the lesson's Common Mistake 5):

```
SELECT a.name AS author,
       b.title,
       b.price,
       FIRST_VALUE(b.title) OVER (PARTITION BY a.id ORDER BY b.price) AS cheapest_
FROM books AS b
JOIN authors AS a ON a.id = b.author_id
WHERE a.id IN (2, 6)
ORDER BY a.name, b.price;
```

```
+----------------+-------------------+-------+--------------------+
| author         | title             | price | cheapest_by_author |
+----------------+-------------------+-------+--------------------+
| George Orwell  | Animal Farm       |  8.49 | Animal Farm        |
| George Orwell  | 1984              |  9.99 | Animal Farm        |
| J.R.R. Tolkien | The Hobbit        | 10.99 | The Hobbit         |
| J.R.R. Tolkien | Lord of the Rings | 15.99 | The Hobbit         |
+----------------+-------------------+-------+--------------------+
```

◆ GUIDED LAB

- SOURCE `labs/module-08/l5_2-window-functions/leaderboards_starter.sql`. Only ROW_NUMBER is there.
- Before TODO 1, predict and write: what do RANK and DENSE_RANK give Dave and Frank (both 4 copies)? Bob and Grace (both 3)?
- TODO 2: make the numbering restart in every category. TODO 3: a payroll per department, then the percentage.
- Finished early? `order_trends_starter.sql` in the same folder.

CUSTOMER	COPIES	ROW_NUMBER	RANK	DENSE_RANK
Dave	4			
Frank	4			
Bob	3			
Grace	3			

PREDICT, THEN RUN

In the freshly loaded bookstore (these only read). Write your prediction first, then run it. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (WHERE runs before the windows.)

```
SELECT first_name,
       COUNT(*) OVER ()                          AS everyone,
       COUNT(*) OVER (PARTITION BY department) AS in_dept
FROM employees
WHERE department = 'support'
ORDER BY first_name;
```

P4

Predict the output, then run it. (LAG with a default.)

```
SELECT id,
       LAG(id, 1, 0) OVER (ORDER BY id) AS previous_id
FROM orders
WHERE customer_id = 1
ORDER BY id;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Sales Dashboard (lesson page, "Project"), `sales_dashboard.sql`. In class: Steps 1 and 2. At home: Steps 3 to 5.

- ☐ Step 1: revenue so far: running total ends at 237.29, pct_done at 100.0
- ☐ Step 2: month over month: LAG and a three-month moving average with one named window
- ☐ Step 3: the best-seller leaderboard: 1984 alone in first place, four places in all
- ☐ Step 4: the top customer in every city (Grace for Chicago)
- ☐ Step 5: SOURCE runs the whole file without an error

SELF-CHECK

Q7

`employees` has 10 rows in 4 departments. How many rows does this return? `SELECT first_name, AVG(salary) OVER (PARTITION BY department) FROM employees;`

A) 4

B) 1

C) 10

D) None: `ERROR 1140` (a column without GROUP BY)

Q8

Copies sold, best first, are 5, 4, 4 and 3. What do `RANK()` and `DENSE_RANK()` (both `OVER (ORDER BY copies DESC)`) give the book with 3?

Q9

`SELECT title FROM books WHERE RANK() OVER (ORDER BY price DESC) <= 3;` fails with error 3593. What is the standard fix?

A) Move the condition to `HAVING`

B) Compute the rank in a CTE, then filter on it in the outer query

C) Replace `WHERE` with `QUALIFY`

D) Give the rank a column alias and filter on the alias in WHERE

Q10

In `SUM(price) OVER (ORDER BY price)`, The Catcher in the Rye and The Great Gatsby (both 8.99) show the same running total, 34.46. Why?

A) `SUM` skips duplicate values

B) Prices are sorted as text

C) The window has no `PARTITION BY`

D) The default frame is RANGE: rows with the same ORDER BY value are added in one step

Q11

Bob (customer 2) has two orders. What does this print?

```
SELECT id,
       order_date,
       LEAD(order_date) OVER (ORDER BY order_date) AS next_order
FROM orders
WHERE customer_id = 2
ORDER BY order_date;
```

Q12

You need exactly one top earner per department, even if two salaries ever tie. Which function numbers the rows?

A) `ROW_NUMBER()` with a tie-breaker, e.g. `ORDER BY salary DESC, id`

B) `RANK()`

C) `DENSE_RANK()`

D) `NTILE(1)`

◆ JOURNAL

Pick a ranking you know from outside databases: a race, a league table, a music chart. When two entries tie, which function does it behave like: ROW_NUMBER, RANK or DENSE_RANK? Then describe one question from your work or hobby that needs "each row next to a total" and sketch its OVER clause.

DEBUG CLINIC

◆ SIX ERRORS, ONE FIX EACH

SOURCE `labs/module-08/l5_1-ctes/debug_ctes.sql` and `labs/module-08/l5_2-window-functions/debug_windows.sql`. For each ERROR line write the number, the cause in one sentence and the fix in one line.

QUERY	ERROR NUMBER	CAUSE	FIX
5.1 A			
5.1 B			
5.1 C			
5.2 A			
5.2 B			
5.2 C			

EXIT TICKET

Three questions, no notes.

E1

Write the skeleton of a recursive CTE that counts from 1 to 10, and label the anchor, the recursive member and the stop condition.

E2

`GROUP BY department` versus `PARTITION BY department`: what is the difference in the rows you get back?

E3

Why can't WHERE filter on `ROW_NUMBER()`, and what do you do instead?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish `org_report.sql` (Steps 2, 4, 5) and `sales_dashboard.sql` (Steps 3 to 5)
- ☐ Lesson 5.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 5.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Labs not finished in class: `category_steps_starter.sql`, `order_trends_starter.sql`
- ☐ 5.2 "Before the Next Lesson": run sales_dashboard.sql from a fresh session; rewrite one query with a named WINDOW
- ☐ Bring to Session 9: a READY laptop with `bookstore.sql`, the course folder with `labs/`, both project files, this workbook and my journal

GLOSSARY

TERM	MEANING
CTE (common table expression)	A named, temporary result written with WITH; it lives for one statement
derived table	A subquery in FROM, with an alias (Lesson 3.4); read inside out
recursive CTE	A CTE that reads itself, round by round (WITH RECURSIVE)
anchor	The part of a recursive CTE that runs once and makes the starting rows
recursive member	The part after UNION ALL that runs every round, from the previous round's rows
stop condition	The condition in the recursive member that eventually lets a round make no rows
`cte_max_recursion_depth`	The session's cap on rounds (1000); error 3636 when it is reached

TERM	MEANING
window function	A function that computes across a window of rows and returns a value for every row
`OVER`	The clause that turns a function into a window function and describes its window
`PARTITION BY`	Splits the rows into separate windows, one per group
frame	The rows around the current row that an aggregate sees (`ROWS BETWEEN ...`)
`ROW_NUMBER` / `RANK` / `DENSE_RANK`	Numbering without ties / with ties and gaps / with ties, no gaps
`LAG` / `LEAD`	The value from the previous / next row of the window
top N per group	Number the rows per group in a CTE, keep the low numbers in the outer query

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 9: Indexes and Views

Your workbook for Session 9: a warm-up on Session 8, the key ideas of Lessons 6.1 and 6.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, the clinic's error table and your exit ticket.

Lessons 6.1–6.2 | Session 9 · 3 hours | Name: _____

SESSION GOALS

- Measure a query with EXPLAIN ANALYZE, add the index the plan points at, and measure again.
- Choose the column order of a composite index, and know which indexes every table already has.
- Spot and rewrite the habits that stop an index from being used.
- Save queries as views, read them like tables, and know which views can be updated.
- Guard an updatable view with WITH CHECK OPTION, and use a view to show an account only what it should see.

WARM-UP

FIVE QUESTIONS ON SESSION 8, NO NOTES

W1

What does this print?

```
WITH RECURSIVE s AS (
    SELECT 1 AS n UNION ALL SELECT n + 2 FROM s WHERE n < 7
)
SELECT GROUP_CONCAT(n ORDER BY n) AS odd FROM s;
```

W2

Which price_level do The Catcher in the Rye and The Great Gatsby (both 8.99) get, and Animal Farm after them?

```
SELECT title, price,
       DENSE_RANK() OVER (ORDER BY price DESC) AS price_level
FROM books
WHERE price < 9.50
ORDER BY price DESC, title;
```

W3

Why can't WHERE filter on ROW_NUMBER(), and what do you write instead?

W4

In a recursive CTE over employees, which join walks UP the hierarchy?

W5

AVG(salary) with GROUP BY department versus AVG(salary) OVER (PARTITION BY department): how many rows from the 10 employees?

LESSONS

6.1 Indexes & Query Optimization

BY THE END YOU CAN

- Explain what an index is, what it costs, and which indexes every bookstore table already has
- Read a query plan from EXPLAIN and measure a query with EXPLAIN ANALYZE
- Create single-column, composite and unique indexes, and choose the column order of a composite index
- Spot and rewrite the patterns that stop an index from being used: functions on the column, leading wildcards, type mismatches
- List, rename, hide and drop indexes, and fix the errors MySQL raises while you do it

KEY IDEAS

- **An index** is a sorted B-tree copy of one or more columns: it turns a full table scan into a jump to the matching rows. It costs space and write time.
- **Automatic indexes:** every primary key (the clustered index: the table itself), every UNIQUE, every foreign key.
- **EXPLAIN** shows the plan (read the most indented line first); **EXPLAIN ANALYZE** runs the query and adds real times and row counts. End with `\G`.
- **Plan words:** Rows fetched before execution · Single-row lookup · Index lookup · Index range scan · Covering · Index scan · Table scan · Sort.
- **Composite index:** serves its leftmost prefix. Equality columns first, the range or sort column last.
- **Index killers:** a function or arithmetic around the column, a leading % in LIKE, a text column compared with a number.
- **Manage:** SHOW INDEX, RENAME INDEX, INVISIBLE before DROP; 1061 duplicate name, 1091 no such index, 1553 needed by a foreign key.

WORKED EXAMPLE: THE INDEXES YOU HAVE, AND A PLAN IN TABLE FORM

Every table already has indexes you never created by name. The employees table has three: the primary key, the UNIQUE email, and the index InnoDB made for the manager_id foreign key:

```
SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'employees'
ORDER BY INDEX_NAME;
```

```
+----------------------+-------------+------------+
| INDEX_NAME           | COLUMN_NAME | NON_UNIQUE |
+----------------------+-------------+------------+
| email                | email       |          0 |
| fk_employees_manager | manager_id  |          1 |
| PRIMARY              | id          |          0 |
+----------------------+-------------+------------+
```

The traditional plan format, one row per table, for the self-join of Lesson 4.3. Read `type` and `key`: the employees `e` are scanned (10 rows, `ALL`), and each manager `m` is one row found through the primary key (`eq_ref`, `PRIMARY`):

```sql
EXPLAIN FORMAT=TRADITIONAL
SELECT e.first_name, m.first_name AS manager
FROM employees AS e
JOIN employees AS m ON m.id = e.manager_id
WHERE e.department = 'support';
```

```
+----+-------------+-------+------------+--------+----------------------+--------
| id | select_type | table | partitions | type   | possible_keys        | key
+----+-------------+-------+------------+--------+----------------------+--------
|  1 | SIMPLE      | e     | NULL       | ALL    | fk_employees_manager | NULL
|  1 | SIMPLE      | m     | NULL       | eq_ref | PRIMARY              | PRIMARY
+----+-------------+-------+------------+--------+----------------------+--------
```

On ten rows the scan is the right plan. On a million employees you would index department, the column the WHERE filters.

GUIDED LAB

- SOURCE `labs/module-09/l6_1-indexes/plan_reading_starter.sql`: about a second to build 100,000 visits; every plan says Table scan.
- TODO 1: predict the new plan, then turn the CREATE INDEX on.
- TODO 2: the composite index for "customer 3 in June". Which column first?
- TODO 3: a UNIQUE index on ticket. Finished early? `index_killers_starter.sql`.

QUERY	PLAN BEFORE (SHAPE)	PLAN AFTER (SHAPE)
visits on June 1		
customer 3 in June		
ticket = '004242'		

PREDICT, THEN RUN

In a freshly loaded bookstore. The 6.2 items create a view, use it and drop it again; P3 changes a price inside a transaction and rolls it back. Write your prediction first.

P1

Predict the output, then run it. (categories.name is UNIQUE.)

```
SELECT INDEX_NAME, COLUMN_NAME
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'categories'
ORDER BY INDEX_NAME;
```

P2

Predict the output, then run it. (email is UNIQUE in employees.)

```
EXPLAIN SELECT * FROM employees WHERE email = 'nina.kowalski@bookstore.example
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Tune the Sales Reports (lesson page, "Project"), `tune_sales.sql`. In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: report A measured; why it needs no new index, as a comment
- ☐ Steps 2 and 3: report B's table scan, then its range rewrite (Index range scan)
- ☐ Step 4: report C with a three-column index: no Filter or Sort line left
- ☐ Step 5: keep both composite indexes? (reviewed with EXPLAIN)
- ☐ Step 6: the final index list

SELF-CHECK

Q1

Which index does every InnoDB table have, even though you never created one by name?

A) One index per column

B) The primary key: the rows themselves are stored in its order (the clustered index)

C) An index on every VARCHAR column

D) None: MySQL scans every table until you add indexes

Q2

`sales_log` has an index on `(customer_id, sold_on)` and no other index. Which query can NOT seek with it?

A) `WHERE customer_id = 3`

B) `WHERE customer_id = 3 AND sold_on >= '2025-03-01'`

C) `WHERE sold_on >= '2025-03-01' AND customer_id = 3`

D) `WHERE sold_on BETWEEN '2025-03-01' AND '2025-03-31'`

Q3

`sold_on` has an index. Rewrite `WHERE YEAR(sold_on) = 2024` so MySQL can use it.

Q4

`receipt_no` is `CHAR(7)` with a unique index, yet `WHERE receipt_no = 4242` shows `Table scan`. Why?

A) The text column is converted to a number on every row, so the index order no longer helps

B) Unique indexes cannot be used for equality

C) The table needs ANALYZE TABLE first

D) 4242 is too small: receipt numbers start at 1000000

Q5

An index `idx_sales_customer_date (customer_id, sold_on)` exists. You run `CREATE INDEX idx_sales_customer ON sales_log (customer_id);`. What happens?

A) `ERROR 1061: Duplicate key name`

B) MySQL refuses: the column is already indexed

C) It succeeds without an error or a warning, and the new index is pure overhead

D) It replaces the composite index

Q6

`DROP INDEX fk_orders_customer ON orders;` fails. Which error number, and why does MySQL refuse?

JOURNAL

Think of a physical index you use (a contact list, a library catalog, a recipe book). What is it sorted by? Which questions does it answer instantly, and which force you to scan every page? Map one of each to a SQL query and the index that would or would not help.

6.2 Views

BY THE END YOU CAN

- Explain what a view stores (a query, not data) and read the definition MySQL saves
- Create, query, replace and drop views, including views over joins and aggregates
- Predict whether a view is updatable, and protect it with WITH CHECK OPTION
- Use a view to hide sensitive columns, and grant access to the view alone
- Recognize and fix the errors views raise (1050, 1288, 1356, 1369, 1393)

KEY IDEAS

- **A view** is a saved SELECT: it stores the query, never the rows, and is recomputed every time you read it.
- **Read it like a table;** put ORDER BY in the query that reads it, not in the view.
- **CREATE OR REPLACE VIEW** makes scripts rerunnable; DROP VIEW removes only the definition.
- **Updatable** when each view row is one base row: no GROUP BY, aggregates, DISTINCT, UNION or window functions. A join view may change one table per statement.
- **WITH CHECK OPTION** refuses writes that would push a row out of the view (1369).
- **SQL SECURITY DEFINER:** an account with SELECT on the view alone sees the view's columns and rows, and nothing else of the table.
- **Frozen and fragile:** SELECT * is expanded at creation; renamed or dropped columns break a view (1356).

WORKED EXAMPLE: A VIEW THAT FILTERS A WINDOW

A view can hold a window function. Since a view is read like a table, the query that reads it can filter on the window's result: the CTE pattern of Session 8, saved under a name:

```
CREATE VIEW v_price_rank AS
SELECT title, price,
       ROW_NUMBER() OVER (ORDER BY price DESC, title) AS pos
FROM books;
SELECT pos, title, price FROM v_price_rank WHERE pos <= 3 ORDER BY pos;
```

```
+-----+-----------------------+-------+
| pos | title                 | price |
+-----+-----------------------+-------+
|   1 | Lord of the Rings     | 15.99 |
|   2 | Dune                  | 14.99 |
|   3 | To Kill a Mockingbird | 11.99 |
+-----+-----------------------+-------+
```

Is it updatable? A window function makes each view row depend on other rows, so MySQL says no:

```
SELECT TABLE_NAME, IS_UPDATABLE
FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'v_price_rank';
DROP VIEW v_price_rank;
```

```
+--------------+--------------+
| TABLE_NAME   | IS_UPDATABLE |
+--------------+--------------+
| v_price_rank | NO           |
+--------------+--------------+
```

GUIDED LAB

- SOURCE `labs/module-09/l6_2-views/report_views_starter.sql`: seven authors, and an empty "sold nothing" list.
- TODO 1: LEFT JOINs and COALESCE. Predict first: how many authors now?
- TODO 2: create `v_city_orders` (no GROUP BY inside the view). TODO 3: replace v_author_sales with one more column.
- Finished early? `guarded_views_starter.sql`.

PREDICT, THEN RUN

In a freshly loaded bookstore. The 6.2 items create a view, use it and drop it again; P3 changes a price inside a transaction and rolls it back. Write your prediction first.

P3

Predict the output, then run it. (A view is recomputed every time.)

```
CREATE VIEW v_pricey AS SELECT title FROM books WHERE price > 12;
START TRANSACTION;
UPDATE books SET price = 9.00 WHERE id = 1;
SELECT title FROM v_pricey ORDER BY title;
ROLLBACK;
DROP VIEW v_pricey;
```

P4

Predict the output, then run it. (DISTINCT in a view.)

```
CREATE VIEW v_cities AS SELECT DISTINCT city FROM customers;
SELECT TABLE_NAME, IS_UPDATABLE
FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'v_cities';
DROP VIEW v_cities;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Reporting Layer (lesson page, "Project"), `reporting_views.sql`. In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Steps 1 to 3: v_book_catalog, v_order_totals, v_book_sales (Gatsby and Catcher at 0 copies)
- ☐ Step 4: v_best_customers on v_customer_summary
- ☐ Step 5: the IS_UPDATABLE list (all NO)
- ☐ Step 6: the script runs twice without an error

SELF-CHECK

Q7

What does MySQL store when you create a view?

A) Only the query: the rows are computed every time the view is read

B) A copy of the rows, refreshed every night

C) A copy of the rows, refreshed on every INSERT

D) An index on the base tables

Q8

What happens when the second statement runs?

```
CREATE VIEW v_ny AS
SELECT id, first_name, city
FROM customers
WHERE city = 'New York'
WITH CHECK OPTION;
UPDATE v_ny SET city = 'Boston' WHERE id = 1;
```

Q9

Which of these views accepts an UPDATE that passes through to its base table?

A) `SELECT author_id, COUNT(*) AS books FROM books GROUP BY author_id`

B) `SELECT DISTINCT city FROM customers`

C) `SELECT id, title, price FROM books WHERE price < 10`

D) `SELECT title FROM books UNION SELECT name FROM authors`

Q10

`v_all` was created as `SELECT * FROM products`. Then a column `color` is added to products. What does `SELECT * FROM v_all` return?

- A) `ERROR 1356`
- B) All columns, including color
- C) An empty result until the view is refreshed
- D) The old columns only, without color

Q11

Why does the course put ORDER BY in the query that reads a view, and not inside the view?

Q12

An account has `SELECT` on `v_staff_directory` only. The view uses the default `SQL SECURITY DEFINER`. What can the account do?

- A) Read every column of employees, salaries included
- B) Read the directory through the view, but not the employees table itself
- C) Nothing: it has no privilege on employees
- D) Update salaries through the view

◆ JOURNAL

Think of a spreadsheet or report several people share. Who should see which columns and rows? Sketch two or three views with names, and note which one (if any) should allow edits and what its check option would protect.

CLINIC

◆ EIGHT ERRORS, ONE FIX EACH

SOURCE `labs/module-09/l6_1-indexes/debug_indexes.sql` and `labs/module-09/l6_2-views/debug_views.sql`.

SECTION	ERROR NUMBER	CAUSE	FIX
6.1 A			
6.1 B			
6.1 C			
6.1 D			
6.2 A			
6.2 B			
6.2 C			
6.2 D			

EXIT TICKET

Three questions, no notes.

E1

A query filters `WHERE customer_id = 5 AND book_id = 9 ORDER BY sold_on`. Write the one index that serves the filter and the sort.

E2

Name two habits that stop MySQL from using an index on a column.

E3

Give one reason to create a view, and one thing a view does NOT do for you.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish `tune_sales.sql` (Steps 4 to 6) and `reporting_views.sql` (Steps 4 to 6)
- ☐ Lesson 6.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 6.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Labs not finished in class: `index_killers_starter.sql`, `guarded_views_starter.sql`
- ☐ Find a spreadsheet value that repeats on many rows (for Lesson 6.3)
- ☐ Bring to Session 10: a working bookstore, my spreadsheet example, the course folder with `labs/`, both project files, this workbook

GLOSSARY

TERM	MEANING
index	A sorted B-tree copy of one or more columns with pointers to the rows
full table scan	Reading every row to answer a query (Table scan / type ALL)
clustered index	InnoDB's primary key: the rows themselves, stored in key order
secondary index	Any other index; it stores the column values plus the primary key
composite index	An index on several columns, sorted by the first, then the second ...

TERM	MEANING
leftmost prefix	The first column(s) of a composite index, which it can seek on
covering index	An index that holds every column a query needs, so the table is never read
`EXPLAIN` / `EXPLAIN ANALYZE`	Show the plan / run the query and add real times and rows
invisible index	Maintained on writes but ignored by the optimizer (test a drop safely)
view	A named, saved SELECT; holds no rows
base table	A real table a view reads from
updatable view	A view whose rows map one-to-one to base rows, so writes pass through
`WITH CHECK OPTION`	Rejects writes through a view that would produce rows it cannot show
`SQL SECURITY DEFINER`	The view reads its tables with its creator's privileges

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 10: Normalization and Stored Programs

Your workbook for Session 10: a warm-up on Session 9, the key ideas of Lessons 6.3 and 7.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, the clinic's design review and error table, and your exit ticket.

Lessons 6.3–7.1 | Session 10 · 3 hours | Name: _____

SESSION GOALS

- Recognize update, insert and delete anomalies, and say what causes all three.
- Find a table's functional dependencies and key, and normalize it to 3NF.
- Migrate data into normalized tables with INSERT ... SELECT DISTINCT and prove with EXCEPT that nothing was lost.
- Write stored procedures with IN and OUT parameters, and functions with true characteristics.
- Refuse bad input with SIGNAL, and read the errors broken routines raise.

WARM-UP

FIVE QUESTIONS ON SESSION 9, NO NOTES

W1

Which plan line do you expect, and why is there an index at all?

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 4\G
```

W2

Is this view updatable?

```
CREATE VIEW v_cheap AS SELECT id, title, price FROM books WHERE price < 9;
SELECT TABLE_NAME, IS_UPDATABLE FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = 'bookstore' AND TABLE_NAME = 'v_cheap';
DROP VIEW v_cheap;
```

W3

An index on (customer_id, sold_on): can a query on a sold_on range alone seek with it?

W4

What does MySQL store when you create a view?

W5

What does WITH CHECK OPTION refuse?

My spreadsheet: one value that repeats on many rows, and the table it should live in:

LESSONS

6.3 Database Design & Normalization

BY THE END YOU CAN

- Recognize update, insert and delete anomalies, and show each one with a query
- Find the functional dependencies and the key of a table
- Normalize a flat table to first, second and third normal form (1NF, 2NF, 3NF)
- Build the normalized tables with INSERT ... SELECT DISTINCT and prove with EXCEPT that no data was lost
- Read an ER diagram, and decide when storing derived or duplicated data is worth the risk

KEY IDEAS

- **Anomalies:** update (copies drift), insert (a fact needs another fact to exist), delete (removing one fact removes another). One cause: facts about different things in one row.
- **A → B:** same A, same B. Dependencies come from business rules; data can disprove one, never prove it.
- **1NF:** one value per cell, no repeating columns, a key. **2NF:** no column depends on part of a composite key. **3NF:** no column depends on a non-key column.
- **Migrate** with INSERT ... SELECT DISTINCT, parents first, children last, joining back on the natural identifiers.
- **Prove it lossless** with EXCEPT in both directions: two empty sets.
- **ER diagrams:** a line's "one" end holds the primary key, the "many" end the foreign key; many at both ends means a missing junction table.
- **Do not store what you can compute.** Denormalize only for a measured problem, with a plan to keep the copy in sync.

WORKED EXAMPLE: A KEY AND A DEPENDENCY, TESTED ON THE BOOKSTORE

Finding a key with data: if a set of columns has as many distinct values as the table has rows, it COULD be the key (the business decides whether it IS). In order_items, an order has several lines, but each book appears once per order:

```
SELECT COUNT(*)                          AS item_rows,
       COUNT(DISTINCT order_id)          AS orders,
       COUNT(DISTINCT order_id, book_id) AS order_book_pairs
FROM order_items;
```

```
+-----------+--------+------------------+
| item_rows | orders | order_book_pairs |
+-----------+--------+------------------+
|        18 |     12 |               18 |
+-----------+--------+------------------+
```

Testing a dependency: has any book ever been sold at two different prices? No counterexample today:

```sql
SELECT book_id, COUNT(DISTINCT price_at_time) AS prices_paid
FROM order_items
GROUP BY book_id
HAVING COUNT(DISTINCT price_at_time) > 1;
```

```
Empty set
```

But `book_id → price_at_time` is NOT a rule: price_at_time records what was paid on that order, and prices change. The data cannot tell you that; the business can. That is why price_at_time stays in order_items.

◆ GUIDED LAB

- SOURCE `labs/module-10/l6_3-normalization/anomalies_starter.sql`: the workshop sheet, six rows.
- TODO 1: the update anomaly. Predict first: how many dates will Bookbinding Basics have?
- TODO 2: the delete anomaly. What does the store still know about Speed Reading?
- TODO 3: two dependency tests. Which one finds a counterexample?

DEPENDENCY	COUNTEREXAMPLE?	SO IT IS ...
workshop → date, room, host, fee		
host_email → host_name		
host_email → workshop		

The key of the sheet, and the four tables it suggests:

◆ PREDICT, THEN RUN

P1 runs in a practice database (`CREATE DATABASE scratch; USE scratch;` first, `DROP DATABASE scratch;` after); P2 in the freshly loaded bookstore. Write your prediction first, then run it.

P1

Predict the output, then run it. (Five sheet rows, how many members?)

```
CREATE TABLE sheet (member VARCHAR(30), club VARCHAR(30));
INSERT INTO sheet VALUES ('Ana', 'Chess'), ('Ben', 'Chess'), ('Ana', 'Poetry')
                         ('Cy', 'Poetry'), ('Ben', 'Chess');
CREATE TABLE members (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(30) UNIC
INSERT INTO members (name) SELECT DISTINCT member FROM sheet ORDER BY member;
SELECT COUNT(*) AS sheet_rows, (SELECT COUNT(*) FROM members) AS members FROM
```

P2

Predict the output, then run it. (EXCEPT on the bookstore.)

```
SELECT title FROM books WHERE price < 9
EXCEPT
SELECT title FROM books WHERE in_stock = FALSE
ORDER BY title;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Normalize the Book Club Sheet (lesson page, "Project"), `book_club.sql`. In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: the sheet imported into normalize_lab (6 sign-ups)
- ☐ Step 2: dependencies and key written as comments, then checked with the query
- ☐ Step 3: members, leaders, clubs, memberships (composite key, two foreign keys)
- ☐ Step 4: migrated, parents first: 4 members, 2 leaders, 3 clubs, 6 memberships
- ☐ Step 5: both EXCEPT queries print Empty set

- ☐ Step 6: the Sci-Fi Circle moves to Wednesday with one updated row

SELF-CHECK

Q1

Alice's email is stored on three order rows. Someone corrects it on one row only, and now the table holds two emails for her. Which anomaly is this?

A) An update anomaly

B) An insert anomaly

C) A delete anomaly

D) A first normal form violation

Q2

A contacts table has the columns `name`, `phone1`, `phone2` and `phone3`. Which normal form does it break?

A) Second normal form

B) Third normal form

C) First normal form: a repeating group of columns

D) None: three columns are fine

Q3

`enrollments(student_id, course, course_room, grade)` has the key `(student_id, course)`. Each course meets in one room. Which rule does `course → course_room` break?

A) 1NF

B) 2NF: course_room depends on only part of the key

C) 3NF: a transitive dependency

D) None: course is part of the key

Q4

`employees(id, name, dept_id, dept_name)` has the key `id`. Which dependency breaks 3NF, and what is the fix?

Q5

You split a flat table into four, then run two EXCEPT queries in opposite directions (old rows EXCEPT the rebuilt join, and the other way). Both print Empty set. What does that prove?

A) Every new table has a primary key

B) The new tables contain no duplicates

C) The design is in 3NF

D) The four tables, joined back, hold exactly the original rows: the split lost nothing and added nothing

Q6

`SELECT member_email FROM club_signups GROUP BY member_email HAVING COUNT(DISTINCT member_name) > 1;` prints Empty set. What can you conclude?

A) `member_email → member_name` is proved

B) The data does not contradict member_email → member_name; whether it is a rule is a business decision

C) `member_name → member_email` holds

D) The table is in 3NF

JOURNAL

Find a spreadsheet in your own life. Write down one value that repeats on many rows, the update anomaly it could cause, and the table you would move it to. Which column would be the key of that new table?

7.1 Stored Procedures & Functions

BY THE END YOU CAN

- Explain what the DELIMITER client command does, and when you do not need it
- Create and call stored procedures with IN, OUT and INOUT parameters
- Write stored functions with the right characteristic (DETERMINISTIC, READS SQL DATA) and use them inside queries
- Use local variables, IF, CASE and WHILE in a routine, and reject bad input with SIGNAL
- List, inspect, replace and drop routines, and read the errors MySQL raises for broken ones (1064, 1418, 1172, 1304, 1644)

KEY IDEAS

- **Procedure:** `CALL name(...)`; result sets, OUT and INOUT parameters, changes data. **Function:** returns one value, used inside expressions.
- **DELIMITER // ... //** is a client command: the client waits for // so a body with semicolons is sent whole. Switch back with `DELIMITER ;`.
- **OUT parameters** land in user variables: `CALL p(1, @x); SELECT @x;`. SELECT ... INTO must find at most one row (1172).
- **DECLARE** local variables first in a block; `p_` and `v_` prefixes so names never shadow columns.
- **Function characteristics** (DETERMINISTIC, NO SQL, READS SQL DATA) are required with binary logging on (1418), and must be true.
- **SIGNAL SQLSTATE '45000'** refuses bad input with your own message (1644). Check before you change.
- **Rerunnable scripts:** `DROP PROCEDURE IF EXISTS` / `DROP FUNCTION IF EXISTS`, then CREATE (no CREATE OR REPLACE in MySQL).

WORKED EXAMPLE: A DISPLAY FUNCTION AND A DATE-RANGE PROCEDURE

A function for display, pure string work (DETERMINISTIC, NO SQL):

```
DELIMITER //
CREATE FUNCTION initials(p_name VARCHAR(100))
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
    RETURN CONCAT(LEFT(p_name, 1), '.', LEFT(SUBSTRING_INDEX(p_name, ' ', -1), 1),
END //
DELIMITER ;
SELECT name, initials(name) AS initials
FROM authors
WHERE id IN (1, 5, 9)
ORDER BY id;
```

```
+---------------+----------+
| name          | initials |
+---------------+----------+
| Frank Herbert | F.H.     |
| Jane Austen   | J.A.     |
| Ray Bradbury  | R.B.     |
+---------------+----------+
```

A procedure that takes two dates and returns a result set; the range "from, up to but not including, to" is the index-friendly habit from Session 9:

```
DELIMITER //
CREATE PROCEDURE orders_between(IN p_from DATE, IN p_to DATE)
BEGIN
    SELECT o.id, o.order_date, c.first_name
    FROM orders AS o
    JOIN customers AS c ON c.id = o.customer_id
    WHERE o.order_date >= p_from AND o.order_date < p_to
    ORDER BY o.order_date;
END //
DELIMITER ;
CALL orders_between('2026-03-01', '2026-04-01');
```

```
+----+------------+------------+
| id | order_date | first_name |
+----+------------+------------+
|  3 | 2026-03-10 | Bob        |
|  7 | 2026-03-28 | Bob        |
+----+------------+------------+
```

◆ GUIDED LAB

- SOURCE `labs/module-10/l7_1-stored-procedures/procedures_starter.sql`. Every routine starts with DROP ... IF EXISTS: edit, save, SOURCE again.
- TODO 1: category_shelf filters on its parameter. TODO 2: author_stats's second OUT value.
- TODO 3: the percent guard, before the other check; then `CALL raise_price(6, 80);` at the prompt.
- Finished early? `functions_starter.sql`.

PREDICT, THEN RUN

In the freshly loaded bookstore (each creates a procedure; `DROP PROCEDURE` it afterward, or reload). Write your prediction first, then run it.

P3

Predict the output, then run it. (INOUT, called twice.)

```
DELIMITER //
CREATE PROCEDURE double_it(INOUT p_n INT)
BEGIN
    SET p_n = p_n * 2;
END //
DELIMITER ;
SET @n = 10;
CALL double_it(@n);
CALL double_it(@n);
SELECT @n AS n;
```

P4

Predict the output, then run it. (Asimov has no books.)

```
DELIMITER //
CREATE PROCEDURE first_title(IN p_author_id INT, OUT p_title VARCHAR(200))
BEGIN
    SELECT MIN(title) INTO p_title FROM books WHERE author_id = p_author_id;
END //
DELIMITER ;
SET @t = 'nothing yet';
CALL first_title(10, @t);
SELECT @t AS asimov_first_title;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Bookstore Routine Library (lesson page, "Project"), `bookstore_routines.sql`. In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: order_total (READS SQL DATA): Alice's orders 34.97, 7.99, 15.99
- ☐ Step 2: customer_tier: Gold for Alice, New for Carol and Hank
- ☐ Step 3: customer_report: two result sets
- ☐ Step 4: add_book with SIGNAL and an OUT id
- ☐ Step 5: a free book is refused and nothing is inserted
- ☐ Step 6: the file runs twice; test CALLs stay at the prompt

SELF-CHECK

Q7

In the mysql client, without changing the delimiter, you type a procedure whose body is `BEGIN SELECT ...; END;`. What happens?

A) Two `ERROR 1064` lines: the client sent the text in two pieces at the semicolons

B) `ERROR 1418`

C) The procedure is created

D) `ERROR 1304: PROCEDURE already exists`

Q8

What does this print? (Books under $9: The Great Gatsby, The Catcher in the Rye, Animal Farm, Pride and Prejudice.)

```
CREATE PROCEDURE count_cheaper(IN p_price DECIMAL(8, 2), OUT p_n INT)
    SELECT COUNT(*) INTO p_n FROM books WHERE price < p_price;
CALL count_cheaper(9.00, @n);
SELECT @n AS cheaper;
```

Q9

A stored function returns `ROUND(p_amount * 1.08, 2)` and reads no table. Which characteristics are true of it?

A) `READS SQL DATA`

B) `MODIFIES SQL DATA`

C) `NOT DETERMINISTIC` (the default)

D) `DETERMINISTIC` and `NO SQL`

Q10

A procedure runs `SELECT title INTO p_title FROM books WHERE author_id = p_author_id;`. What happens when it is called for J.R.R. Tolkien, who has two books?

A) p_title gets the first title in id order

B) p_title is NULL

C) `ERROR 1172 (42000): Result consisted of more than one row`

D) p_title holds both titles, separated by a comma

Q11

What does the CALL print?

```
DELIMITER //
CREATE PROCEDURE check_qty(IN p_qty INT)
BEGIN
    IF p_qty < 1 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Quantity must be at least
    END IF;
    SELECT p_qty AS accepted;
END //
DELIMITER ;
CALL check_qty(0);
```

Q12

Which of these can a stored function NOT do?

A) Return one value

B) Read tables (declared READS SQL DATA)

C) Run `COMMIT` or `START TRANSACTION`

D) Be used inside `WHERE`

JOURNAL

Pick one rule from a business you know (a café's loyalty card, a library's late fee, a gym's levels). Function, procedure or app? Write your choice and two reasons, including one reason against it.

CLINIC

DESIGN REVIEW

The two spreadsheets from the warm-up, designed together:

SPREADSHEET	DEPENDENCIES (A → B)	KEY	TABLES
1			
2			

EIGHT ERRORS, ONE FIX EACH

SOURCE `labs/module-10/l6_3-normalization/debug_design.sql` and `labs/module-10/l7_1-stored-procedures/debug_routines.sql`.

SECTION	ERROR NUMBER	CAUSE	FIX
6.3 A			
6.3 B			
6.3 C			
7.1 A			
7.1 B			
7.1 C			
7.1 D			
7.1 E			

EXIT TICKET

Three questions, no notes.

E1

`order_lines(order_id, book_title, order_date, book_price, quantity)`, key `(order_id, book_title)`: name one partial dependency and the table it moves to.

E2

Why does a procedure that changes data check its input before the UPDATE, and how does it refuse?

E3

Function or procedure: "show each book's price with tax in a SELECT"? "Add a book and its new author, and return the new id"?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish `book_club.sql` (Steps 4 to 6) and `bookstore_routines.sql` (Steps 3 to 6)
- ☐ Lesson 6.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 7.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)

- ☐ Both Quick Quizzes and both journal entries
- ☐ Labs: `normalize_starter.sql`, `functions_starter.sql`
- ☐ Check that I can open two mysql client windows at once (Lesson 7.2)
- ☐ Review for Checkpoint 2 (Modules 5 and 6) in Session 11
- ☐ Bring to Session 11: a laptop with two client windows, `bookstore.sql`, `book_club.sql`, `bookstore_routines.sql`, the course folder with `labs/`, this workbook

GLOSSARY

TERM	MEANING
anomaly (update / insert / delete)	Trouble caused by storing facts about different things in one table
functional dependency (A → B)	Same A, same B: a business rule
key	A minimal set of columns that determines every other column
1NF / 2NF / 3NF	One value per cell / no dependency on part of the key / no dependency on a non-key column
transitive dependency	A non-key column that depends on another non-key column
surrogate key	An AUTO_INCREMENT id used as the primary key
junction table	A table with a composite primary key that carries a many-to-many relationship
lossless decomposition	A split whose tables, joined back, give exactly the original rows
denormalization	Storing a copy or a computed value on purpose, for speed
stored procedure	A named routine run with CALL; result sets, OUT parameters, changes data
stored function	A named routine that returns one value inside an expression
`DELIMITER`	A client command that changes the statement end marker for a while
characteristic	DETERMINISTIC, NO SQL, READS SQL DATA ...: what a function promises it does
`SIGNAL`	Raises your own error (SQLSTATE '45000', error 1644) and stops the routine

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 11: Transactions and JSON

Your workbook for Session 11: the warm-up, Checkpoint 2, the key ideas of Lessons 7.2 and 7.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 7.2–7.3 | Session 11 · 3 hours | Name: _____

SESSION GOALS

- Show what you can do with Modules 5 and 6 in Checkpoint 2: CTEs, window functions, indexes, views and normalization.
- Group changes into all-or-nothing transactions with COMMIT, ROLLBACK and savepoints, and know which statements commit by themselves.
- Predict what a second connection sees and when it has to wait, and read lock errors 1205 and 1213.
- Write a procedure whose handler rolls back and passes the error on.
- Store, read, change, build and index JSON, and decide when a column or a table is the better choice.

WARM-UP: NORMALIZATION AND STORED PROGRAMS

FIVE QUESTIONS (LESSONS 6.3 AND 7.1)

W1

A book-club sheet has a cell that reads `Dune, 1984`. Which normal form does that break, and what is the fix?

W2

"The key, the whole key, and nothing but the key." Which part of the phrase is 2NF, and which is 3NF?

W3

Which program reads the line `DELIMITER //`: the server, the client or the procedure?

W4

You want `SELECT title, sale_price(price) FROM books` to work. Do you write a stored procedure or a stored function?

W5

Predict the on_sale column, then run it (it creates a small function in bookstore).

```
DROP FUNCTION IF EXISTS sale_price;
CREATE FUNCTION sale_price(p_price DECIMAL(8, 2))
RETURNS DECIMAL(8, 2) DETERMINISTIC
RETURN ROUND(p_price * 0.8, 2);
SELECT title, price, sale_price(price) AS on_sale
FROM books
WHERE id IN (1, 2)
ORDER BY id;
```

CHECKPOINT 2: MODULES 5 AND 6 (30 MINUTES)

Alone, open notes: lesson pages and your own project files are allowed, a neighbor is not. Parts 1 to 4 go in `labs/module-11/checkpoint-2/checkpoint2_starter.sql`: replace each TODO line with your query and SOURCE the file again. Part 5 is on paper, here. About five minutes per part; a part you cannot finish, describe in a comment what you would do.

PART	LESSONS	TASK
Part 1	5.1, 5.2	The best seller(s) of each category by copies sold (SUM of order_items.quantity), ties sharing first place: a CTE for the copies and a window function to rank. Columns category, title, copies, sorted by category.
Part 2	5.1	Everyone who reports to Daniel Reyes (employee 2), directly or through someone else, with a level column (1 = reports to Daniel directly), sorted by level and id: a recursive CTE.
Part 3	6.2	The view v_customer_revenue (customer_id, customer, orders, revenue) for customers who have ordered, then a query on the view: customers with revenue above 30, highest first.
Part 4	6.1	The plan of SELECT id, first_name FROM customers WHERE city = 'Chicago' before and after an index that helps it: the first line of each plan.
Part 5	6.3	On paper: the table class_signups below. Name its key, the dependencies that break 2NF and 3NF, and the tables (with keys) you would split it into.

YOUR KEY NUMBERS (COPY THEM FROM YOUR OUTPUT)

PART	WHAT TO WRITE DOWN	MY ANSWER
1	One row per category: the title(s) and copies	
2	How many people, and who is at level 2	
3	The customers above 30 and Alice's orders and revenue	
4	The first line of the plan before and after your index	

PART 5: NORMALIZE CLASS_SIGNUPS

An art school keeps its sign-ups in one sheet. Each row is one student in one class.

STUDENT_EMAIL	STUDENT_NAME	CLASS_CODE	CLASS_TITLE	ROOM	TEACHER	TEACHER_PHONE
ana@example.com	Ana Ruiz	POT1	Pottery Basics	R12	Lee Wong	555-0101
ana@example.com	Ana Ruiz	WAT2	Watercolor II	R7	Mia Ford	555-0102
ben@example.com	Ben Ito	POT1	Pottery Basics	R12	Lee Wong	555-0101
cy@example.com	Cy Park	GLS1	Glass Fusing	R12	Lee Wong	555-0101

1. The key of this table (which column or columns identify a row):
2. A dependency that breaks 2NF, and why:
3. A dependency that breaks 3NF, and why:
4. The tables you would build instead, with their keys and foreign keys:

What is assessed (five criteria, one per part): the CTE and window function; the recursive CTE; the view; the index and reading EXPLAIN; normalization to 3NF. Your instructor returns the page in Session 12 with one sentence of advice.

LESSONS

7.2 Transactions

BY THE END YOU CAN

- Group changes with START TRANSACTION, COMMIT and ROLLBACK, and explain autocommit and implicit commits
- Undo part of a transaction with SAVEPOINT and ROLLBACK TO SAVEPOINT
- Predict what a second connection sees and when it has to wait, under MySQL's default REPEATABLE READ isolation
- Lock rows on purpose with SELECT ... FOR UPDATE, and read lock errors 1205 and 1213
- Write a stored procedure whose error handler rolls back the whole transaction and passes the error on

KEY IDEAS

- **A transaction is all or nothing:** `START TRANSACTION`, your statements, then `COMMIT` (keep) or `ROLLBACK` (discard).
- **Autocommit** (on by default) commits every statement by itself. START TRANSACTION suspends it until COMMIT or ROLLBACK.
- **Implicit commits:** CREATE, ALTER, DROP, TRUNCATE and account statements commit the open transaction before they run.
- **A failed statement undoes only itself:** after an error inside a transaction, ROLLBACK (or let a handler do it).
- **Savepoints** undo part of a transaction and keep it open: `SAVEPOINT s;` ... `ROLLBACK TO SAVEPOINT s;`
- **Isolation (REPEATABLE READ):** plain reads see committed data from a snapshot and never wait; writers wait for row locks (1205 after the timeout).
- **FOR UPDATE** locks the rows you are about to change; a **deadlock** (1213) rolls one transaction back: retry it whole.
- **In procedures:** `DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;`

WORKED EXAMPLE: A GIFT CARD PAYS FOR A BOOK

A gift card pays for a book: read the balance with a lock, charge the card, create the order and its line, all in one transaction. The lock (FOR UPDATE) keeps a second cashier from spending the same balance between the check and the charge.

```
CREATE TABLE gift_cards (
    code    VARCHAR(10) PRIMARY KEY,
    balance DECIMAL(8, 2) NOT NULL CHECK (balance >= 0)
);
INSERT INTO gift_cards VALUES ('GC-100', 25.00);
```

```
START TRANSACTION;
SELECT balance FROM gift_cards WHERE code = 'GC-100' FOR UPDATE;
UPDATE gift_cards SET balance = balance - 14.99 WHERE code = 'GC-100';
INSERT INTO orders (customer_id, order_date) VALUES (3, '2026-06-30');
INSERT INTO order_items (order_id, book_id, quantity, price_at_time)
VALUES (LAST_INSERT_ID(), 1, 1, 14.99);
COMMIT;
```

```
+---------+
| balance |
+---------+
|   25.00 |
+---------+
```

Both the charge and the order are there, or neither would be:

```
SELECT (SELECT balance FROM gift_cards WHERE code = 'GC-100') AS card_left,
       (SELECT COUNT(*) FROM orders WHERE customer_id = 3)        AS carol_orders
```

```
+-----------+--------------+
| card_left | carol_orders |
+-----------+--------------+
|     10.01 |            1 |
+-----------+--------------+
```

In an app, the error branch would ROLLBACK; in a procedure, a handler does it (Lesson 7.2's transfer).

GUIDED LAB

- SOURCE `labs/module-11/l7_2-transactions/loyalty_points_starter.sql` (log in with `mysql -u root -p bookstore`).
- TODO 1 together, TODOs 2 to 4 in pairs. Each is a small edit, then SOURCE again. Predict before each run.
- TODO 4: write in one sentence why the ROLLBACK did nothing as shipped.

- Done when your results match the "Expected when done" comment at the end of the file.
- Finished early? `return_item_starter.sql`, then `debug_lost_points.sql` in the same folder.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code in a practice database (`CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done).

P1

Predict the output, then run it. (autocommit switched off, then ROLLBACK.)

```
CREATE TABLE counter (n INT);
INSERT INTO counter VALUES (1);
SET autocommit = 0;
UPDATE counter SET n = n + 10;
ROLLBACK;
SET autocommit = 1;
SELECT n FROM counter;
```

P2

Predict the output, then run it. (Two seats and a savepoint.)

```
CREATE TABLE seats (id INT PRIMARY KEY, taken INT NOT NULL);
INSERT INTO seats VALUES (1, 0), (2, 0);
START TRANSACTION;
UPDATE seats SET taken = 1 WHERE id = 1;
SAVEPOINT one_seat;
UPDATE seats SET taken = 1 WHERE id = 2;
ROLLBACK TO SAVEPOINT one_seat;
COMMIT;
SELECT SUM(taken) AS seats_taken FROM seats;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

An All-or-Nothing Checkout (lesson page, "Project"). In class: the setup and Step 1. At home: Steps 2 to 6.

- ☐ Setup: the accounts table (the lesson's Setup block)
- ☐ Step 1: checkout.sql creates the procedure; `CALL checkout(1, 11, 1, @order_id);` works
- ☐ Steps 2 to 4: a good order; four calls that must fail and leave no order behind
- ☐ Step 5: the integrity audit is an Empty set; I can explain the gap in the order ids
- ☐ Step 6: the file runs twice in a row without an error

SELF-CHECK

Q1

You type `START TRANSACTION;` and an UPDATE that succeeds, then close the terminal window without COMMIT. What happens to the change?

A) It is kept: the UPDATE succeeded

B) The server rolls it back when the connection ends

C) It waits, and is committed the next time you log in

D) It is kept, but only your account can see it

Q2

What does the last statement print?

```
CREATE TABLE stock (id INT PRIMARY KEY, qty INT NOT NULL);
INSERT INTO stock VALUES (1, 5), (2, 5);
START TRANSACTION;
UPDATE stock SET qty = qty - 1 WHERE id = 1;
SAVEPOINT s1;
UPDATE stock SET qty = qty - 1 WHERE id = 2;
ROLLBACK TO SAVEPOINT s1;
UPDATE stock SET qty = qty + 3 WHERE id = 2;
COMMIT;
SELECT id, qty FROM stock ORDER BY id;
```

Q3

Which statement, typed inside an open transaction, commits that transaction on the spot?

A) `SAVEPOINT before_change;`

B) `SELECT price FROM books WHERE id = 1 FOR UPDATE;`

C) `ALTER TABLE books ADD COLUMN rating INT;`

D) `ROLLBACK TO SAVEPOINT before_change;`

Q4

Window A runs `START TRANSACTION;` and `UPDATE accounts SET balance = 0 WHERE id = 1;` and does not commit. Window B then runs `SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;`. What does B get?

A) 0, A's new value

B) The last committed balance, at once

C) ERROR 1213 at once: a deadlock

D) Nothing yet: B waits for A's row lock, and fails with ERROR 1205 if A holds it past the timeout

Q5

In the transfer procedure's handler, why does RESIGNAL come right after ROLLBACK?

A) So the caller still receives the original error after the half-done changes are undone

B) RESIGNAL commits whatever the handler did not roll back

C) Without it, the handler would run again and again

D) Every handler must end with RESIGNAL or the procedure does not compile

Q6

An app's checkout fails with `ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction`. What should the app do?

A) Nothing: the statements before the deadlock were saved

B) Run only the statement that failed again

C) Raise innodb_lock_wait_timeout so it never happens again

D) Run the whole transaction again from the start

JOURNAL

Describe the two-window experiment in your own words, as if explaining it to a friend: who changed what, who could see it, who had to wait, and why. Then name one situation outside computing that works like a row lock.

7.3 JSON Columns

BY THE END YOU CAN

- Add a JSON column and explain how MySQL validates and stores what you put in it
- Read values with ->, ->> and JSON_EXTRACT, and filter rows on them
- Change parts of a document with JSON_SET, JSON_INSERT, JSON_REMOVE and JSON_ARRAY_APPEND
- Turn rows into JSON with JSON_OBJECT and JSON_ARRAYAGG, and JSON back into rows with JSON_TABLE
- Index a JSON path through a generated column, and decide when data belongs in real columns instead

KEY IDEAS

- **A JSON column** holds one validated document per row (error 3140 for invalid JSON), stored in a binary form with sorted keys.
- **Paths** start at `$`: `$.format`, `$.awards[0]`, `$.awards[last]`, `$.awards[*]`.
- `->` (= JSON_EXTRACT) returns JSON, with quotes; `->>` returns plain text for display, comparison and sorting.
- **A missing key is NULL, not an error,** so a misspelled key fails silently.
- **JSON_SET / INSERT / REPLACE / REMOVE / ARRAY_APPEND** return a changed copy; UPDATE stores it.
- **JSON_OBJECT and JSON_ARRAYAGG** build JSON from rows; **JSON_TABLE** turns JSON back into rows.
- **Index a path** through a generated column; protect important keys with a CHECK.
- **Core data stays in columns and tables:** what every row has, what you join on or calculate with.

WORKED EXAMPLE: A JSON MESSAGE BECOMES ROWS

A review widget sends one JSON message with several reviews. JSON_TABLE turns it into rows that join and group like any table.

```
SET @reviews = '{"source": "web", "reviews": [
    {"book_id": 1, "stars": 5}, {"book_id": 1, "stars": 4},
    {"book_id": 6, "stars": 5}, {"book_id": 2, "stars": 3}]}';
```

```
SELECT b.title, COUNT(*) AS reviews, AVG(r.stars) AS avg_stars
FROM JSON_TABLE(@reviews, '$.reviews[*]'
         COLUMNS (book_id INT PATH '$.book_id', stars INT PATH '$.stars')) AS r
JOIN books AS b ON b.id = r.book_id
GROUP BY b.id, b.title
ORDER BY avg_stars DESC, b.title;
```

```
+------------+---------+-----------+
| title      | reviews | avg_stars |
+------------+---------+-----------+
| The Hobbit |       1 |    5.0000 |
| Dune       |       2 |    4.5000 |
| 1984       |       1 |    3.0000 |
+------------+---------+-----------+
```

The stars come from the message, but the titles come from books: never trust names or prices sent by a browser.

GUIDED LAB

- SOURCE `labs/module-11/l7_3-json-columns/reader_prefs_starter.sql`.
- TODOs 1 and 2 together, 3 to 6 in pairs. The file adds the prefs column and removes it at the end, so SOURCE it after every edit.
- TODO 4: why is has_genres 0 for Dave but NULL for Carol?

- If a run stopped halfway (`ERROR 1060` next time): `SOURCE labs/_states/bookstore.sql`, then run the lab again.
- Finished early? `order_documents_starter.sql`, then `debug_bad_json.sql`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code in a practice database (`CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` when you are done).

P3

Predict the output, then run it. (Valid or not, and what kind?)

```
SELECT JSON_VALID('{"a": 1}') AS ok,
       JSON_VALID("{'a': 1}")  AS single_quotes,
       JSON_TYPE('[1, 2]')     AS kind;
```

P4

Predict the output, then run it. (INSERT and REPLACE on keys that are and are not there.)

```
SET @doc = '{"format": "paperback"}';
SELECT JSON_INSERT(@doc, '$.format', 'ebook')  AS ins,
       JSON_REPLACE(@doc, '$.pages', 300)      AS rep;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Catalog Feed (lesson page, "Project"). In class: the setup and Step 1. At home: Steps 2 to 6.

- ☐ Setup: the attributes column and its UPDATEs, the format column, the chk_books_format CHECK
- ☐ Step 1: two ebooks with a file_mb key
- ☐ Steps 2 and 3: titles per format; four award rows
- ☐ Steps 4 and 5: the feed (four documents); the series index
- ☐ Step 6: catalog_feed.sql reloads the database and runs twice

SELF-CHECK

Q7

On the web page, `CONCAT(title, ': ', attributes->'$.series')` prints `Dune: "Dune Chronicles"`. Which change removes the quotes?

A) Wrap the whole CONCAT in `TRIM()`

B) Use `JSON_EXTRACT` instead of `->`

C) Use `->>` instead of `->`

D) Store the series without quotes in the document

Q8

What does the SELECT print?

```
CREATE TABLE shelf (id INT PRIMARY KEY, info JSON);
INSERT INTO shelf VALUES (1, '{"tags": ["new", "signed", "gift"], "copies": 2}
SELECT info->>'$.tags[last]' AS last_tag, JSON_LENGTH(info->'$.tags') AS tag_c
FROM shelf;
```

Q9

A document already has a `format` key. What does `JSON_INSERT(doc, '$.format', 'ebook')` return?

A) The document unchanged

B) The document with format replaced by ebook

C) The document with a second format key

D) An error: the key already exists

Q10

`WHERE attributes->>'$.format' = 'hardcover'` is slow on a table of a million books. What is MySQL's fix?

A) `CREATE INDEX idx_attr ON books (attributes);`

B) A generated column for $.format, with an index on that column

C) Change the column type from JSON to TEXT

D) Write `JSON_EXTRACT` instead of `->>`

Q11

What does this print?

```
SELECT JSON_OBJECT('title', 'Dune', 'id', 1, 'price', 14.99) AS doc;
```

Q12

The store wants to record four things. Which ONE belongs in a JSON column rather than a regular column or table?

A) The customer an order belongs to

B) A book's price

C) The line items of an order

D) An optional gift-wrap note some orders have, with a color and a message

JOURNAL

Think of an app you use. List five pieces of data it stores, and decide for each: regular column, separate table, or JSON? One sentence of reasoning each, using the lesson's "when not to use JSON" table.

EXIT TICKET

Three questions, no notes.

E1

Inside a transaction, an UPDATE fails with error 3819 and you then type COMMIT. What is saved?

E2

Write the handler that makes a procedure undo its transaction on any error and still report the error.

E3

Name one bookstore fact that belongs in JSON and one that must stay a column, with a reason each.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish checkout.sql (Steps 2 to 6) and catalog_feed.sql (Steps 2 to 6)
- ☐ Lesson 7.2 Hands-on Exercises 1 to 3 (Exercise 3 needs two windows; Exercise 4 is a stretch)

- ☐ Lesson 7.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Labs not reached: `return_item_starter.sql`, `debug_lost_points.sql`, `order_documents_starter.sql`, `debug_bad_json.sql`
- ☐ Checkpoint 2: finish any part I did not finish, and bring it
- ☐ Reload `bookstore.sql` before Session 12
- ☐ Bring to Session 12: laptop and root password, two terminal windows, my finished project files, this workbook and my journal

GLOSSARY

TERM	MEANING
transaction	Several statements treated as one unit: all of them are kept (COMMIT) or none (ROLLBACK)
ACID	Atomicity, consistency, isolation, durability: the guarantees a transaction gives
autocommit	The default mode: each statement is its own transaction, committed at once
implicit commit	A commit MySQL does by itself before DDL and account statements
savepoint	A named bookmark inside a transaction; ROLLBACK TO it undoes only the later part
isolation level	What a transaction sees of others' work; MySQL's default is REPEATABLE READ
row lock	A writer's claim on a row; other writers wait (1205 after innodb_lock_wait_timeout)
deadlock	Two transactions waiting for each other; InnoDB rolls one back with 1213
EXIT HANDLER	Code that runs when a statement in a routine fails, then leaves the routine
JSON	A text format of objects, arrays, strings, numbers, true, false and null
path	Where a value sits in a document: $, $.key, $.list[0], $.list[*]
-> / ->>	Extract a value as JSON (with quotes) / as plain text
JSON_TABLE	Turns a JSON array into rows with typed columns
generated column	A column computed from other columns of the row; can be indexed

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 12: Users, Permissions and Backups

Your workbook for Session 12: the warm-up, the key ideas of Lessons 7.4 and 8.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, the final-project brief with your Milestone 1 notes, and your exit ticket.

Lessons 7.4–8.1 | Session 12 · 3 hours | Name: _____

SESSION GOALS

- Create accounts as 'user'@'host', grant and revoke privileges at the database, table and column level, and read SHOW GRANTS.
- Log in as a restricted account and recognize the refusals (1142, 1143, 1044, 3118).
- Bundle privileges into roles and make them active.
- Back up a database with mysqldump, prove the backup by restoring it, and avoid the 9.7 GTID trap.
- Export rows to CSV and import a messy file safely through a staging table.
- Know the final project's client and brief, and start your design.

WARM-UP: TRANSACTIONS AND JSON

FIVE QUESTIONS (LESSONS 7.2 AND 7.3)

W1

`START TRANSACTION;` , an UPDATE, an `ALTER TABLE` , then `ROLLBACK;` . Is the UPDATE undone?

W2

What is the difference between ERROR 1205 and ERROR 1213?

W3

In `WHERE attributes ? '$.format' = 'hardcover'`, which arrow goes where the ? is, and why?

W4

What does JSON_TABLE do, in one sentence?

W5

Predict both columns, then run it.

```
CREATE TABLE notes (id INT PRIMARY KEY, doc JSON);
INSERT INTO notes VALUES (1, '{"qty": 2, "tags": ["gift"]}');
START TRANSACTION;
UPDATE notes SET doc = JSON_SET(doc, '$.qty', 5);
ROLLBACK;
SELECT doc->>'$.qty' AS qty, doc->'$.tags' AS tags FROM notes;
```

Checkpoint 2: my instructor's advice, and the part I will redo:

LESSONS

7.4 User Management & Permissions

BY THE END YOU CAN

- Create accounts as 'user'@'host' and explain how MySQL 9.7 stores their passwords (caching_sha2_password)
- Grant and revoke privileges at the global, database, table and column level, and read SHOW GRANTS
- Log in as a restricted user and recognize the access-denied errors (1142, 1143, 1044, 3118)
- Group privileges into roles, assign them, and make them active with SET DEFAULT ROLE
- Design least-privilege accounts for an application, a report and staff, and script them

KEY IDEAS

- **Least privilege:** every account gets the minimum it needs; nobody shares root, applications never use it.
- **An account is 'user'@'host'.** The same name with another host is another account.
- **New accounts have only USAGE** (may connect). `GRANT what ON where TO whom` adds; `REVOKE ... FROM` removes, at the same level.
- **Levels:** `*.*` global, `db.*` database, `db.table`, columns `SELECT (a, b)`, routines `EXECUTE ON PROCEDURE`.
- **Test both directions** as the account: 1142 (table), 1143 (column), 1044 (database at login), 3118 (locked).
- **Roles** bundle privileges; they work only once active: `SET DEFAULT ROLE ALL TO ...`.
- **ACCOUNT LOCK** disables an account and keeps its grants; DROP USER removes both.
- **caching_sha2_password** is the default; `mysql_native_password` is gone in 9.x (1524): update the driver.

WORKED EXAMPLE: A COLUMN-LEVEL GRANT AND AN AUDIT

An intern may see each book's id, title and price, and nothing else: a column-level grant. Then an audit of what the account really has, read from information_schema (handy when SHOW GRANTS gets long), and the cleanup.

```
CREATE USER 'kit_intern'@'localhost' IDENTIFIED BY 'CHANGE-ME-intern-1';
GRANT SELECT (id, title, price) ON bookstore.books TO 'kit_intern'@'localhost';
SHOW GRANTS FOR 'kit_intern'@'localhost';
```

```
+-----------------------------------------------------------------------------
| Grants for kit_intern@localhost
+-----------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `kit_intern`@`localhost`
| GRANT SELECT (`id`, `price`, `title`) ON `bookstore`.`books` TO `kit_intern`@`l
+-----------------------------------------------------------------------------
```

```
SELECT GRANTEE, TABLE_NAME, COLUMN_NAME, PRIVILEGE_TYPE
FROM information_schema.COLUMN_PRIVILEGES
WHERE GRANTEE = '''kit_intern''@''localhost'''
ORDER BY COLUMN_NAME;
```

```
+--------------------------+------------+-------------+----------------+
| GRANTEE                  | TABLE_NAME | COLUMN_NAME | PRIVILEGE_TYPE |
+--------------------------+------------+-------------+----------------+
| 'kit_intern'@'localhost' | books      | id          | SELECT         |
| 'kit_intern'@'localhost' | books      | price       | SELECT         |
| 'kit_intern'@'localhost' | books      | title       | SELECT         |
+--------------------------+------------+-------------+----------------+
```

```
DROP USER 'kit_intern'@'localhost';
```

Logged in as kit_intern, `SELECT title, price FROM books` would work and `SELECT * FROM books` would fail with 1142 (not every column is granted).

GUIDED LAB

- SOURCE `labs/module-12/l7_4-user-management/warehouse_accounts_starter.sql` as root, with bookstore as the current database.
- TODO 2 together; TODOs 3 and 5 in pairs. TODO 6: test kit_warehouse from a second window (the README has the steps).
- Write the one thing kit_warehouse may change, and the error you saw when it tried something else:

- Only kit_ demo accounts, never root. The file drops its accounts at the end.
- Finished early? `staff_roles_starter.sql`, then `debug_grants.sql`.

PREDICT, THEN RUN

Write your prediction first, then run the code (as root; the date and NULLIF items in a practice database: `CREATE DATABASE scratch; USE scratch;`, then `DROP DATABASE scratch;`).

P1

Predict the output, then run it. (As root on a fresh server, before any lab.)

```
SELECT COUNT(*) AS kit_accounts
FROM mysql.user
WHERE user LIKE 'kit\_%';
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Accounts for the Bookstore Team (lesson page, "Project"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: DROP USER IF EXISTS / DROP ROLE IF EXISTS at the top
- ☐ Step 2: the three roles
- ☐ Step 3: team_web, table by table
- ☐ Step 4: people and the dashboard, with SET DEFAULT ROLE
- ☐ Step 5: the inventory query; Step 6: run twice, test as team_sofia (price yes, title 1143); Dune back to 14.99

SELF-CHECK

Q1

A server has `'app'@'localhost'` and `'app'@'%'`. How many accounts is that?

A) One account that may connect from two places

B) Two separate accounts, each with its own password and privileges

C) One account: % already includes localhost, so the second row is ignored

D) None: the second CREATE USER failed because the name was taken

Q2

How many lines does the SHOW GRANTS print, and what are they?

```
CREATE USER 'kit_quiz'@'localhost' IDENTIFIED BY 'CHANGE-ME-quiz-1';
GRANT SELECT ON bookstore.* TO 'kit_quiz'@'localhost';
GRANT INSERT ON bookstore.orders TO 'kit_quiz'@'localhost';
SHOW GRANTS FOR 'kit_quiz'@'localhost';
```

Q3

An app account gets `ERROR 1143 (42000): SELECT command denied ... for column 'city' in table 'customers'`. What does it mean?

A) The password is wrong

B) The account may not read the customers table at all

C) The account is locked

D) The account has column privileges on customers, and city is not one of them

Q4

Tom and Lena have the role bookstore_sales as a default role. An administrator runs `GRANT INSERT ON bookstore.customers TO 'bookstore_sales';`. When can they insert customers?

A) Right away, both of them, and every future account given the role

B) Only after FLUSH PRIVILEGES

C) Only after each is granted INSERT on customers directly

D) Only accounts created after the GRANT

Q5

`shop_reports` has `GRANT SELECT ON bookstore.*`. What does `REVOKE SELECT ON bookstore.books FROM 'shop_reports'@'localhost';` do?

A) Hides books and keeps every other table readable

B) Removes SELECT on the whole database

C) Fails with ERROR 1147: there is no table-level grant on books to take away

D) Works only after FLUSH PRIVILEGES

Q6

Write the statement that disables `'kit_mailer'@'localhost'` without losing its grants.

JOURNAL

Pick a building you know. Who has a master key, who has a key to one room, who only gets into the lobby? Map each to a MySQL privilege level, and write one thing that would go wrong if everyone had the master key.

8.1 Importing & Exporting Data

BY THE END YOU CAN

- Back up a database with mysqldump, choose the right options for MySQL 9.7, and restore the file into a test database
- Explain what secure_file_priv and local_infile control, and find their values on your server
- Export query results to a CSV file with SELECT ... INTO OUTFILE, including a header row
- Import a CSV file through a staging table with LOAD DATA INFILE, converting dates and empty values on the way in
- Recognize and fix the errors file operations raise (1290, 1086, 3948, 2068, 3546)

KEY IDEAS

- **Whole databases:** mysqldump at the terminal writes SQL; the mysql client restores it. **Rows:** INTO OUTFILE and LOAD DATA at mysql>.
- **On 9.7:** `--single-transaction --routines --events --set-gtid-purged=OFF`; without the last one a same-server restore fails with 3546.
- **A backup counts after a test restore** into another database, with the counts compared and written down.
- **A dump holds no accounts:** keep your CREATE USER and GRANT scripts.
- **Server-side files** live in the secure_file_priv folder (1290 elsewhere, 1086 if the file exists) and need FILE.
- **LOCAL** sends a file from your computer; local_infile must be on at both ends, and errors become warnings: read them.
- **Staging tables:** load as text, convert with `@variables`, `NULLIF`, `STR_TO_DATE`, check with a LEFT JOIN, copy in one transaction.

WORKED EXAMPLE: A HEADER ROW FOR A CSV

INTO OUTFILE writes no column names. Glue a header row on with UNION ALL, and sort with a helper column so the header always comes first. On screen it looks like this; add the INTO OUTFILE clause at the end to write the file.

```
SELECT title, price
FROM (
    SELECT 0 AS part, 'title' AS title, 'price' AS price
    UNION ALL
    SELECT 1, title, price FROM books WHERE price > 12
) AS t
ORDER BY part, title;
```

```
+------------------+-------+
| title            | price |
+------------------+-------+
| title            | price |
| Dune             | 14.99 |
| Lord of the Rings | 15.99 |
+------------------+-------+
```

The header made every column text, so the prices print left-aligned: spreadsheets do not mind.

GUIDED LAB

- SOURCE `labs/module-12/l8_1-importing-exporting/new_titles_import_starter.sql`. Read the warnings: which rows would have been wrong?
- TODO 1 together; TODOs 2 and 3 in pairs. Look at `new_titles.csv` in a text editor: the staging table holds exactly its text.
- Which two rows were held back, and why?

- Reload `labs/_states/bookstore.sql` before you run the finished file a second time.
- Finished early? `backup_check_starter.sql`, then `debug_import.sql`.

PREDICT, THEN RUN

Write your prediction first, then run the code (as root; the date and NULLIF items in a practice database: `CREATE DATABASE scratch; USE scratch;`, then `DROP DATABASE scratch;`).

P2

Predict the output, then run it. (The same text, two patterns.)

```
SELECT STR_TO_DATE('31.12.1999', '%d.%m.%Y') AS eu_date,
       STR_TO_DATE('31.12.1999', '%m/%d/%Y') AS us_pattern;
```

P3

Predict the output, then run it. (Empty strings and NULL.)

```
SELECT NULLIF('', '') AS empty_to_null,
       NULLIF('412', '') AS pages_text,
       COALESCE(NULL, '') AS for_people;
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Bookstore Backup Kit (lesson page, "Project"). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: backup_user.sql (bookstore_backup, five read privileges)
- ☐ Step 2: a dated dump made AS bookstore_backup, no warnings
- ☐ Step 3: test restore into bookstore_test; the result in restore_log.txt
- ☐ Step 4: sales_by_month.csv with a header, checked by loading it back
- ☐ Steps 5 and 6: import_new_books.sql runs twice without duplicates; backup.sh with a login path

SELF-CHECK

Q7

Where do you type mysqldump?

A) At the mysql> prompt, ending with ;

B) In phpMyAdmin's SQL tab

C) At the terminal's shell prompt ($ or PS>), not inside the mysql client

D) Inside a SOURCE file

Q8

Which mysqldump option puts the stored procedures and functions into the backup?

A) `--routines`

B) `--events`

C) `--single-transaction`

D) `--databases`

Q9

What does this print?

```
SELECT STR_TO_DATE('07/04/1976', '%m/%d/%Y') AS published,
       NULLIF('', '') AS pages;
```

Q10

You restore a dump made WITHOUT `--databases` using `mysql -u root -p < bookstore_backup.sql` and get `ERROR 1046 (3D000) at line 22: No database selected`. What is the fix?

A) Take the dump again with `--set-gtid-purged=OFF`

B) Run `SET FOREIGN_KEY_CHECKS = 0` first

C) Use `SOURCE` instead of `<`

D) Name the target database: `mysql -u root -p bookstore_test < bookstore_backup.sql` (or dump with --databases)

Q11

Why load a stranger's CSV into a staging table before touching books?

A) LOAD DATA cannot load into a table that has foreign keys

B) To look at, convert and check the rows (dates, empty values, missing authors) before any real table changes

C) Staging tables load faster than real tables

D) LOAD DATA LOCAL only works with staging tables

Q12

You run `SELECT title FROM books INTO OUTFILE '/var/lib/mysql-files/titles.txt';` a second time. What happens?

A) The file is overwritten with the new rows

B) The rows are added to the end of the file

C) ERROR 1086: the file already exists

D) MySQL writes titles(1).txt

JOURNAL

Think of data you would hate to lose. Where are its copies today, and does that pass the 3-2-1 rule? Then write the bookstore's backup plan in three sentences: what runs, when, where the copies go, and how you know a restore would work.

FINAL PROJECT: THE MAPLE STREET TOOL LIBRARY

THE BRIEF, IN SHORT (READ THE WHOLE BRIEF ON THE LESSON 8.2 PAGE)

- **Client:** Rosa Delgado, coordinator of a volunteer-run library that lends tools instead of books.
- **Her problems today:** one shared spreadsheet; two people borrowed the same ladder on the same day; nobody knows who has the drill; someone without the course took the chainsaw.
- **You hand in:** 01_schema.sql, 02_data.sql, 03_queries.sql, 04_logic.sql, 05_security.sql, a tested backup and a one-page README.md.
- **Assessed:** design, schema and constraints, test data, queries, business logic, security, backup and write-up (100 points; 80 is portfolio-ready).
- **Sessions 13 and 14 are studios:** you build, your instructor circulates. Arrive with your Milestone 1 draft.

MILESTONE 1 DRAFT (HOMEWORK, ON PAPER)

QUESTION	MY NOTES
Which nouns become tables? Their main columns?	
Each relationship: one or many, in both directions? Which needs a junction table?	
Rosa's five rules: where will each be enforced (type or constraint, foreign key, procedure)?	
What will I NOT store because it can be calculated?	

EXIT TICKET

Three questions, no notes.

E1

Why does the website's account get INSERT on orders but no UPDATE or DELETE anywhere?

E2

Name the two steps that turn a dump file into a backup you can trust.

E3

Who is the final project's client, and name two rules her database must enforce.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish team_access.sql (Steps 4 to 6) and the backup kit (Steps 3 to 6)
- ☐ Lesson 7.4 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 8.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Labs not reached: `staff_roles_starter.sql`, `debug_grants.sql`, `backup_check_starter.sql`, `debug_import.sql`
- ☐ Final project: read the whole brief; my Milestone 1 draft (above)
- ☐ Skim Lessons 6.3 and 7.4 again
- ☐ Bring to Session 13: laptop and root password, my Milestone 1 draft, team_access.sql and the backup kit, this workbook and my journal

GLOSSARY

TERM	MEANING
account	A user name and the host it may connect from: 'user'@'host'
privilege	Permission for one kind of statement (SELECT, INSERT, ...) at one level
least privilege	Every account gets the minimum it needs, nothing more
USAGE	No privileges: the account may connect, nothing else
role	A named bundle of privileges granted to accounts; active only after SET DEFAULT ROLE or SET ROLE
ACCOUNT LOCK	Disables a login without deleting the account or its grants (3118 at login)
mysqldump	A terminal program that writes a database as SQL statements (a logical backup)
test restore	Loading a backup into another database and comparing it with the original
GTID	Global transaction IDs used by replication; on in 9.7, hence --set-gtid-purged=OFF
secure_file_priv	The only folder the server reads files from and writes files to
INTO OUTFILE	Writes a query's rows to a file on the server
LOAD DATA [LOCAL] INFILE	Reads a file's rows into a table (LOCAL: the file is on your computer)
staging table	A table that receives raw imported rows for checking before the real tables

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 13: Final Project Kickoff and Studio I

Your workbook for Session 13: the brief in one page, the design table for Milestone 1, the guided lab, a worked constraint test, the milestone tracker, a self-check and your exit ticket.

Lesson 8.2 · Milestones 1–3 | Session 13 · 3 hours | Name: _____

SESSION GOALS

- Turn Rosa's brief into tables, keys and relationships, with each of her eight requirements mapped to the place your design enforces or answers it.
- Build `01_schema.sql`: re-runnable, parents first, every constraint named, and a test statement that each constraint must refuse.
- Build `02_data.sql` with test data that plants the edge cases a broken query would hide.
- Know what goes home (Milestone 4) and what Session 14 adds (logic, security, backup, the showcase).

WARM-UP

Answer from memory, then compare with the class.

W1

The treasurer's account was granted the reporting role, but every SELECT gives ERROR 1142. What was forgotten? *(Lesson 7.4)*

W2

Someone types `mysqldump -u root -p bookstore > backup.sql` at the `mysql>` prompt. What happens? *(Lesson 8.1)*

W3

A dump of one database restores perfectly on a new laptop, but the web form cannot log in. Why? *(Lesson 8.1)*

W4

Which mysqldump option puts the stored procedures into the dump? *(Lesson 8.1)*

W5

From Rosa's brief: name the tables you would build, and one rule that cannot be a CHECK constraint. *(the brief)*

LESSON

8.2 Final Project: Design and Build a Database

BY THE END YOU CAN

- Turn a written brief into entities, relationships and business rules, and draw them as a diagram
- Build a normalized schema in a re-runnable script, with keys, foreign keys and CHECK constraints that you have tested
- Answer business questions with joins, GROUP BY, CTEs and window functions, using a fixed report date
- Enforce rules that span several rows in a stored procedure with a transaction
- Secure the database with roles and least-privilege accounts, then back it up and prove the backup restores

Today: the first two. Session 14: the other three.

THE BRIEF IN ONE PAGE

Rosa Delgado's Maple Street Tool Library (full text on the lesson page, "The Brief"). Her eight requirements, in short:

1. Members: name, email (one account per email), joined date, active or suspended.
2. About a dozen tools: name, one category, condition (good, fair, out for repair), replacement value, date acquired.
3. Safety courses: power tools need Power Tools Basics, the chainsaw needs Chainsaw Safety; who finished which course, and when.
4. A loan: one tool, one member, out, due (normally a week later), back.
5. Rules: one borrower per tool at a time; nothing out while in repair; suspended members cannot borrow; three tools at most; no course, no chainsaw.
6. Every Saturday: what is on the shelf, what is overdue and whom to email, which tools are popular, how busy each month was.
7. Desk volunteers check tools out and in through a web form; the treasurer only reads reports; nobody may delete the loan history.
8. "If the laptop dies, I do not want to start over."

Hand in: `01_schema.sql`, `02_data.sql`, `03_queries.sql`, `04_logic.sql`, `05_security.sql`, a tested backup file and `README.md`. **Timeline:** Milestones 1 to 3 today, 4 at home, 5 to 7 in Session 14, 8 and the README at home; due one week after Session 14.

KEY IDEAS

- **Nouns become tables, verbs become relationships.** Ask "one or many?" in both directions. Many-to-many needs a junction table; this one carries its own fact (`completed_on`).
- **Every rule gets a home.** One value: a type or a constraint (NOT NULL, UNIQUE, ENUM, CHECK). Between tables: a foreign key. Looks at other rows: the procedure that writes, inside a transaction.
- **No derived data.** Overdue, days late, available, times borrowed: compute them in queries.
- **A CHECK must give the same answer forever:** compare columns of the same row, never `CURDATE()` (error 3814).
- **Build scripts are re-runnable:** `DROP DATABASE IF EXISTS` first, parents first, children last, every constraint named.
- **A constraint is proven** when a statement that breaks it is refused. Test inside a transaction and roll it back.
- **Test data is a design tool:** plant the edge cases (suspended, never borrowed, in repair, never lent, a tie, overdue, late) so a broken query shows.
- **SOURCE looks in the folder the client was started in.** Start `mysql` inside your `tool-library` folder so `SOURCE 01_schema.sql` inside `02_data.sql` works.

MILESTONE 1: DESIGN (30 MINUTES IN CLASS)

Fill in where each requirement lives. Your check-in: point at every row and say why.

#	ROSA'S REQUIREMENT	WHERE MY DESIGN ENFORCES OR ANSWERS IT	WHY THERE
1	members' facts		
2	tools' facts		
3	safety courses and completions		
4	a loan's facts		
5	the five borrowing rules		
6	the Saturday questions		
7	desk, treasurer, no deleting history		
8	the dead laptop		

Starter schema checklist (the rubric looks for these):

- ☐ Every table has a primary key; the junction table's key is the pair of foreign keys

- ☐ Every foreign key is named, has the same type as its target, and a deliberate ON DELETE rule (or none = refuse)
- ☐ Required facts NOT NULL; unique values (email, category name) UNIQUE
- ☐ Closed lists: ENUM or a lookup table, and I can say why
- ☐ Money DECIMAL, dates DATE, searched text a sensible VARCHAR
- ☐ At least two CHECKs for single-row rules
- ☐ Indexes on the columns my Saturday queries filter and join on
- ☐ No derived data; snake_case, plural table names, no reserved words

GUIDED LAB: GIFT CARDS ON THE BOOKSTORE

- Reload the bookstore: `SOURCE labs/_states/bookstore.sql`. Then `SOURCE labs/module-13/l8_2-final-project/gift_cards_starter.sql`.
- TODOs 1 to 4: one named constraint each. Save, SOURCE again, watch the constraint list grow to nine.
- Then `debug_gift_card_proofs.sql`: write the error number you expect next to each test FIRST, then run it.
- Copy the test pattern (transaction, one good row, one bad statement per rule, ROLLBACK, a count) into your project.

TEST	RULE IT PROVES	ERROR I EXPECT	ERROR I GOT
A			
B			
C			
D			
E			
F			

WORKED EXAMPLE: A CONSTRAINT TEST BLOCK

The lesson's pattern, run on the reference schema. The parents need one row each first, inside the transaction:

```
START TRANSACTION;

INSERT INTO categories (name) VALUES ('Hand Tools');
INSERT INTO members (first_name, last_name, email, joined_on)
VALUES ('Test', 'Member', 'test@example.org', '2026-01-01');
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Test hammer', 1, 15.00, '2026-01-01');
```

Then one statement per rule; each must be refused:

```
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Free saw', 1, 0, '2026-01-01');
```

```
ERROR 3819 (HY000): Check constraint 'chk_tools_value' is violated.
```

```
INSERT INTO members (first_name, last_name, email, joined_on)
VALUES ('Second', 'Account', 'test@example.org', '2026-02-01');
```

```
ERROR 1062 (23000): Duplicate entry 'test@example.org' for key 'members.uq_member
```

Last, ROLLBACK and a count: nothing stays behind.

```
ROLLBACK;

SELECT COUNT(*) AS members_left FROM members;
```

```
+--------------+
| members_left |
+--------------+
|            0 |
+--------------+
```

◆ PREDICT, THEN RUN

In a practice database (`CREATE DATABASE scratch; USE scratch;`, and `DROP DATABASE scratch;` afterwards). Write your prediction first.

P1

Predict the output, then run it. (a team's group name)

```
CREATE TABLE kit_teams (
    id    INT PRIMARY KEY,
    group VARCHAR(10)
);
```

P2

Predict the output, then run it. (community garden plots)

```
CREATE TABLE plots (
    id          INT PRIMARY KEY,
    gardener_id INT NOT NULL,
    CONSTRAINT fk_plots_gardener FOREIGN KEY (gardener_id) REFERENCES gardener
);
```

MILESTONES 2 AND 3: TRACKER

- ☐ M2: `01_schema.sql` starts with `DROP DATABASE IF EXISTS`; tables parents first; constraints named
- ☐ M2: it runs twice in a row with no ERROR
- ☐ M2: one test per constraint, each refused; ROLLBACK; the count is 0
- ☐ M3: `02_data.sql` starts with `SOURCE 01_schema.sql` and uses explicit ids
- ☐ M3: edge cases planted: suspended · never borrowed · in repair · never lent · a tie · open · overdue · returned late
- ☐ M3: my counts: members ___ categories ___ courses ___ member_courses ___ tools ___ loans ___
- ☐ At home, M4: six Saturday queries with `@today = '2026-06-15'`; two checked by hand: _____

SELF-CHECK

Q1

Rosa's rule "a tool can only be out with one person at a time": where does the reference design enforce it?

A) A `CHECK` constraint on `loans`

B) `UNIQUE (tool_id)` on `loans`

C) In `checkout_tool`, inside a transaction that locks the tool's row

D) An `is_available` column on `tools`, set to 0 on check-out

Q2

In the reference design, which fact belongs in the junction table `member_courses` and nowhere else?

A) The member's email

B) `completed_on`, the date the member finished the course

C) The course's name

D) The member's status

Q3

What does the last statement print?

```
CREATE TABLE riders (id INT PRIMARY KEY, name VARCHAR(20));
CREATE TABLE rides  (rider_id INT, started_on DATE);
INSERT INTO riders VALUES (1, 'Ana'), (2, 'Ben'), (3, 'Cy');
INSERT INTO rides  VALUES (1, '2026-06-03'), (1, '2026-06-09'), (2, '2026-05-2

SELECT r.name, COUNT(x.rider_id) AS june_rides
FROM riders AS r
LEFT JOIN rides AS x ON x.rider_id = r.id
WHERE x.started_on >= '2026-06-01'
GROUP BY r.id, r.name
ORDER BY r.name;
```

Q4

What happens when this runs?

```
CREATE TABLE tools (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE loans (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    tool_id INT UNSIGNED NOT NULL,
    CONSTRAINT fk_loans_tool FOREIGN KEY (tool_id) REFERENCES tools (id)
);
```

A) Both tables are created

B) `ERROR 3780 (HY000): Referencing column 'tool_id' and referenced column 'id' in foreign key constraint 'fk_loans_tool' are incompatible.`

C) `ERROR 1824 (HY000): Failed to open the referenced table 'tools'`

D) `ERROR 1452 (23000): Cannot add or update a child row`

Q5

Why does the reference `01_schema.sql` begin with `DROP DATABASE IF EXISTS tool_library;`?

A) So the script can be run again from scratch and always gives the same result

B) Because MySQL cannot create a table in a database that has data in it

C) To delete the test rows the constraint tests left behind

D) So that the backup in Milestone 7 is smaller

Q6

Which of these should NOT be a column in the tool library's tables?

A) `tools.replacement_value`

B) `loans.due_on`

C) The number of days a loan is overdue

D) `members.joined_on`

Q7

What does the last statement print?

```
CREATE TABLE courses (
    id    INT PRIMARY KEY,
    seats INT NOT NULL,
    CONSTRAINT chk_courses_seats CHECK (seats > 0)
);
START TRANSACTION;
INSERT INTO courses VALUES (1, 12);
INSERT INTO courses VALUES (2, 0);
ROLLBACK;
SELECT COUNT(*) AS courses_left FROM courses;
```

Q8

In which order can `01_schema.sql` create these tables without an error?

A) `loans`, `tools`, `members`, `categories`

B) `categories`, `members`, `tools`, `loans`

C) `tools`, `categories`, `loans`, `members`

D) Any order: MySQL checks foreign keys only when rows are inserted

Q9

Rosa: "Nobody should be able to delete the loan history." How does the reference solution make sure?

A) The loans foreign keys have no ON DELETE rule (refuse), and no account is granted DELETE

B) `ON DELETE CASCADE` on both foreign keys of `loans`

C) A `CHECK (deleted = 0)` constraint on `loans`

D) The desk volunteers are asked not to delete rows

Q10

Your overdue query and your "never borrowed" query both return nothing on your test data. What is the most likely problem?

A) Both queries need ORDER BY

B) `@today` was set to the wrong year

C) MySQL caches empty results

D) The test data has no overdue loan and every member has borrowed something

Q11

The reference uses an ENUM for `tool_condition` but a table for categories. Give the reason in one sentence.

Q12

What happens when this runs?

```
CREATE TABLE gear (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    condition VARCHAR(10)
);
```

A) `ERROR 1064 (42000): You have an error in your SQL syntax ... near 'condition VARCHAR(10)`

B) The table is created with a column named condition

C) `ERROR 3814 (HY000): ... disallowed function`

D) MySQL renames the column to condition_1

JOURNAL

Which of Rosa's requirements was hardest to place in your design, where did you put it, and what would break if you had put it somewhere else? (The lesson's own journal prompt comes after Session 14.)

EXIT TICKET

Three questions, no notes.

E1

Name one of Rosa's rules that must live in a procedure, and say why it cannot be a CHECK.

E2

Your `01_schema.sql` ran once. What must be true for it to run a second time without an error?

E3

Write one test statement that proves a CHECK on your loans table works. What should MySQL answer?

NOTES

HOMEWORK CHECKLIST

- ☐ Milestones 1 to 3 finished (design mapped, schema runs twice with tests, data with edge cases)
- ☐ Milestone 4: `03_queries.sql`, six queries, `@today = '2026-06-15'`, two hand checks written down
- ☐ Lab: `edge_cases_starter.sql` (four queries only edge cases can test)
- ☐ Optional: `debug_build_order.sql`; the self-check above

- ☐ Journal: the hardest requirement to place
- ☐ Read Milestones 5 to 7 on the lesson page; Lesson 7.2's EXIT HANDLER
- ☐ Bring: the tool-library folder that rebuilds everything, design notes, the rubric, a laptop with the bookstore loaded

GLOSSARY

TERM	MEANING
brief	The client's description of what they need, in their words; the starting point of a design
entity	A thing with its own facts (member, tool, loan): becomes a table
junction table	A table that links two others in a many-to-many relationship; its key is the pair of foreign keys
business rule	A rule from the brief ("three tools at most"); it needs a home: a constraint, a foreign key or a procedure
derived data	A value that can be computed from other data (days overdue); computed in queries, not stored
re-runnable script	A script that rebuilds everything from scratch each time it runs (`DROP DATABASE IF EXISTS` first)
constraint test	A statement that breaks a rule on purpose; the constraint is proven when MySQL refuses it
edge case	Test data at the boundary (never borrowed, overdue, a tie) that exposes a broken query
report date	A fixed date in `@today` so results can be checked; production sets it to `CURDATE()`
reserved word	A word SQL itself uses (`condition`, `order`, `group`, `rank`, `key`): not a column name

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 14: Final Project Studio II, Showcase and Next Steps

Your workbook for Session 14: the rebuild check, the Milestone 5 to 7 tracker, a worked example of the reference answering, your showcase plan, Lesson 8.3's key ideas and project, a self-check for each lesson and the final submission checklist.

Lessons 8.2-8.3 | Session 14 · 3 hours | Name: _____

SESSION GOALS

- Write `04_logic.sql`: a view and the check-out and return procedures, every rule refused with its own message inside a transaction.
- Write `05_security.sql`: roles and accounts that are not root and cannot delete; prove it by logging in as the desk account.
- Back up tool_library, restore it into a test database, compare the counts.
- Present your project in three minutes: a design decision, a checked answer, a rule refused live.
- Point an information_schema health check at your own database, write parameterized queries, and choose your next step.

WARM-UP: THE REBUILD CHECK

W1

From a client started in your tool-library folder: `SOURCE 02_data.sql`, then `SOURCE 03_queries.sql`. Which line fails, if any? *(Run your scripts)*

W2

Saturday question 3 uses RANK(). What would ROW_NUMBER() lose? *(Milestone 4)*

W3

Why does every Saturday query read `@today` instead of `CURDATE()`? *(Milestone 4)*

LESSONS

8.2 Final Project: Design and Build a Database

BY THE END YOU CAN

- Turn a written brief into entities, relationships and business rules, and draw them as a diagram
- Build a normalized schema in a re-runnable script, with keys, foreign keys and CHECK constraints that you have tested
- Answer business questions with joins, GROUP BY, CTEs and window functions, using a fixed report date
- Enforce rules that span several rows in a stored procedure with a transaction
- Secure the database with roles and least-privilege accounts, then back it up and prove the backup restores

Today: the last two, and all five in the showcase.

KEY IDEAS

- **The procedure is the only door into loans.** The desk's account gets EXECUTE, not INSERT; the procedure runs with its definer's rights, so every loan passes every rule.
- **Shape:** START TRANSACTION → read and lock (`SELECT ... FOR UPDATE`) → check each rule, `SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '...'` → INSERT → COMMIT.
- **One handler:** `DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;` undoes everything and passes the real message on.
- **Every rule gets a call that must fail**, and one call must succeed.
- **Roles per job, accounts per program or person,** `SET DEFAULT ROLE` , no DELETE anywhere. Test by logging in as the account.
- **A backup is only a backup once it has been restored:** mysqldump with `--single-transaction --routines --events --set-gtid-purged=OFF` , restore into a test database, compare counts. Accounts are not in the dump: `05_security.sql` is their backup.

WORKED EXAMPLE: THE REFERENCE ANSWERING

Lesson 8.2's reference procedure, called with a rule broken each time (real output). Your messages may differ; each rule needs its own.

```
CALL checkout_tool(8, 1, '2026-06-15');   -- Victor is suspended
CALL checkout_tool(6, 7, '2026-06-15');   -- ladder is out with Amara
CALL checkout_tool(2, 8, '2026-06-15');   -- a fourth tool for Kevin
```

```
ERROR 1644 (45000): Member is suspended
ERROR 1644 (45000): Tool is already on loan
ERROR 1644 (45000): Loan limit reached (3 open loans)
```

And the backup test from Milestone 7, after the restore:

```
SELECT
    (SELECT COUNT(*) FROM tool_library.loans)      AS loans_original,
    (SELECT COUNT(*) FROM tool_library_test.loans) AS loans_copy;
```

```
+----------------+------------+
| loans_original | loans_copy |
+----------------+------------+
|             18 |         18 |
+----------------+------------+
```

STUDIO TRACKER: MILESTONES 5 TO 7

- ☐ M5: `v_open_loans` lists the open loans
- ☐ M5: `checkout_tool` refuses: suspended · in repair · already on loan · no course · fourth tool (five messages)
- ☐ M5: one successful call prints the new loan; `return_tool` refuses a tool that is not out
- ☐ M6: roles for the desk and the reports, two accounts, `SET DEFAULT ROLE` for both
- ☐ M6: logged in as the desk account: CALL works, DELETE and INSERT on loans refused (1142)
- ☐ M7: dump taken, restored into tool_library_test; counts ___ and ___; procedures present
- ☐ At home, M8: rebuilt from the files on a clean server; self-review list; README.md; rubric score

GUIDED LAB (IF YOU ARE STUCK ON THE SHAPE)

- Reload the bookstore, then `SOURCE labs/module-14/l8_2-final-project/add_item_rules_starter.sql`: a clerk adds a book to an order; TODOs add the lock, the handler and three rules; the quantity rule becomes a CHECK.
- Then `debug_add_item_rules.sql`: write the message you expect next to each bad call, then run it.
- Milestone 6 practice: `least_privilege_starter.sql` (a till and an owner; the audit at the end must show no DELETE).

SHOWCASE: PLAN YOUR THREE MINUTES

WHAT I SHOW	MY NOTES
One design decision, and why	
One Saturday answer, checked by hand	
One rule refused live (the CALL I will run)	

Strengths and questions I received:

PREDICT, THEN RUN

P1 in a practice database (`CREATE DATABASE scratch; USE scratch;`).

P1

Predict the output, then run it. (a tie for first place)

```
CREATE TABLE tool_loans (tool VARCHAR(20), loans INT);
INSERT INTO tool_loans VALUES ('Hammer', 1), ('Socket set', 1), ('Tile cutter'

SELECT tool, loans,
       RANK()       OVER (ORDER BY loans DESC) AS rnk,
       ROW_NUMBER() OVER (ORDER BY loans DESC, tool) AS row_num
FROM tool_loans
ORDER BY loans DESC, tool;
```

SELF-CHECK

Q1

On the reference tool_library (Milestone 3 data), member 7 is Tessa (active, no safety course) and tool 6 is the chainsaw, on the shelf. What does this print?

```
CALL checkout_tool(7, 6, '2026-06-15');
```

A) A new loan: `loan_id 17, due_on 2026-06-22`

B) `ERROR 1644 (45000): Member is suspended`

C) `ERROR 1644 (45000): Safety course required for this tool`

D) `ERROR 1142 (42000): INSERT command denied`

Q2

Why does `checkout_tool` declare `EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END` ?

A) So a failed check-out never leaves a half-finished transaction, and the caller still receives the real error message

B) So that errors are ignored and the loan is created anyway

C) Because SIGNAL only works inside a handler

D) To make the procedure run with the definer's rights

Q3

Logged in as `tl_desk_app` (the reference roles from Milestone 6), what does this print?

```
DELETE FROM loans WHERE id = 1;
```

Q4

Milestone 7: your dump of tool_library restored into tool_library_test with matching counts. What is NOT in that dump?

A) The tables and their rows

B) The view v_open_loans

C) The two procedures (taken with --routines)

D) The accounts tl_desk_app and tl_treasurer and their roles

Q5

Why does the reference `checkout_tool` read the tool with `SELECT ... FOR UPDATE`?

A) To make the SELECT faster

B) So a second check-out of the same tool waits until the first commits, then sees the new loan

C) Because SELECT INTO needs FOR UPDATE inside a procedure

D) To update the tool's condition

Q6

On the Milestone 3 data, Rosa runs the overdue query (Saturday question 2) with a later report date. How many rows come back, and which loan is first?

```
SET @today = '2026-06-20';
-- then the lesson's query 2: open loans with due_on < @today, worst first
```

JOURNAL (THE LESSON'S PROMPT)

Which design decision did you change your mind about while building, and what made you change it? If you compared your work with the reference, name one thing yours does better and one thing you would borrow from it.

8.3 Next Steps & Best Practices

BY THE END YOU CAN

- Summarize what each module of the course taught and where to look things up again
- Apply a short list of design, query, security and operations practices to any MySQL project
- Audit a database with information_schema queries for missing keys, risky types and old character sets
- Explain SQL injection and write parameterized queries, in SQL and in an application language
- Choose a next learning step that fits your goal

KEY IDEAS

- **Design:** a primary key everywhere; normalize first; DECIMAL for money, DATE for dates, utf8mb4 for text; constraints the database enforces; consistent names.
- **Queries:** name the columns; ORDER BY when order matters; preview with SELECT before UPDATE or DELETE; never switch off strict mode to hide an error; EXPLAIN before tuning.
- **Security:** parameterized queries always; one least-privilege account per application; secrets outside the code; `caching_sha2_password`; supported releases.
- **Operations:** automatic backups with tested restores; the schema in version-controlled scripts; the slow query log; LTS releases (9.7, 8.4), asked for by name.
- **information_schema** describes every database as tables: TABLES, COLUMNS, TABLE_CONSTRAINTS, REFERENTIAL_CONSTRAINTS, KEY_COLUMN_USAGE, STATISTICS.
- **SQL injection:** input glued into SQL can rewrite it. A placeholder (`?`, `%s`, `:name`, `@name`) sends the value separately; at the prompt: `PREPARE ... EXECUTE ... USING`.

WORKED EXAMPLE: ONE VALUE, TWO WAYS

The lesson's demo on the bookstore (real output). Glued in, the crafted email leaks every customer:

```
SET @input = 'nobody@example.com'' OR ''1''=''1';

SET @sql = CONCAT('SELECT first_name, email FROM customers WHERE email = ''', @inp
SELECT @sql AS query_sent;

PREPARE lookup FROM @sql;
EXECUTE lookup;
DEALLOCATE PREPARE lookup;
```

```
+-------------------------------------------------------------------------------
| query_sent
+-------------------------------------------------------------------------------
| SELECT first_name, email FROM customers WHERE email = 'nobody@example.com' OR '
+-------------------------------------------------------------------------------
Statement prepared
+------------+-------------------+
| first_name | email             |
+------------+-------------------+
| Alice      | alice@example.com |
| Bob        | bob@example.com   |
| Carol      | carol@example.com |
| Dave       | dave@example.com  |
| Eve        | eve@example.com   |
| Frank      | frank@example.com |
| Grace      | grace@example.com |
| Hank       | hank@example.com  |
+------------+-------------------+
```

As a parameter, it is only a strange email address:

```
PREPARE lookup FROM 'SELECT first_name, email FROM customers WHERE email = ?';
EXECUTE lookup USING @input;
DEALLOCATE PREPARE lookup;
```

```
Statement prepared
Empty set
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Health Check and Learning Plan (lesson page). In class: Steps 1 to 4. At home: Step 5.

- ☐ Step 1: `health_check.sql` with `SET @db = 'bookstore';` (8 tables)
- ☐ Step 2: checks 1 to 3; on the bookstore, checks 1 and 2 print Empty set
- ☐ Step 3: the unindexed-foreign-key check (Empty set on the bookstore)
- ☐ Step 4: `SET @db = 'tool_library';` (or my project's name): what it reported, and what I fixed: _____
- ☐ Step 5 at home: `learning_plan.md`: a path, a first topic, a small project, a date next to each

GUIDED LAB (FAST FINISHERS AND HOMEWORK)

- `labs/module-14/l8_3-next-steps/health_check_starter.sql`: two checks the lesson does not have, on a planted legacy table.
- `safe_search_starter.sql`: the leak as shipped, then placeholders (TODOs 1 and 2).
- `debug_unsafe_habits.sql`: three changes under safe-update mode; two are refused. Which one gets through, and why?

PREDICT, THEN RUN

P2 on the bookstore.

P2

Predict the output, then run it. (a crafted last name)

```
SET @who = 'x'' OR ''1''=''1';
PREPARE find FROM 'SELECT first_name FROM customers WHERE last_name = ?';
EXECUTE find USING @who;
DEALLOCATE PREPARE find;
```

SELF-CHECK

Q7

What does this print on the bookstore?

```
SET @input = 'Chicago'' OR ''x''=''x';
SET @sql = CONCAT('SELECT first_name FROM customers WHERE city = ''', @input,
PREPARE search FROM @sql;
EXECUTE search;
```

A) `Empty set`

B) Two rows: Bob and Grace

C) `ERROR 1064 (42000): You have an error in your SQL syntax`

D) All eight customers

Q8

Health check 3 lists the bookstore's foreign keys. What happens to an order's rows in order_items when the order is deleted?

A) They are deleted too (fk_items_order is CASCADE)

B) The DELETE is refused (RESTRICT)

C) Their order_id becomes NULL (SET NULL)

D) Nothing: they stay and point at a missing order

Q9

What does the DELETE do?

```
SET SQL_SAFE_UPDATES = 1;
DELETE FROM customers WHERE city = 'Houston';
```

Q10

Why does the "tables without a primary key" check use LEFT JOIN ... WHERE c.CONSTRAINT_NAME IS NULL?

A) Because information_schema does not support inner joins

B) To sort the tables by name

C) It is the "find what is missing" pattern: keep every table, then keep only those that found no PRIMARY KEY row

D) To skip views

Q11

Where should a web application's database password live?

A) In the source code, next to the connection string

B) In an environment variable or a secrets manager, never in the code or the repository

C) In a comment in 05_security.sql, so the team can find it

D) Nowhere: use root without a password on the web server

Q12

A statement has two placeholders, `WHERE a.name = ? AND b.price < ?`. How must the values be given to EXECUTE?

◆ JOURNAL (THE LESSON'S PROMPT)

Look back at your first journal entry from Lesson 1.1. What did you think a database was then, and how would you explain it now? Which lesson changed your thinking the most, and which skill are you proudest of?

EXIT TICKET

Three questions, no notes.

E1

Why can the desk's web form create a loan through `checkout_tool` although its account may not INSERT into loans?

E2

Rewrite `CONCAT('... WHERE email = ''', @input, '''')` safely.

E3

Name your next step from Lesson 8.3's table, and the date you start.

NOTES

HOMEWORK AND FINAL SUBMISSION CHECKLIST

- ☐ Milestones 5 to 7 finished; Milestone 8: rebuilt from the files in order on a clean server, self-review list answered
- ☐ `README.md`: design, rebuild steps, backups (restore date and counts), next steps; rubric score and one improvement
- ☐ Submit the zipped tool-library folder within one week (placeholder passwords only)
- ☐ Lesson 8.2: Exercises 1 to 3, the Quick Quiz, the journal
- ☐ Lesson 8.3: `learning_plan.md` with dates, Exercises 1 and 2, the Quick Quiz, the journal
- ☐ Optional: `safe_search_starter.sql`, `health_check_starter.sql`, `least_privilege_starter.sql`
- ☐ After the course: the first date of my plan in my calendar; my project in a repository

GLOSSARY

TERM	MEANING
SIGNAL	Raises an error with your own message from a stored program (`SQLSTATE '45000'` , error 1644)
EXIT HANDLER	Code that runs when an error happens in a procedure; with ROLLBACK and RESIGNAL it undoes and reports
SELECT ... FOR UPDATE	Reads rows and locks them until the transaction ends, so a second writer waits
definer's rights	A procedure runs with the privileges of the account that defined it (SQL SECURITY DEFINER)
least privilege	Every account gets exactly what its job needs, nothing more
restore test	Loading a backup into a test database and comparing it with the original
information_schema	The read-only database in which MySQL describes every database as tables
SQL injection	Input that changes the meaning of SQL because it was glued into the statement text
parameterized query	SQL with placeholders; the values travel separately and stay values
LTS release	A long-term support version (9.7, 8.4) for production; Innovation releases (26.x) in between

Complete Participant Workbook · all 14 sessions · v1.0 · © 2026 Ray de la Paz